TUGAS AKHIR – EE234899

PENDEKATAN SEMI-SUPERVISED LEARNING UNTUK SISTEM DETEKSI INTRUSI JARINGAN REAL-TIME BERBASIS GNN-AE PADA ARSITEKTUR *STRUCTEMP*

Hafizh Athallah Rinandri

NRP 5022211233

Dosen Pembimbing

Dr.Ir. Achmad Affandi, DEA.

NIP 196510141990021001

Program Studi Sarjana Teknik Elektro

Departemen Teknik Elektro

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

Tahun 2025

TUGAS AKHIR – EE234899

PENDEKATAN SEMI-SUPERVISED LEARNING UNTUK SISTEM DETEKSI INTRUSI JARINGAN REAL-TIME BERBASIS GNN-AE PADA ARSITEKTUR *STRUCTEMP*

Hafizh Athallah Rinandri

NRP 5022211233

Dosen Pembimbing

Dr.Ir. Achmad Affandi, DEA.

NIP 196510141990021001

Program Studi Sarjana Teknik Elektro

Departemen Teknik Elektro

Fakultas Teknologi Elektro dan Informatika Cerdas

Institut Teknologi Sepuluh Nopember

Surabaya

Tahun 2025

FINAL PROJECT – EE234899

SEMI-SUPERVISED LEARNING APPROACH FOR GNN-AE BASED REAL-TIME *NETWORK INTRUSION DETECTION SYSTEM* ON *STRUCTEMP* ARCHITECTURE

Hafizh Athallah Rinandri

NRP 5022211233

Advisor

Dr.Ir. Achmad Affandi, DEA.

NIP 196510141990021001

Study Program Electrical Engineering

Department of Electrical Engineering

Faculty of Intelligent Electrical and Informatics Technology

Institut Teknologi Sepuluh Nopember

Surabaya

2025

LEMBAR PENGESAHAN

PENDEKATAN SEMI-SUPERVISED LEARNING UNTUK SISTEM DETEKSI INTRUSI JARINGAN REAL-TIME BERBASIS GNN-AE PADA ARSITEKTUR *STRUCTEMP*

TUGAS AKHIR

Diajukan untuk memenuhi salah satu syarat

memperoleh gelar Sarjana Teknik pada

Program Studi S-1 Teknik Elektro

Departemen Teknik Elektro

Fakultas Teknologi

Institut Teknologi Sepuluh Nopember

Oleh : Hafizh Athallah Rinandri

NRP. 5022211233

Disetujui oleh Tim Penguji Tugas Akhir :

1.	Dr.Ir. Achmad Affandi, DEA.	Pembimbing
2.	Dr.techn. Prasetiyono Hari Mukti, S.T., M.T.	Penguji
3.	Ahmad Sirojuddin, S.T., M.Sc, Ph.D	Penguji
4.	Prof. Eko Setijadi, S.T., M.T., Ph.D.	Penguji

SURABAYA

Juli, 2025

PERNYATAAN ORISINALITAS

Yang bertanda tangan di bawah ini:

Nama mahasiswa / NRP	:	Hafizh Athallah Rinandri / 5022211233
Program studi	:	Teknik Elektro
Dosen Pembimbing / NIP	:	Dr.Ir. Achmad Affandi, DEA. / 196510141990021001

dengan ini menyatakan bahwa Tugas Akhir dengan judul "Pendekatan *Semi-Supervised Learning* untuk Sistem Deteksi Intrusi Jaringan *Real-Time* Berbasis GNN-AE pada Arsitektur *Structemp*" adalah hasil karya sendiri, bersifat orisinal, dan ditulis dengan mengikuti kaidah penulisan ilmiah.

Bilamana di kemudian hari ditemukan ketidaksesuaian dengan pernyataan ini, maka saya bersedia menerima sanksi sesuai dengan ketentuan yang berlaku di Institut Teknologi Sepuluh Nopember.

Surabaya, 25 Juli 2025

Mengetahui Dosen Pembimbing	Mahasiswa
Dr.Ir. Achmad Affandi, DEA. 196510141990021001	Hafizh Athallah Rinandri 5022211233

ABSTRAK

PENDEKATAN *SEMI-SUPERVISED LEARNING* UNTUK SISTEM DETEKSI INTRUSI JARINGAN REAL-TIME BERBASIS GNN-AE PADA ARSITEKTUR *STRUCTEMP*

Nama Mahasiswa / NRP	**:**	**Hafizh Athallah Rinandri / 5022211233**
Departemen	**:**	**Teknik Elektro**
Dosen Pembimbing	**:**	**Dr.Ir. Achmad Affandi, DEA.**

Abstrak

Penelitian ini mengembangkan sistem deteksi intrusi jaringan berbasis *Graph Neural Network Autoencoder* (GNN-AE) yang terintegrasi dalam arsitektur *Structemp*-GNN dengan pendekatan *semi-supervised learning* untuk mendeteksi anomali pada trafik jaringan secara real-time. Model dirancang menggunakan arsitektur *dual-path encoding* yang memisahkan pemrosesan informasi struktural dan informasi temporal, yang kemudian akan digabungkan melalui fusi representasi laten yang dikontrol bobotnya oleh α . Kedua model lalu digabungkan menggunakan mekanisme fusi yang dikontrol bobotnya oleh β. Evaluasi dilakukan menggunakan enam *dataset* publik, dengan NF-UQ-NIDS-v2 sebagai *dataset* utama dan lima *dataset* tambahan untuk mengukur kemampuan generalisasi model. Hasil eksperimen menunjukkan bahwa model pada *dataset* NF-UQ-NIDS-v2 mencapai akurasi 97,51%, Presisi 99,34%, *recall* 97,51%, F1-score 97,53%, dan rata-rata waktu inferensi 10.07 mikrodetik per *flow*, menandakan efektivitas pendekatan ini dalam mendeteksi serangan jaringan secara adaptif, efisien, dan akurat dalam skenario operasional nyata.

Kata kunci: *Autoencoder*, Deteksi Intrusi, *Graph Neural Network*, Real-Time Detection, *Semi-supervised learning*

ABSTRACT

SEMI-SUPERVISED LEARNING APPROACH FOR GNN-AE BASED REAL-TIME *NETWORK INTRUSION DETECTION SYSTEM* ON *STRUCTEMP* ARCHITECTURE

Student Name / NRP	**:**	**Hafizh Athallah Rinandri / 5022211233**
Department	**:**	**Teknik Elektro**
Advisor	**:**	**Dr.Ir. Achmad Affandi, DEA.**

Abstract

This research develops a *Graph Neural Network Autoencoder* (GNN-AE) based *Network Intrusion Detection System* integrated in *Structemp*-GNN architecture with semi-supervised learning approach to detect anomalies in network trafik in real-time. The model is designed using a dual-path encoding architecture that separates the processing of structural information and temporal information, which will then be combined through the fusion of latent representations controlled by α . The two models are then combined using a weight-controlled fusion mechanism β. The evaluation is conducted using six public *dataset*s, with NF-UQ-NIDS-v2 as the main *dataset* and five additional *dataset*s to measure the generalization ability of the model. Experimental results show that the model on the NF-UQ-NIDS-v2 *dataset* achieves 97.51% accuracy, 99.34% precision, 97.51% recall, 97.53% F1-score, and an average inference time of 10.07 microseconds per flow, signifying the effectiveness of this approach in adaptively, efficiently, and accurately detecting network attacks in real operational scenarios.

Keywords: *Autoencoder*, *Graph Neural Network*, Intrusion Detection, Real-Time Detection, *Semi-supervised learning*

DAFTAR ISI

LEMBAR PENGESAHAN 4
PERNYATAAN ORISINALITAS 6
ABSTRAK 8
ABSTRACT 10
DAFTAR ISI 11
DAFTAR GAMBAR 13
DAFTAR TABEL 15
DAFTAR SIMBOL ..... **Error! Bookmark not defined.**
BAB 1 PENDAHULUAN 17
1.1 Latar Belakang 17
1.2 Rumusan Masalah 18
1.3 Batasan Masalah 19
1.4 Tujuan 19
1.5 Manfaat 19
BAB 2 TINJAUAN PUSTAKA 21
2.1 Keamanan Jaringan 21
2.2 Klasifikasi Serangan Siber 24
2.3 Hasil Penelitian Terdahulu 28
2.4 *Graph Neural Network* (GNN) 29
2.5 *Autoencoder*s 33
2.6 *Network Intrusion Detection System* (NIDS) 37
2.7 PyTorch 38
2.8 *Structemp* 39
2.9 Semi Supervised Learning 40
2.10 *Dataset* 42
2.11 Evaluasi metrik 45
2.12 Bayesian *Hyperparameter* Tuning 47
2.13 *Optimizer* 49
2.14 Fungsi Aktivasi 53
BAB 3 METODOLOGI 59
3.1 Metode yang digunakan 59
3.2 Urutan pelaksanaan penelitian 61
3.2.1 Pemilihan Data Publik 61
3.2.2 Data *Preprocessing* 62

3.2.3 Pengembangan Model Dasar 73
3.2.4 Evaluasi *Baseline* 74
3.2.5 Pengembangan *Autoencoder* berbasis GNN (GNN_AE) 75
3.2.5 Integrasi Model 77
3.2.7 *Hyperparameter Tuning* 79
3.2.8 Evaluasi dan Analisis Hasil Model Usulan 80
3.3 Aristektur Sistem yang Diusulkan 81
3.3.1 Arsitektur Model yang Diimplementasikan 81
3.3.2 Rancangan Sistem untuk Penerapan *Real-Time* 83
3.4 Rancangan Pengujian 84
3.4.1 Setup Pengujian 84
3.4.2 Skenario Pengujian 85
3.4.2 Mekansime Pengujian 87
3.5 Jadwal Pelaksanaan Penelitian 90
BAB 4 Hasil dan Pembahasan 93
4.1 Penelitian Baseline 93
4.1.1 Deskripsi *Structemp*-GNN Sebagai Baseline 93
4.1.2 Hasil Evaluasi *Structemp*-GNN Baseline 93
4.1.3 Analisis Kinerja Baseline 93
4.2 Hasil Tuning model 94
4.2.1 Proses Tuning *Hyperparameter* 94
4.2.2 Search Space *Hyperparameter* 94
4.2.3 Hasil *Hyperparameter* Terbaik 96
4.3 Hasil dan Analisis Model 102
4.3.1 Evaluasi pada *Dataset* Utama 102
4.3.2 Evaluasi Fleksibilitas Arsitektur Model terhadap Variasi *Dataset* 106
4.4 Perbandingan dengan Model Lain 109
BAB 5 Kesimpulan dan Saran 117
5.1 Kesimpulan 117
5.2 Saran 118
DAFTAR PUSTAKA 120
LAMPIRAN 124
BIODATA PENULIS 126

DAFTAR GAMBAR

Gambar 2.1 Model CIA Triad 21
Gambar 2.2 Arsitektur IDS dan IPS 23
Gambar 2.3 Klasifikasi Serangan Siber 25
Gambar 2.4 Gambar Representasi *Graph Neural Network* 30
Gambar 2.5 Gambar *Undercomplete Autoencoder* 35
Gambar 2.6 *Overcomplete Autoencoder* 36
Gambar 2.7 Arsitektur NIDS dan HIDS 38
Gambar 2.8 *PyTorch* 39
Gambar 2.9 Semi-supervised learning 41
Gambar 2.10 Confusion Matrix 46
Gambar 2.11 Gambar *Hyperparameter Optimization* 47
Gambar 2.12 Perbandingan Fungsi Aktivasi 53
Gambar 2.13 Fungsi Aktivasi ReLu 54
Gambar 2.14 Fungsi Aktivasi *Leaky ReLu* 55
Gambar 2.15 Fungsi Aktivasi GeLu 56
Gambar 2.16 Fungsi Aktivasi SiLU 56
Gambar 3.1 Arsitektur Penempatan IDS dalam Jaringan *Enterprise* 59
Gambar 3.2 Gambar *Fishbone Diagram* 60
Gambar 3.3 Diagram Alir Penelitian 61
Gambar 3.4 Diagram Alur Global Data *preprocessing* 63
Gambar 3.5 Diagram Alur Data *Preparation* 64
Gambar 3.6 Diagram Alur Data *Cleaning* 66
Gambar 3.7 Diagram Alur *Feature Engineering* 67
Gambar 3.8 Diagram Alur Data *Transformation* 69
Gambar 3.9 Diagram Alur *Snapshot Generation* 71
Gambar 3.10 Diagram Alur *Graph Construction* 72
Gambar 3.11 Arsitektur Baseline Model 74
Gambar 3.12 Arsitektur Model GNN-AE 76
Gambar 3.13 Proses Integrasi Kedua Model 78
Gambar 3.14 Proses *Hyperparameter Tuning* Menggunakan Metode *Bayesian* 80
Gambar 3.15 Arsitektur sistem yang diimplementasikan 82
Gambar 3.16 Rancangan sistem *real-time* 83
Gambar 3.17 Diagram Alir Skenario Pengujian yang Dilaksanakan dalam Penelitian 86
Gambar 3.18 Diagram Alur mekanisme Evaluasi Model 88
Gambar 3.18 Diagram Alur mekanisme Evaluasi *Hyperparameter Tuning* 90

Gambar 4.1 Best F1 Score berdasarkan Fungsi Aktivasi 97
Gambar 4.2 Best F1 Score berdasarkan *Alpha* 97
Gambar 4.3 Best F1 Score berdasarkan *Beta* 98
Gambar 4.4 Best F1 Score Berdasarkan *Hidden Dim* 98
Gambar 4.5 Best F1 Score Berdasarkan *Latent Dim* 99
Gambar 4.6 Best F1 Score Berdasarkan *Learning Rate* 99
Gambar 4.7 Best F1 Score Berdasarkan *Optimizer* 100
Gambar 4.8 Best F1 Score Berdasarkan *Weight Decay* 101
Gambar 4.9 Perkembangan F1 Score selama Proses Optimasi 101
Gambar 4.10 Confusion Matrix *Dataset* Utama 103
Gambar 4.11 Kurva Loss Pelatihan dan Validasi *Dataset* Utama 104
Gambar 4.12 Evolusi F1-Score Per Epoch 105
Gambar 4.13 Evolusi Precision Per Epoch 105
Gambar 4.14 Evolusi Akurasi Per Epoch 106
Gambar 4.15 Evolusi Recall Per Epoch 106
Gambar 4.16 Perbandingan metrik evaluasi model pada berbagai *dataset*. 108
Gambar 4.17 Perbandingan pada NF-UQ-NIDS-v2 110
Gambar 4.18 Perbandingan pada NF-ToN-IoT-v2 111
Gambar 4.19 Perbandingan pada NF-UNSW-NB15-v2 111
Gambar 4.20 Perbandingan pada NF-ToN-IoT-v3 112
Gambar 4.21 Perbandingan pada UNSW-NB15 113
Gambar 4.22 Perbandingan pada HIKARI 2021 113

DAFTAR TABEL

Tabel 2.1 Klasifikasi *Dataset* 42
Tabel 3.1 Alat dan Bahan 84
Tabel 3.2 Jadwal Pelaksanaan Penelitian 91
Tabel 4.1 Hasil Model Baseline 93
Tabel 4.2 Rentang pencarian *hyperparameter* 95
Tabel 4.3 Hasil *Hyperparameter* Terbaik 96
Tabel 4.4 Hasil Proposed model NF-UQ-NIDS-v2 102
Tabel 4.5 Hasil Proposed model pada *dataset* lain 107
Tabel 4.6 Perbandingan dengan model lain 114

BAB 1 PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi dan komunikasi telah mengubah cara manusia berinteraksi, bekerja, dan mengakses informasi. Di era digital ini, jaringan komputer telah menjadi bagian penting dalam menunjang infrastruktur berbagai sektor vital seperti pendidikan, pemerintahan, layanan publik, dan industri. Penggunaan jaringan internet dan sistem berbasis *cloud* yang semakin meluas memang memberikan kemudahan akses data secara global, tetapi juga memperbesar peluang dieksploitasi oleh pihak-pihak yang tidak bertanggung jawab. Menurut IBM X-*Force Threat Intelligence Index* 2024, vektor serangan paling umum adalah pencurian kredensial (30%) dan *phishing* (30%), diikuti oleh eksploitasi kerentanan sebesar 14% [1]. *Check Point Research* dalam laporan Q1 2025 juga menyebutkan kenaikan 47% rata-rata serangan siber mingguan per organisasi secara global, dengan sektor yang paling banyak diserang adalah pendidikan, pemerintahan, dan telekomunikasi [2]. Selain itu, laporan FBI IC3 2024 mencatat 859.532 insiden kejahatan siber dengan total kerugian mencapai USD 16,6 miliar di Amerika Serikat, meningkat 33% dibandingkan 2023 [3]. Secara global, kerugian akibat kejahatan siber diperkirakan mencapai USD 9,5 triliun pada 2024 dan diproyeksikan melonjak hingga USD 10,5 triliun pada 2025 [4]. Peningkatan baik dari sisi jumlah maupun kompleksitas teknik serangan ini membuat metode pertahanan tradisional menjadi semakin kurang efektif untuk menghadapi ancaman yang dinamis dan kompleks [5].

Dalam menghadapi tantangan ini, sistem deteksi intrusi jaringan atau *Network Intrusion Detection System* (NIDS) menjadi salah satu komponen penting dalam menjaga integritas dan keamanan infrastruktur jaringan. NIDS berfungsi untuk mendeteksi aktivitas yang tidak seperti biasa dengan cara menganalisis trafik data yang terjadi di jaringan komputer secara *real-time*. Terdapat dua pendekatan utama dalam sistem NIDS, yaitu *Signature-Based Detection System* (SIDS) dan *Anomaly-Based Detection System* (ABDS). Pendekatan berbasis *signature* bekerja dengan mencocokkan pola aktivitas terhadap basis data tanda tangan serangan yang telah diketahui, namun lemah dalam mendeteksi serangan baru. Sementara itu, pendekatan berbasis anomali tradisional yang menggunakan metode statistik memiliki keunggulan dalam mendeteksi perilaku yang tidak biasa, namun sering kali menghasilkan tingkat kesalahan deteksi atau *false positive* yang tinggi karena tidak semua anomali merupakan serangan [6]. Keterbatasan dari kedua pendekatan ini mendorong peneliti untuk mengembangkan sistem yang lebih adaptif, cerdas, dan mampu menangani volume serta kompleksitas data yang tinggi secara efisien.

Salah satu solusi yang muncul dalam beberapa tahun terakhir adalah penerapan metode *machine learning* dan *deep learning* pada sistem deteksi intrusi. Khususnya, *Graph Neural Networks* (GNN) telah menarik perhatian besar karena kemampuannya dalam mengolah data berbasis graf, yang secara alami sesuai dengan struktur jaringan komputer. Dalam jaringan komputer, komponen seperti perangkat, alamat IP, dan juga *port* dapat dimodelkan sebagai *node*. Sementara koneksi atau interaksi antar *node* tersebut dapat direpresentasikan sebagai *edge*. GNN dirancang untuk memanfaatkan informasi struktur topologi dan hubungan antar *node* dalam proses pembelajarannya, sehingga mampu mengidentifikasi pola serangan yang bersifat kompleks, termasuk serangan yang dilakukan secara terdistribusi seperti DDoS.

Dibandingkan dengan arsitektur *neural network* konvensional seperti CNN atau RNN yang hanya efektif untuk data yang berbentuk grid atau sekuensial, GNN menunjukkan keunggulan dalam menangani data yang memiliki struktur relasional [7].

Namun demikian, salah satu tantangan dalam GNN terdapat pada kebutuhan data berlabel dalam jumlah yang besar untuk proses pelatihan yang optimal. Dalam jaringan nyata, terutama pada lingkungan jaringan besar, data yang tersedia sebagian besar tidak memiliki label dikarenakan proses anotasi yang mahal dan memakan waktu. Oleh karena itu, pendekatan *semi-supervised learning* menjadi sangat relevan, karena mampu memanfaatkan sebagian kecil data berlabel untuk melatih model dan juga menggunakan data tak berlabel untuk meningkatkan kinerja prediksi. Salah satu metode yang populer dalam pendekatan ini adalah penggunaan *Graph Autoencoder* (GNN-AE), yang menggabungkan kekuatan *autoencoder* untuk pembelajaran representasi laten dengan struktur graf dari GNN. Model ini mampu memetakan data ke dalam ruang laten, dan anomali dapat dideteksi berdasarkan deviasi dari hasil rekonstruksi. Salah satu studi mengusulkan pendekatan berbasis *Adversarially Regularized Graph Autoencoder* (ARGANIDS) yang berhasil meningkatkan akurasi deteksi serangan dengan memanfaatkan regularisasi *adversarial* selama pelatihan, yang menjadikan model lebih *robust* terhadap *noise* dan variasi data [8].

Meskipun pendekatan ini efektif dalam menangkap hubungan struktural antar komponen pada jaringan, selain dari struktur graf, data jaringan juga memiliki dinamika temporal yang penting untuk dianalisis. Banyak serangan yang terjadi dalam pola waktu tertentu atau memiliki korelasi antar kejadian yang berurutan. Oleh karena itu, model deteksi intrusi yang hanya mempertimbangkan struktur topologi jaringan tanpa memperhitungkan aspek temporal berisiko kehilangan informasi penting. Terdapat studi yang memperkenalkan *Structemp*-GNN, sebuah arsitektur yang menggabungkan informasi struktural dan temporal secara bersamaan untuk membangun model deteksi intrusi yang lebih akurat dan adaptif dalam konteks jaringan IoT yang dinamis. Model ini mampu menangkap hubungan struktural dan temporal dalam GNN, serta menunjukkan potensi dalam mendeteksi serangan baru [9].

Penelitian ini berfokus pada pengembangan sistem deteksi intrusi jaringan berbasis GNN dengan pendekatan *semi-supervised*, melalui integrasi model GNN-AE ke dalam arsitektur *Structemp*. Dengan menggabungkan kemampuan representasi laten dari *autoencoder* dan kekuatan struktural-temporal GNN, berdasarkan sistem yang dikembangkan diharapkan dapat mendeteksi berbagai jenis serangan, meningkatkan akurasi deteksi, serta mengurangi ketergantungan terhadap data berlabel. Pendekatan ini diharapkan dapat memberikan kontribusi terhadap pengembangan sistem deteksi intrusi jaringan yang lebih adaptif dan efisien dalam menghadapi serangan yang modern.

1.2 Rumusan Masalah

Berdasarkan latar belakang yang telah diuraikan, maka rumusan masalah dalam penelitian ini adalah sebagai berikut:

1. Bagaimana cara mengintegrasikan pendekatan *semi-supervised learning* menggunakan *Graph Neural Network Autoencoder* (GNN-AE) ke dalam arsitektur *Structemp* yang telah ada?
2. Bagaimana performa *Structemp*-GNN yang telah terintegrasi dengan GNN-AE dibandingkan dengan versi *Structemp*-GNN tanpa integrasi tersebut?

1.3 Batasan Masalah

Dalam merampungkan tugas akhir ini, penulis mengonsep batasan ruang lingkup masalah untuk memberikan arah yang lebih terfokus. Berikut merupakan batasan-batasan masalah yang diberikan:

1. Metrik evaluasi yang digunakan meliputi akurasi, recall, F1-score, dan waktu inferensi.
2. Model yang dikembangkan berbasis *Graph Neural Network* (GNN) dengan integrasi *Autoencoder* (GNN-AE) untuk pendekatan *semi-supervised*.
3. Penelitian hanya menggunakan *dataset* publik, dengan NF-UQ-NIDS-v2 sebagai *dataset* utama dan lima *dataset* tambahan untuk pengujian generalisasi.
4. Arsitektur yang digunakan adalah *Structemp*-GNN, yang memproses informasi struktural dan temporal pada data jaringan.
5. Implementasi dilakukan pada lingkungan GPU NVIDIA dengan dukungan CUDA untuk mempercepat komputasi.
6. Penelitian ini tidak mencakup implementasi pada perangkat keras maupun pengujian langsung di jaringan produksi.

1.4 Tujuan

Penelitian ini bertujuan untuk:

1. Mengintegrasikan *Structemp*-GNN dengan pendekatan *semi-supervised learning* berbasis *Graph Neural Network Autoencoder* (GNN-AE).
2. Mengevaluasi kinerja *Structemp*-GNN yang telah diintegrasikan dengan GNN-AE menggunakan metrik akurasi, presisi, recall, F1-score, dan waktu inferensi.

1.5 Manfaat

Penelitian ini diharapkan memberikan manfaat sebagai berikut:

1. Memberikan pemahaman tentang cara mengintegrasikan pendekatan *semi-supervised learning* berbasis GNN-AE ke dalam arsitektur *Structemp* untuk meningkatkan akurasi deteksi intrusi jaringan..
2. Memberikan wawasan mengenai pemanfaatan data tidak berlabel untuk meningkatkan efisiensi komputasi dan efektivitas sistem deteksi intrusi.

BAB 2 TINJAUAN PUSTAKA

2.1 Keamanan Jaringan

Keamanan jaringan merupakan cabang penting dalam keamanan informasi yang berfokus pada perlindungan infrastruktur jaringan dari akses tanpa izin, penyalahgunaan, dan serangan siber yang berpotensi merusak kerahasiaan, integritas, dan ketersediaan data. Seiring dengan meningkatnya ketergantungan terhadap sistem digital dan internet dalam berbagai sektor, keamanan jaringan menjadi elemen kritis dalam menjamin operasional yang andal dan aman. Sistem keamanan jaringan tidak hanya mencakup perlindungan fisik perangkat keras, tetapi juga mencakup mekanisme logis seperti enkripsi, kontrol akses, *firewall*, serta sistem deteksi dan pencegahan intrusi (IDS/IPS). Keamanan jaringan melibatkan serangkaian tindakan teknis dan administratif yang bertujuan untuk menjaga data sensitif dari ancaman umum seperti *malware*, *phishing*, serangan *distributed denial-of-service* (DDoS), serta potensi ancaman dari dalam organisasi itu sendiri. Implementasi sistem keamanan jaringan yang efektif membutuhkan kombinasi antara teknologi seperti VPN, IDS, dan audit keamanan, serta kepatuhan terhadap standar regulasi seperti GDPR dan HIPAA yang mendefinisikan pedoman perlindungan data dalam jaringan modern[10].

Dalam keamanan jaringan, salah satu kerangka fundamental yang dijadikan dasar dalam perancangan sistem pertahanan informasi adalah CIA *Triad*, yang terdiri dari tiga elemen penting*: Confidentiality, Integrity*, dan *Availability*. Kerangka ini telah digunakan secara luas baik dalam dunia akademik maupun industri sebagai tolok ukur dasar dalam mengevaluasi sejauh mana suatu sistem dapat dianggap aman. CIA *Triad* berfungsi sebagai fondasi konseptual untuk mengembangkan kebijakan, mekanisme teknis, dan prosedur operasional yang mampu mengamankan data dan layanan digital dari berbagai bentuk ancaman siber [11].



Gambar 2.1 Model CIA *Triad*

Prinsip pertama, *Confidentiality*, mengacu pada upaya menjaga agar data tidak dapat diakses oleh pihak yang tidak memiliki wewenang. Dalam lingkungan jaringan, pelanggaran terhadap *confidentiality* dapat berdampak besar, karena dapat menyebabkan kebocoran data pribadi, informasi sensitif perusahaan, hingga strategi bisnis. *Confidentiality* biasanya dijaga dengan menggunakan teknik enkripsi, otentikasi berlapis, pengelolaan hak akses, serta pembatasan segmentasi jaringan yang ketat. Enkripsi dan otorisasi adalah bentuk proteksi

paling umum yang digunakan untuk menjamin *confidentiality*, dan keduanya berperan penting dalam mencegah serangan seperti *sniffing*, *spoofing*, dan *eavesdropping* yang menyerang lapisan komunikasi jaringan [11].

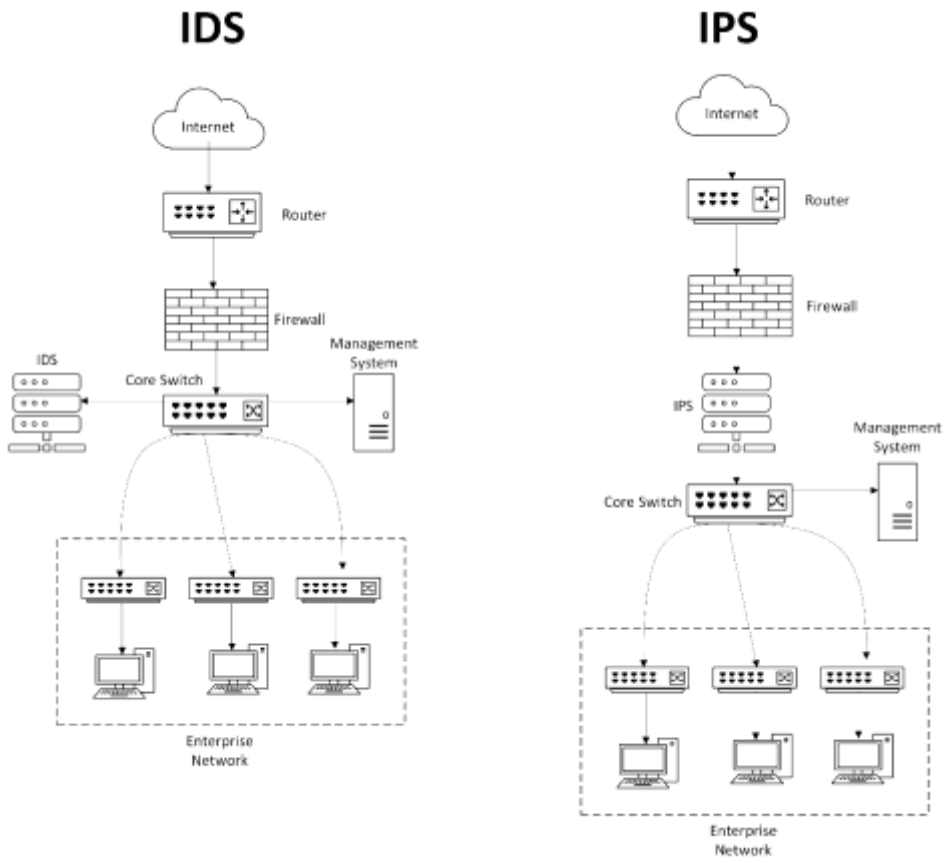
Selanjutnya, prinsip *Integrity* menekankan pada pentingnya menjaga data tetap utuh dan konsisten selama proses penyimpanan maupun transmisi. *Integrity* menjamin bahwa data tidak mengalami modifikasi tanpa izin, baik oleh kesalahan sistem, pengguna internal, maupun penyerang eksternal. Teknik seperti *hashing*, *checksum*, dan *digital signature* merupakan metode utama dalam mendeteksi perubahan yang tidak memiliki izin pada data. Misalnya, jika sebuah *file* log pada sistem IDS dimodifikasi oleh *malware*, mekanisme *hashing* dapat mengindikasikan adanya penyimpangan dan memicu sistem alarm. *Integrity* bukan hanya soal akurasi data, tetapi juga soal membangun kepercayaan digital antara sistem dan penggunanya [11].

Prinsip terakhir, *Availability*, berfokus pada upaya memastikan bahwa layanan jaringan dan data dapat diakses secara stabil oleh pengguna yang sah setiap saat dibutuhkan. *Availability* menjadi sangat krusial, khususnya pada sistem yang bergantung pada waktu nyata, seperti sistem keuangan, transportasi, atau sistem deteksi intrusi. Serangan siber seperti *Distributed Denial of Service* (DDoS) secara langsung menargetkan aspek *availability* dengan membanjiri layanan hingga tidak dapat diakses. Strategi seperti redundansi server, *load balancing*, dan *disaster recovery plan* merupakan pendekatan umum untuk menjaga *availability* dalam sistem jaringan modern [11].

Ketiga elemen dalam CIA *Triad* tidak dapat dipisahkan dengan satu sama lain. Sebuah sistem mungkin memiliki tingkat kerahasiaan yang tinggi, namun tanpa ketersediaan yang baik, maka data tersebut tidak berguna dalam praktik. Demikian pula, jika *integrity* tidak dijaga, maka *confidentiality* pun menjadi tidak berarti karena data yang tersimpan dapat dimanipulasi. Oleh sebab itu, desain sistem keamanan jaringan harus mempertimbangkan ketiga aspek ini secara bersamaan dan seimbang, terutama di era serangan siber yang semakin kompleks dan canggih [11].

Sebagaimana telah dijelaskan sebelumnya, ketiga elemen dalam CIA *Triad confidentiality, integrity*, dan *availability* berperan penting dalam menjaga keamanan dan keandalan sistem jaringan. Namun, dalam praktiknya, menjaga keseimbangan di antara ketiganya menjadi semakin kompleks karena lanskap ancaman siber yang terus berkembang secara dinamis. Serangan yang dulunya bersifat acak kini telah berevolusi menjadi lebih terarah, terstruktur, dan sulit dideteksi. Kondisi ini menuntut sistem keamanan untuk tidak hanya mengandalkan kebijakan statis, tetapi juga mengembangkan kemampuan responsif dan adaptif terhadap perubahan pola ancaman dan infrastruktur jaringan [12].

Dalam konteks ini, dua komponen utama dalam arsitektur pertahanan jaringan adalah *Intrusion Detection System* (IDS) dan *Intrusion Prevention System* (IPS). IDS bekerja secara pasif dengan memantau trafik jaringan untuk mendeteksi pola yang mencurigakan, baik melalui pencocokan *signature* dari serangan yang dikenal, maupun melalui deteksi anomali yang mengidentifikasi deviasi dari *baseline* perilaku normal. *Signature-based* IDS unggul dalam efisiensi dan kecepatan, tetapi memiliki kelemahan mendasar dalam menghadapi *zero-day attacks*. Sebaliknya, *anomaly-based* IDS tradisional bergantung pada model statistik atau untuk mengklasifikasikan perilaku tidak normal, meskipun metode ini menghasilkan tingkat *false positive* yang tinggi [13].



Gambar 2.2 Arsitektur IDS dan IPS

IPS, di sisi lain, berfungsi sebagai sistem aktif yang beroperasi secara *inline* dan dapat langsung mengambil tindakan terhadap trafik mencurigakan. Ketika ancaman terdeteksi, IPS dapat memblokir paket, mengakhiri sesi, atau membatasi komunikasi berdasarkan aturan yang telah ditetapkan. Metode deteksinya dapat berbasis *signature, anomaly*, maupun *stateful protocol analysis*, yang memastikan urutan komunikasi sesuai dengan standar protokol. Meskipun efektif untuk perlindungan *real-time*, IPS menghadapi tantangan dalam menjaga kinerja jaringan karena keterlibatannya langsung dalam jalur komunikasi, sehingga memerlukan algoritma deteksi yang efisien dan sistem dengan latensi rendah [12].

Selain diklasifikasikan berdasarkan metode deteksinya, sistem IDS dan IPS juga dapat dibedakan berdasarkan arsitektur penempatan dan metode pengumpulan datanya. Berdasarkan arsitektur, sistem ini dibagi menjadi beberapa tipe: *Host-based* IDS (HIDS), *Network-based* IDS (NIDS), dan versi gabungan yang menggabungkan keduanya. HIDS beroperasi langsung pada *endpoint* atau *host* (seperti server atau *workstation*) dan memantau aktivitas sistem lokal seperti log *file*, integritas *file* sistem, serta proses dan komunikasi antar aplikasi. Sistem ini sangat efektif dalam mendeteksi serangan yang dilakukan secara lokal atau dari dalam sistem itu sendiri, seperti perubahan file tanpa izin atau aktivitas mencurigakan oleh pengguna internal. Sebaliknya, NIDS bekerja pada level jaringan dan biasanya dipasang di titik-titik strategis seperti *router, gateway*, atau *tap port*. NIDS menganalisis trafik jaringan secara *real-time* dan mampu mendeteksi serangan yang berasal dari luar atau antar *node* dalam jaringan yang diawasi. Kelebihan utama NIDS terletak pada kemampuannya mengawasi banyak perangkat secara bersamaan, sedangkan HIDS menawarkan deteksi yang lebih terperinci namun terbatas pada *host* tertentu [13].

IPS juga memiliki klasifikasi serupa. *Host-based* IPS (HIPS) bekerja dengan prinsip yang mirip dengan HIDS namun tidak hanya mendeteksi, HIPS juga dapat langsung menghentikan proses yang mencurigakan di level OS. *Network-based* IPS (NIPS) ditempatkan *inline* dan memproses setiap paket yang melewati jaringan, memungkinkan sistem untuk menganalisis, mendeteksi, dan memblokir serangan sebelum paket tersebut mencapai *host* tujuan. Selain itu, ada juga *Wireless* IDS/IPS (WIDS/WIPS) yang dirancang khusus untuk memantau jaringan nirkabel dan mendeteksi ancaman seperti *spoofing, rogue access points*, atau serangan *jamming*. [12].

Metode pengumpulan data dalam IDS dan IPS pun bervariasi tergantung arsitekturnya. NIDS/NIPS biasanya menggunakan *port mirroring* (SPAN) atau *network taps* untuk menyalin trafik yang melewati *switch* atau *router* tanpa memengaruhi jalur utama. Sedangkan HIDS/HIPS mengakses data melalui *driver kernel*, audit logs, atau sistem file hooks yang memungkinkan *monitoring* langsung terhadap aktivitas di level *host*. Dalam implementasi skala besar, kombinasi dari berbagai jenis IDS/IPS sering digunakan untuk memberikan cakupan deteksi yang lebih menyeluruh dan mengurangi *blind spot* yang mungkin terjadi jika hanya mengandalkan satu jenis sistem saja [12][13].

Seiring berkembangnya lanskap ancaman siber dan meningkatnya kompleksitas lingkungan jaringan akibat adopsi teknologi seperti IoT, *cloud computing*, dan jaringan 5G, muncul kebutuhan akan sistem deteksi intrusi yang lebih adaptif. Dalam konteks ini, konsep *adaptive* IDS dikembangkan untuk menjawab keterbatasan metode konvensional. *Adaptive* IDS mengandalkan pendekatan berbasis *machine learning* dan *deep learning*, yang memungkinkan sistem untuk belajar dari data trafik jaringan secara berkelanjutan, mengenali pola baru secara mandiri, dan menyesuaikan diri terhadap perubahan perilaku pengguna.

Dengan memanfaatkan analitik perilaku, pembelajaran sekuensial, dan deteksi anomali berbasis data historis maupun kontekstual, *adaptive* IDS dapat memberikan deteksi yang lebih fleksibel, kontekstual, dan proaktif. Sistem ini tidak hanya mendeteksi serangan berdasarkan pola yang telah ditentukan sebelumnya, tetapi juga mampu mengantisipasi serangan yang tidak dikenal dengan mendeteksi deviasi dari perilaku normal. Dengan demikian, *adaptive* IDS menjadi komponen penting dalam membangun arsitektur keamanan jaringan modern yang mampu menghadapi dinamika ancaman secara *real-time* dan berkelanjutan.

2.2 Klasifikasi Serangan Siber

Dalam era digital modern, hampir seluruh aspek kehidupan baik ekonomi, sosial, pemerintahan, maupun infrastruktur vital telah bergeser ke ruang siber. Ketergantungan terhadap teknologi informasi dan komunikasi telah menjadikan ruang siber sebagai medan baru bagi berbagai jenis serangan yang tidak hanya merusak sistem teknis, tetapi juga berpotensi menimbulkan kerugian ekonomi, instabilitas politik, bahkan ancaman terhadap keselamatan publik [14]. Kompleksitas dan sifat tak kasat mata dari serangan siber menyebabkan tantangan besar dalam hal deteksi, mitigasi, dan penegakan hukum, terutama ketika pelaku tidak dapat diidentifikasi secara geografis maupun secara institusional.

Jenis-jenis serangan siber semakin beragam, mulai dari serangan terhadap ketersediaan layanan seperti *Distributed Denial of Service* (DDoS) dan *Denial of Service* (DoS), hingga serangan yang menargetkan kerahasiaan dan integritas data seperti *Injection, Reconnaissance, Password attack*, dan *Cross Site Scripting* (XSS). Ada pula serangan yang bersifat infiltratif

seperti *Backdoor, Bot, Infiltration*, dan *Exploits*, serta bentuk-bentuk serangan lainnya seperti *Scanning*, *Brute Force*, *Generic*, *Fuzzers*, *Shellcode*, *Analysis*, *Man-in-the-Middle* (MITM), *Theft*, *Worm*, dan *Ransomware*. Masing-masing serangan memiliki teknik, vektor, dan dampak yang berbeda-beda.



Gambar 2.3 Klasifikasi Serangan Siber

Untuk menghadapi tantangan ini, pendekatan berbasis *Deep Learning* (DL) dan *machine learning* (ML) semakin banyak diadopsi dalam sistem deteksi intrusi modern. DL dan ML memungkinkan pengenalan pola serangan, prediksi terhadap ancaman baru, serta respons secara *real-time* dengan tingkat akurasi yang lebih tinggi dibandingkan metode tradisional. Oleh karena itu, dibutuhkan kerangka keamanan siber yang adaptif dan mampu merespons dinamika ancaman yang terus berkembang dengan cepat dan tersebar luas di ruang siber. [14].

Distributed Denial of Service (DDoS) adalah serangan siber yang dirancang untuk membuat layanan menjadi tidak tersedia dengan cara membanjiri sistem target menggunakan trafik dari sumber yang terdistribusi. Biasanya serangan ini berasal dari jaringan *botnet* sekumpulan perangkat yang telah terinfeksi *malware* dan dikendalikan oleh pelaku. DDoS tidak hanya berdampak pada pemadaman layanan, tetapi juga dapat merusak reputasi organisasi dan menyebabkan kerugian finansial. Serangan ini umumnya dilakukan dalam skala besar dan sering kali digunakan untuk mengalihkan perhatian dari serangan lain yang lebih tersembunyi seperti infiltrasi data atau *ransomware* [15].

Denial of Service (DoS) memiliki tujuan yang sama dengan DDoS, yaitu menonaktifkan layanan, namun dilakukan dari satu sumber. Meskipun terlihat lebih sederhana, serangan DoS tetap efektif dalam melumpuhkan sistem yang memiliki pertahanan lemah atau tidak memiliki deteksi anomali. Serangan DoS memanfaatkan kelemahan dalam protokol jaringan atau sistem operasi dengan cara mengirimkan permintaan berulang-ulang atau secara tidak normal, hingga sumber daya sistem habis dan layanan menjadi tidak responsif[15]

Injection merupakan salah satu jenis serangan siber yang paling sering ditemukan, terutama dalam aplikasi web yang tidak memiliki validasi *input*. Serangan ini terjadi ketika penyerang memasukan perintah atau kode berbahaya ke dalam *input* dengan tujuan agar sistem target menjalankan perintah tersebut. Jenis *injection* yang paling umum meliputi SQL *Injection*, *Command Injection*, dan Code *Injection*. Masing-masing memiliki target yang berbeda SQL *Injection* menargetkan data base, sementara *Command Injection* dan *Code Injection* dapat menargetkan sistem operasi atau lingkungan *runtime* aplikasi. Dampak dari serangan ini bisa sangat serius, seperti pencurian data, penghapusan informasi penting, hingga pengambilalihan kontrol sistem. Dalam konteks keamanan aplikasi modern, serangan *injection* tetap menjadi salah satu vektor ancaman tertinggi karena eksploitasi sering kali dapat dilakukan hanya melalui manipulasi *form input*, URL, atau parameter API, terutama jika pengembang tidak menerapkan mekanisme sanitasi dan validasi *input* yang benar [15].

Man-in-the-Middle (MITM) adalah serangan siber di mana pelaku menempatkan dirinya di antara dua pihak yang sedang berkomunikasi, dengan tujuan untuk menyadap, mengubah, atau mencuri informasi yang sedang ditransmisikan. Dalam banyak kasus, serangan MITM terjadi melalui jaringan *Wi-Fi* publik yang tidak terenkripsi, atau dengan menggunakan teknik seperti *session hijacking* dan SSL *stripping*. MITM sangat berbahaya karena dapat berlangsung tanpa diketahui kedua belah pihak, dan dapat digunakan untuk mencuri kredensial *login*, data finansial, atau menyisipkan *malware* ke dalam komunikasi. Serangan ini menjadi ancaman serius terutama dalam konteks komunikasi digital yang bergantung pada kepercayaan terhadap kanal komunikasi yang aman[15]

Worm adalah jenis *malware* yang mampu menggandakan dirinya sendiri dan menyebar secara otomatis ke sistem atau perangkat lain melalui jaringan, tanpa memerlukan bantuan pengguna atau *file host*. Berbeda dari virus, *worm* tidak perlu menempel pada *file* tertentu untuk menyebar. *Worm* dapat mengeksploitasi celah keamanan jaringan untuk masuk ke sistem lain dan melanjutkan replikasi. *Worm* dikenal sebagai agen penyebaran cepat yang dapat menyebabkan kemacetan jaringan, kerusakan data, dan sering kali digunakan sebagai pembuka pintu untuk malware lainnya seperti *backdoor* atau *ransomware*. Sifat otonomnya menjadikan *worm* sebagai salah satu bentuk ancaman paling sulit dikendalikan begitu menyebar luas di lingkungan jaringan[15]

Ransomware merupakan jenis serangan siber yang bekerja dengan mengenkripsi *file* atau sistem milik korban, kemudian menuntut pembayaran tebusan agar akses terhadap data tersebut dapat dipulihkan. Mekanisme serangan ini sering diawali melalui penyebaran *email phishing*, eksploitasi kerentanan perangkat lunak, atau penggunaan teknik *social engineering*. Varian *ransomware* modern umumnya menggabungkan teknik enkripsi dengan pencurian data, yang dikenal dengan istilah *double extortion*, di mana pelaku tidak hanya menahan data tetapi juga mengancam untuk mempublikasikan informasi jika tebusan tidak dibayar. Target utama dari serangan *ransomware* umumnya adalah institusi yang sangat bergantung pada ketersediaan data dan layanan, seperti rumah sakit, lembaga pendidikan, serta institusi pemerintahan. Dampak dari serangan ini dapat melumpuhkan seluruh operasi dan menyebabkan kerugian ekonomi yang sangat besar[15]

Bot dalam konteks serangan siber mengacu pada perangkat yang telah diretas dan dapat dikendalikan dari jarak jauh oleh aktor jahat tanpa sepengetahuan pemiliknya. Sekelompok perangkat semacam ini disebut sebagai *botnet*. *Botnet* dikendalikan oleh server pusat,

memungkinkan penyerang melakukan serangan terkoordinasi seperti *Distributed Denial of Service* (DDoS), pengiriman spam, pencurian data, dan penyebaran *malware*. Perangkat dalam *botnet* disebut *zombie* biasanya terinfeksi melalui kerentanan perangkat lunak atau unduhan berbahaya. Karakteristik utama dari serangan bot adalah kemampuannya untuk mengoordinasi banyak perangkat dalam serangan berskala besar yang sulit ditangani oleh mekanisme pertahanan tradisional. *Botnet* sering kali digunakan sebagai bagian dari infrastruktur pendukung dalam banyak serangan siber lainnya seperti *phishing* dan infeksi *ransomware* [15]

Exploit adalah perangkat lunak, data, atau serangkaian perintah yang memanfaatkan kelemahan (*vulnerability*) dalam sistem komputer untuk mendapatkan akses yang tidak sah atau menimbulkan gangguan. Salah satu bentuk *exploit* yang paling berbahaya adalah *zero-day exploit*, yaitu eksploitasi terhadap kerentanan yang belum diketahui atau belum ditambal oleh pengembang perangkat lunak. Serangan jenis ini sangat sulit dideteksi karena tidak ada tanda tangan atau pembaruan keamanan yang tersedia saat serangan terjadi. *Zero-day exploit* digunakan dalam serangan siber yang ditargetkan dan bersifat rahasia, sering kali oleh pelaku dengan sumber daya besar seperti kelompok yang disponsori negara [15].

Cross Site Scripting (XSS) merupakan jenis serangan injeksi pada aplikasi web yang memungkinkan penyerang untuk menyisipkan dan menjalankan skrip berbahaya di dalam halaman web yang dikunjungi oleh pengguna lain. Serangan ini terjadi ketika *input* dari pengguna tidak divalidasi dengan baik, sehingga skrip yang disisipkan dapat dieksekusi oleh browser korban dalam konteks keamanan situs yang sah. Dampak dari serangan XSS meliputi pencurian informasi sensitif seperti *cookie* dan kredensial, pengalihan sesi pengguna, hingga manipulasi antarmuka pengguna. Serangan ini tergolong berbahaya karena tidak secara langsung menyerang server, melainkan mengeksploitasi kepercayaan pengguna terhadap situs yang tampak aman[16]

Password attack adalah jenis serangan yang ditujukan untuk mendapatkan akses tidak sah ke sistem atau layanan dengan cara menebak atau mencuri kata sandi pengguna. Teknik yang digunakan dalam serangan ini yaitu *brute force*, *dictionary attack*, dan *credential stuffing*. Serangan jenis ini memanfaatkan fakta bahwa banyak pengguna menggunakan kata sandi yang lemah atau berulang di berbagai platform. Serangan terhadap kredensial otentikasi seperti kata sandi merupakan salah satu metode paling umum dalam penetrasi jaringan karena keberhasilannya memberikan akses penuh kepada penyerang tanpa perlu eksploitasi teknis lebih lanjut[17].

Infiltration merupakan serangan yang bertujuan menyusup ke dalam jaringan atau sistem dengan cara menyamar sebagai normal trafik atau menggunakan *malware* untuk menghindari deteksi. Serangan ini sering digunakan sebagai langkah awal dalam serangan tingkat lanjut, di mana penyerang berusaha mengakses data atau sistem penting secara tersembunyi. Infiltrasi dikategorikan sebagai serangan yang mengeksploitasi kepercayaan dan celah keamanan dalam sistem, serta menjadi tantangan utama bagi sistem deteksi intrusi karena pola serangannya yang tidak mencolok dan sering kali menyerupai aktivitas normal[17]

Backdoor adalah metode yang memungkinkan penyerang mendapatkan akses ke sistem atau jaringan tanpa melalui prosedur otentikasi normal. *Backdoor* dapat ditanamkan selama pengembangan perangkat lunak oleh pihak internal atau dipasang oleh *malware* setelah sistem dikompromikan. *Backdoor* merupakan ancaman serius karena memberikan akses langsung dan persistensi jangka panjang kepada penyerang, bahkan setelah sistem tampak telah diamankan

atau dipulihkan. Keberadaan *backdoor* sulit dideteksi karena sering kali menyamar sebagai komponen sah dalam sistem[17]

Theft atau pencurian data adalah serangan yang bertujuan untuk memperoleh informasi sensitif dari sistem target, seperti data pribadi, kredensial, atau informasi keuangan. Serangan ini dapat dilakukan secara langsung melalui akses ilegal, atau secara tidak langsung melalui metode seperti *key logging, sniffing*, atau *social engineering*. Dalam konteks keamanan jaringan, pencurian data dapat dikelompokkan sebagai bagian dari kategori serangan yang berorientasi pada kompromi *confidentiality* dari prinsip keamanan informasi. Serangan jenis ini menjadi semakin kompleks karena pelaku sering memanfaatkan enkripsi dan teknik penyamaran untuk menghindari deteksi selama proses ekstraksi data[17]

2.3 Hasil Penelitian Terdahulu

Terdapat sejumlah penelitian terdahulu yang telah mengeksplorasi penggunaan model *Graph Neural Network* (GNN) dalam sistem *Network Intrusion Detection System* (NIDS). Studi-studi ini menjadi landasan penting dalam pengembangan tugas akhir ini karena memberikan wawasan mendalam mengenai pendekatan arsitektur dan pembelajaran yang dapat digunakan dalam mendeteksi intrusi jaringan secara efektif. Dua studi utama yang dijadikan referensi dalam penelitian ini adalah sebagai berikut:

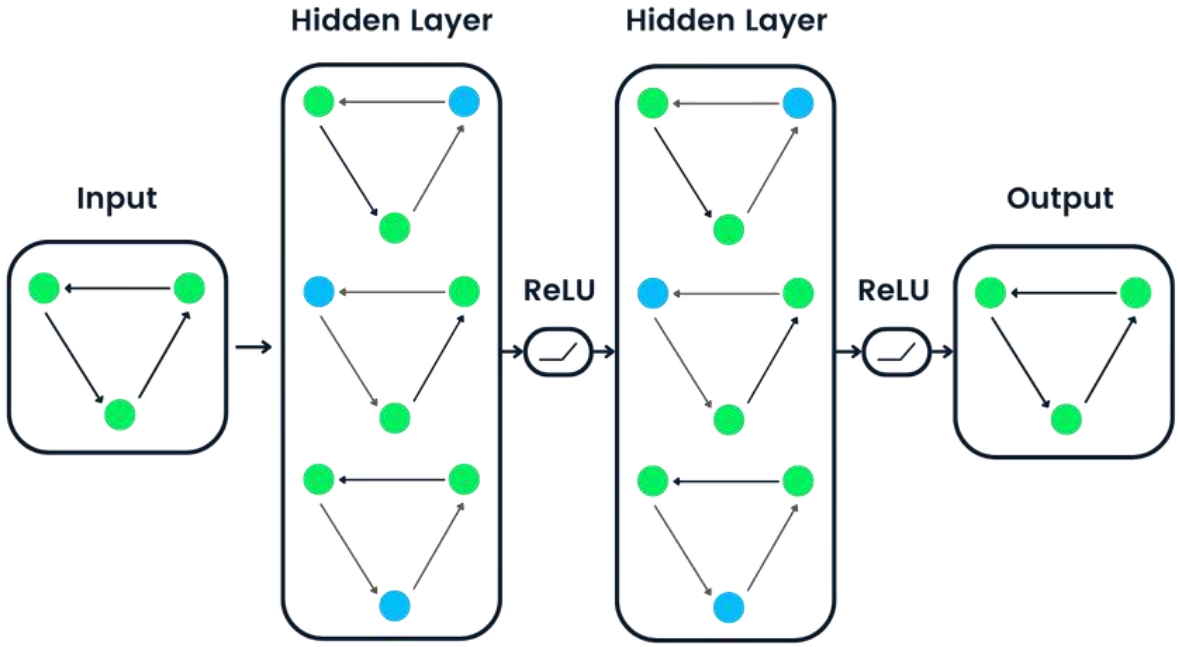
- *Structemp-GNN: An Intrusion Detection Framework in IoT Networks using Dynamic Heterogeneous Graph Neural Networks* yang ditulis oleh Imed Eddine Boukari, Ihab Abderrahmane Derdouha, Samia Bouzefrane, Leila Hamdad, Safia Nait-Bahloul, dan Thomas Huraux. Penelitian ini dipublikasikan pada tahun 2024 dan menawarkan pendekatan inovatif dalam deteksi intrusi dengan memanfaatkan *Dynamic Heterogeneous Graph Neural Network* (DH-GNN) dalam konteks jaringan IoT. Model *Structemp*-GNN yang diusulkan mampu menyesuaikan struktur grafnya secara dinamis terhadap perubahan karakteristik data, sehingga dapat meningkatkan efisiensi komputasi dan mengurangi *overhead* yang umum terjadi pada sistem GNN konvensional. Hasil eksperimen menunjukkan bahwa model ini mampu mendeteksi anomali pada aliran data (*flow*) dengan waktu inferensi yang lebih singkat dibandingkan pendekatan lain yang diuji, sehingga berpotensi digunakan dalam lingkungan yang membutuhkan respons cepat. Meskipun demikian, penelitian ini memiliki beberapa keterbatasan yang perlu diperhatikan, terutama karena fokus pengujiannya berada dalam konteks jaringan IoT yang memiliki karakteristik berbeda secara signifikan dengan jaringan internet umum. Selain itu, pendekatan yang digunakan dalam pelatihan model ini bersifat *supervised learning*, yang memerlukan sejumlah besar data berlabel, menjadikannya kurang ideal dalam skenario dunia nyata di mana data berlabel sering kali sulit didapatkan atau memerlukan biaya anotasi yang tinggi[9].
- *Applying Self-supervised Learning to Network Intrusion Detection for Network Flows with Graph Neural Network*, yang ditulis oleh Renjie Xu, Guangwei Wu, Weiping Wang, Xing Gao, An He, dan Zhengpeng Zhang, juga dipublikasikan pada tahun 2024, memberikan pendekatan berbeda dengan menerapkan *self-supervised learning* untuk membangun sistem NIDS berbasis GNN. Tujuan utama dari penelitian ini adalah mengatasi keterbatasan metode GNN sebelumnya yang masih sangat bergantung pada data berlabel. Untuk itu, mereka mengembangkan model yang terdiri dari *encoder* berbasis *Graph Attention Mechanism* serta strategi *Graph Contrastive Learning*, yang memungkinkan model untuk memahami struktur aliran jaringan tanpa supervisi langsung. Model ini diuji pada empat *dataset NetFlow* yang

berbeda, dan hasil evaluasi menunjukkan bahwa pendekatan ini mampu melakukan klasifikasi multikelas dengan performa yang menjanjikan, bahkan tanpa kehadiran label eksplisit dalam data pelatihan. Pendekatan ini sangat relevan dalam konteks dunia nyata, di mana ketersediaan data berlabel sangat terbatas. Namun, penelitian ini belum memberikan fokus yang cukup terhadap aspek kecepatan deteksi, yang merupakan komponen vital dalam sistem deteksi intrusi *real-time*. Selain itu, walaupun hasil awal menunjukkan potensi besar, metode ini masih memerlukan berbagai penyempurnaan, khususnya dalam hal efisiensi inferensi dan ketahanan terhadap distribusi data yang bervariasi, sebelum dapat diimplementasikan secara luas dalam lingkungan produksi industri yang menuntut stabilitas dan kecepatan tinggi [18].

Dengan mengacu pada dua studi tersebut, penelitian ini mencoba menggabungkan keunggulan pendekatan GNN dinamis seperti pada *Structemp*-GNN dengan efisiensi penggunaan data seperti pada metode *self-supervised learning*. Fokus utama dalam tugas akhir ini adalah membangun sistem deteksi intrusi jaringan *real-time* yang mampu mempertahankan akurasi tinggi dalam mendeteksi anomali meskipun berada dalam kondisi data yang terbatas secara label, serta tetap mampu beroperasi dengan kecepatan deteksi yang optimal.

2.4 *Graph Neural Network* (GNN)

Graph Neural Network (GNN) adalah model *machine learning* yang dirancang untuk memproses data dengan struktur *non-Euclidean*, yakni data yang tidak dapat direpresentasikan dalam bentuk *grid* seperti gambar atau teks. Dalam dunia nyata, banyak fenomena dan sistem yang secara alami membentuk struktur graf, seperti jaringan sosial, jaringan trafik, sistem rekomendasi, dan jaringan komputer. GNN memanfaatkan struktur ini dengan memperlakukan setiap komponen sebagai simpul (*node*) dan hubungan antar komponen sebagai sisi (*edge*), memungkinkan model untuk belajar dari hubungan kompleks antar elemen, bukan hanya fitur individual. Kemampuan ini membedakan GNN dari model *deep learning* konvensional dan menjadikannya sangat cocok untuk menyelesaikan masalah di domain yang membutuhkan pemahaman terhadap keterkaitan data yang kompleks [19].



Gambar 2.4 Gambar Representasi *Graph Neural Network*

Struktur graf yang digunakan dalam GNN dapat dibedakan menjadi beberapa jenis, antara lain *directed* dan *undirected graph*, serta *homogeneous* dan *heterogeneous* graph. *Directed graph* digunakan ketika hubungan antar *node* memiliki arah, seperti dalam kasus pengiriman data dari satu *host* ke *host* lain di jaringan komputer. Sementara itu, *heterogeneous graph* mencakup berbagai tipe *node* dan *edge*, contohnya dalam sistem keamanan jaringan di mana terdapat komponen berbeda seperti *IP address*, *port*, dan aplikasi. Representasi matematis dari graf biasanya terdiri dari *adjacency matrix* untuk menunjukkan konektivitas antar *node*, dan *feature matrix* yang merepresentasikan atribut setiap *node*. Struktur ini menjadi fondasi yang diproses oleh GNN melalui mekanisme pembelajaran representasi [7].

Salah satu kekuatan utama dari GNN adalah kemampuannya dalam melakukan *representation learning* yang mempertimbangkan konteks topologis dari *node*. Ini berarti bahwa representasi sebuah *node* tidak hanya tergantung pada atribut internalnya, tetapi juga pada struktur dan atribut dari *node-node* tetangga di sekitarnya. Proses ini memungkinkan model untuk memahami pola relasional dan hierarki dalam jaringan. Dengan semakin banyaknya lapisan dalam arsitektur GNN, informasi dari *node* yang lebih jauh pun dapat dimanfaatkan. Hal ini memungkinkan GNN untuk mengidentifikasi struktur makro dari graf seperti komunitas atau subnet dalam jaringan komunikasi. Kemampuan ini sangat penting dalam aplikasi keamanan siber yang harus mendeteksi pola interaksi yang tidak biasa atau tersembunyi [5].

Dalam implementasinya, GNN juga memiliki fleksibilitas untuk menangani skenario *semi-supervised learning*, yaitu ketika sebagian besar data tidak memiliki label. Fleksibilitas ini menjadi sangat penting dalam sistem deteksi intrusi, di mana hanya sebagian kecil dari trafik jaringan yang telah diidentifikasi sebagai serangan atau aktivitas normal. Dalam GNN, *node* yang tidak memiliki label dapat tetap memperoleh representasi yang berguna dari informasi struktural di sekitarnya. Dengan demikian, GNN mendukung pembelajaran dari data tidak memiliki label secara efektif, menjadikannya metode yang sangat relevan dalam deteksi anomali atau serangan siber baru yang belum pernah terdeteksi sebelumnya[20].

Namun, seiring dengan manfaatnya yang besar, GNN juga memiliki tantangan teknis yang signifikan. Salah satunya adalah masalah *oversmoothing*, yaitu ketika representasi *node* menjadi terlalu homogen setelah melewati banyak lapisan pembelajaran, yang menyebabkan hilangnya kemampuan model untuk membedakan *node* satu dengan yang lain. Selain itu, GNN juga menghadapi kendala skalabilitas karena kompleksitas komputasi yang tinggi, terutama saat memproses graf besar dengan jutaan *node* dan *edge*. Berbagai teknik telah dikembangkan untuk mengatasi masalah ini, seperti *residual connection*, *mini-batch training* untuk graf, dan strategi *neighbor sampling* yang digunakan dalam model GraphSAGE. Tantangan-tantangan ini mendorong pengembangan arsitektur GNN yang lebih efisien dan adaptif di berbagai domain, termasuk dalam sistem keamanan jaringan[21].

Mekanisme inti dari *Graph Neural Network* (GNN) adalah proses propagasi pesan (*message passing*), yaitu suatu pendekatan iteratif di mana setiap *node* dalam graf memperbarui representasinya dengan cara mengagregasi informasi dari tetangga-tetangganya. Berbeda dengan *neural network* tradisional yang hanya mempertimbangkan fitur lokal suatu titik data, GNN memungkinkan informasi kontekstual dari lingkungan sekitar *node* dipertimbangkan dalam proses pembelajaran. Dengan kata lain, representasi akhir dari sebuah *node* tidak hanya tergantung pada atributnya sendiri, tetapi juga pada struktur dan fitur dari *node-node* lain yang terhubung. Proses ini dilakukan dalam beberapa lapisan, dan pada setiap lapisan, informasi dari tetangga yang semakin jauh dapat dimasukkan, sehingga menghasilkan representasi yang lebih dalam dan menyeluruh [22].

Secara matematis, proses pembaruan representasi fitur pada *node* dalam *Graph Neural Network* (GNN) mengikuti mekanisme propagasi pesan (*message passing*) yang terdiri dari dua langkah utama, yaitu agregasi informasi dari tetangga dan pembaruan representasi *node* itu sendiri. Formulasi umum dari proses ini adalah sebagai berikut

$$\boldsymbol{m_v^{(k)}} = AGGREGATE(\{\boldsymbol{h_u^{(k-1)}}) : u \in N(v)\})$$

$$\boldsymbol{h_v^{(k)}} = UPDATE\left(\boldsymbol{h_v^{(k-1)}}, \boldsymbol{m_v^{(k)}}\right)$$

Di mana *N(v)* merupakan himpunan tetangga dari *node v,* AGGREGATE adalah fungsi yang menyatukan informasi dari *node-node* tetangga, dan UPDATE adalah fungsi yang memproyeksikan hasil agregasi ke dalam ruang representasi baru bersama dengan fitur lama dari *node* tersebut. Fungsi AGGREGATE yang digunakan bisa berupa mean, *sum*, *max*, atau mekanisme *attention*, tergantung pada arsitektur GNN yang digunakan. Formulasi umum ini banyak digunakan dalam berbagai arsitektur GNN modern karena fleksibilitas dan kemampuannya menangkap struktur lokal dari graf[23].

Penggunaan fungsi agregasi yang berbeda akan memberikan pengaruh signifikan terhadap kualitas representasi *node* yang dihasilkan. Sebagai contoh, fungsi mean menghitung rata-rata dari vektor fitur tetangga, bersifat stabil dan efisien, namun dapat kehilangan karakteristik spesifik dari *node* yang menonjol. Sebaliknya, fungsi *max* dapat menangkap fitur dominan namun cenderung rentan terhadap *outlier*. Fungsi sum bersifat akumulatif, tetapi sangat bergantung pada jumlah tetangga dan dapat menyebabkan ketidakseimbangan. Untuk mengatasi keterbatasan tersebut, dikembangkanlah pendekatan berbasis perhatian (*attention-based aggregation*), seperti yang digunakan *pada Graph Attention Network* (GAT). Dalam GAT, setiap *node* tetangga diberikan bobot kontribusi yang dihitung secara dinamis melalui mekanisme *self-attention*, sehingga model dapat menyesuaikan fokusnya terhadap informasi yang paling relevan dalam konteks topologi graf. Pendekatan ini terbukti efektif terutama pada graf yang heterogen dan tidak terstruktur dengan baik [24].

Salah satu pengembangan penting dalam *Graph Neural Network* (GNN) adalah munculnya *Graph Convolutional Network* (GCN), yang dirancang untuk mengadaptasi operasi konvolusi ke dalam domain graf. GCN melakukan propagasi informasi melalui tetangga *node* dengan pendekatan yang efisien menggunakan normalisasi terhadap *adjacency matrix*. Model ini memanfaatkan agregasi lokal, di mana informasi dari tetangga langsung digunakan untuk memperbarui representasi sebuah *node*. Namun, model ini juga menghadapi tantangan, seperti keterbatasan dalam menangani graf dinamis dan masalah *over-smoothing* yang membuat representasi *node* menjadi homogen jika terlalu banyak lapisan digunakan[25].

Sebagai jawaban atas keterbatasan GCN dalam menangani struktur graf yang tidak seragam dan heterogen, dikembangkanlah *Graph Attention Network* (GAT). GAT memperkenalkan mekanisme *self-attention*, yang memungkinkan model untuk secara adaptif menentukan bobot kontribusi dari setiap tetangga berdasarkan kesamaan fitur. Dengan demikian, GAT tidak hanya meningkatkan kemampuan representasi, tetapi juga memberikan fleksibilitas dalam menangani berbagai jenis graf. Meskipun demikian, GAT memiliki kompleksitas komputasi yang lebih tinggi karena memerlukan perhitungan perhatian untuk setiap pasangan *node* bertetangga, terutama dalam graf dengan kepadatan tinggi [24].

Selain GCN dan GAT, varian lain yang menonjol dalam pengembangan *Graph Neural Network* adalah *Graph Isomorphism Network* (GIN). GIN dirancang untuk memiliki kekuatan ekspresif maksimal dalam membedakan struktur graf, mengacu pada kekuatan diskriminatif dari tes isomorfisme *Weisfeiler–Lehman*. GIN menggunakan fungsi agregasi berbasis penjumlahan (sum) yang lebih informatif, serta memasukkan parameter skalar ε yang dapat dilatih untuk menggabungkan fitur *node* itu sendiri dengan fitur tetangganya. Kelebihan ini membuat GIN sangat kuat untuk tugas-tugas klasifikasi graf, seperti dalam analisis data fMRI untuk klasifikasi jenis kelamin berdasarkan konektivitas fungsional otak. Namun, penggunaan fungsi agregasi yang eksplisit menyebabkan tantangan dalam stabilitas pelatihan dan generalisasi, terutama pada graf berskala besar atau dinamis karena tidak mengimplementasikan mekanisme sampling seperti pada GraphSAGE [26].

GraphSAGE merupakan salah satu varian *Graph Neural Network* (GNN) yang dirancang untuk mengatasi tantangan skalabilitas dalam pengolahan graf berskala besar. Tidak seperti GCN yang mengagregasi semua tetangga untuk setiap *node*, GraphSAGE mengadopsi strategi *neighbor sampling* yang hanya memilih sebagian kecil tetangga secara acak pada setiap langkah propagasi. Teknik ini secara signifikan mengurangi beban komputasi dan memungkinkan

efisiensi pelatihan yang lebih tinggi, terutama pada graf dengan jutaan *node*. Selain itu, GraphSAGE menggunakan operasi *concatenation* antara fitur *node* itu sendiri dengan hasil agregasi dari tetangganya. Pendekatan ini memastikan bahwa identitas lokal tetap terpelihara dan tidak hilang dalam informasi tetangga, yang merupakan masalah umum pada model berbasis agregasi penuh [27].

Dalam konteks keamanan jaringan, mekanisme propagasi pesan pada GNN memungkinkan sistem untuk menangkap pola relasional yang tersebar di seluruh jaringan, bukan hanya berdasarkan anomali lokal pada satu *node*. Misalnya, dalam skenario *Distributed Denial of Service* (DDoS), pelaku serangan biasanya menggunakan banyak *node* tersebar untuk membanjiri satu target. Secara individu, setiap *node* mungkin tampak normal, tetapi GNN dapat mengenali pola kolaboratif mereka melalui propagasi informasi struktur komunikasi mereka dalam graf. Hal ini menjadikan GNN unggul dibanding model tradisional yang hanya menganalisis setiap komponen secara terpisah[28].

Dengan mempertimbangkan berbagai kemampuan dan keunggulan arsitektur yang ditawarkan, *Graph Neural Network* (GNN) menjadi pendekatan yang sangat relevan untuk diaplikasikan dalam sistem deteksi intrusi jaringan. Kemampuan GNN dalam menangkap hubungan kompleks antar *node* melalui mekanisme propagasi pesan memungkinkan analisis menyeluruh terhadap struktur komunikasi dalam jaringan, termasuk deteksi pola serangan tersembunyi yang sulit diidentifikasi oleh metode konvensional. Dalam hal ini, GraphSAGE menonjol sebagai varian GNN yang unggul karena mendukung pembelajaran induktif dan efisiensi komputasi melalui teknik sampling tetangga, yang sangat sesuai untuk lingkungan jaringan berskala besar dan dinamis. Kemampuan untuk menggeneralisasi pada *node-node* baru serta adaptasinya terhadap perubahan struktur jaringan menjadikan GraphSAGE solusi yang kuat dalam membangun sistem deteksi intrusi yang responsif dan dapat diskalakan. Dengan demikian, integrasi arsitektur GNN seperti GraphSAGE ke dalam sistem keamanan jaringan memberikan potensi signifikan dalam meningkatkan ketahanan terhadap ancaman siber modern [29].

2.5 *Autoencoder*s

Konsep awal *autoencoder* (AE) pertama kali diperkenalkan pada tahun 1985 oleh Rumelhart sebagai salah satu pendekatan dalam *unsupervised learning* untuk merekonstruksi data *input* melalui proses kompresi dan dekompresi menggunakan *neural network*. Arsitektur dasar dari *autoencoder*, yang kemudian dikenal sebagai vanilla *autoencoder*, terdiri dari tiga komponen utama: *encoder*, ruang laten, dan *decoder*. *Encoder* memetakan data *input* ke dalam representasi berdimensi rendah, sedangkan *decoder* bertugas mengembalikan representasi laten tersebut ke bentuk semirip mungkin dengan data aslinya. Tujuan utama dari pelatihan *autoencoder* adalah meminimalkan perbedaan antara data *input* dan hasil rekonstruksi [30].

Seiring waktu, AE juga menunjukkan kemiripan struktural dan matematis dengan metode lainnya seperti *Principal Component Analysis* (PCA) dan *Restricted Boltzmann Machine* (RBM). Misalnya, AE dengan aktivasi linear dan fungsi kerugian MSE mampu mempelajari subruang yang sama dengan PCA. Namun, AE lebih fleksibel karena mampu memodelkan hubungan non-linear yang tidak bisa ditangkap oleh PCA. Selain itu, AE dan RBM sama-sama merupakan metode *unsupervised learning* yang mengandalkan arsitektur *encoder-decoder*, meskipun RBM lebih bersifat generatif dan memodelkan distribusi probabilistik [31].

Pada tahun-tahun berikutnya, berbagai studi menunjukkan efektivitas AE dalam berbagai bidang seperti pengenalan pola, pengurangan dimensi, deteksi anomali, dan rekonstruksi data. Salah satu pendorong popularitas AE adalah keberhasilannya di awal 2000-an dalam digunakan sebagai metode *pretraining* untuk *deep neural networks*, terutama setelah keberhasilan *deep belief networks* dan pendekatan *greedy layer-wise pretraining* yang melibatkan *stacking autoencoder* sebagai blok penyusun jaringan [31].

Secara matematis, proses *encoding* dan *decoding* dapat dilambangkan sebagai komposisi dua fungsi non-linear, yakni fungsi *encoder* (ϕ) dan fungsi *decoder* (ψ). *Input* (x) diproses oleh *encoder* menjadi representasi laten, kemudian *decoder* menghasilkan *output* rekonstruksi. Selama proses pelatihan, model secara iteratif mengoptimalkan parameter-parameter dari kedua fungsi tersebut untuk meminimalkan perbedaan antara x dan $\hat{x}$, yang biasa disebut sebagai reconstruction error [32].

Untuk mengukur *reconstruction error*, digunakan fungsi kerugian (*loss function*). Dua jenis *loss function* yang paling umum digunakan dalam *autoencoder* adalah *Mean Squared Error* (MSE) dan *Binary Cross-Entropy* (BCE). Pemilihan fungsi *loss* bergantung pada jenis dan distribusi data *input*. MSE digunakan ketika data bersifat kontinu dan numerik, sedangkan BCE digunakan untuk data biner atau data yang telah dinormalisasi dalam rentang 0 sampai dengan 1 [32].

Fungsi *Mean Squared Error* mengukur kesalahan sebagai rata-rata dari kuadrat perbedaan antara nilai *input* dan hasil rekonstruksi. MSE memberikan penalti yang semakin besar terhadap eror yang semakin besar, sehingga model cenderung menghasilkan rekonstruksi yang lebih akurat secara absolut. Rumus matematis MSE untuk *autoencoder* dijelaskan sebagai berikut:

$$L(\phi, \psi) = \left(\frac{1}{n}\right) \sum_{i=1}^{n} \left\| x^{(\mathrm{i})} - \psi\left(\phi\left(x^{(\mathrm{i})}\right)\right) \right\|^2$$

Dalam rumus tersebut, n adalah jumlah data dalam satu *batch*, $x^{(\mathrm{i})}$ adalah data *input* ke-i, dan $\psi\left(\phi\left(x^{(\mathrm{i})}\right)\right)$ adalah hasil rekonstruksi. Fungsi ini menghasilkan nilai *loss* yang kecil jika *output* sangat mirip dengan *input*, menandakan bahwa proses *encoding* dan *decoding* berhasil dengan baik [32].

Sementara itu, *Binary Cross-Entropy* (BCE) lebih sesuai digunakan pada data biner atau data yang memodelkan probabilitas. BCE mengukur perbedaan antara distribusi *input* dengan hasil rekonstruksi dalam konteks probabilistik. Fungsi ini sangat sering digunakan pada *autoencoder* dengan *output* berbasis sigmoid. Berikut adalah rumus fungsi BCE

$$L(\phi, \psi) = -\frac{1}{n} \sum_{i=1}^{n} \left[x^{(i)} \log\left(\psi\left(\phi\left(x^{(i)}\right)\right)\right) + \left(1 - x^{(i)}\right) \log\left(1 - \psi\left(\phi\left(x^{(i)}\right)\right)\right) \right]$$

Fungsi ini memberikan penalti yang tinggi apabila hasil prediksi probabilistik sangat jauh dari nilai *input* sebenarnya, dan karenanya sangat efektif untuk pelatihan model dalam domain klasifikasi atau citra biner [32].

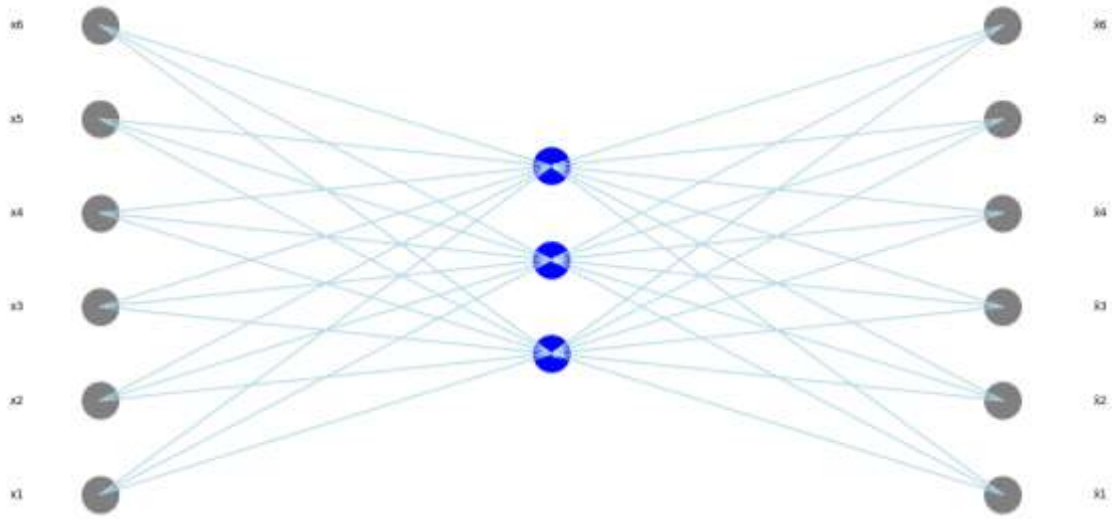
Optimasi terhadap *loss function* dilakukan melalui algoritma seperti *Stochastic Gradient Descent* (SGD). SGD memperbarui parameter jaringan berdasarkan turunan gradien dari fungsi *loss* terhadap parameter model. Proses ini dilakukan dalam iterasi yang melibatkan *minibatch*

dari data pelatihan, memungkinkan model belajar secara efisien dan bertahap. SGD juga dapat dipadukan dengan teknik seperti momentum atau algoritma adaptif seperti Adam untuk meningkatkan konvergensi dan kestabilan pelatihan [32].

Kelebihan *autoencoder* tidak hanya terbatas pada kemampuan rekonstruksi data, tetapi juga pada potensi representasi laten yang dihasilkannya. Representasi ini dapat digunakan untuk tugas-tugas lanjutan seperti klasifikasi, klaster, visualisasi data, hingga deteksi anomali. Dalam banyak studi, *autoencoder* telah terbukti lebih mampu menangkap pola non linear dibandingkan metode linier seperti PCA, karena arsitekturnya yang fleksibel dan non linear. Oleh karena itu, *autoencoder* menjadi komponen penting dalam sistem pembelajaran representasi modern, terutama pada situasi di mana data tidak berlabel dan struktur internalnya kompleks [32].

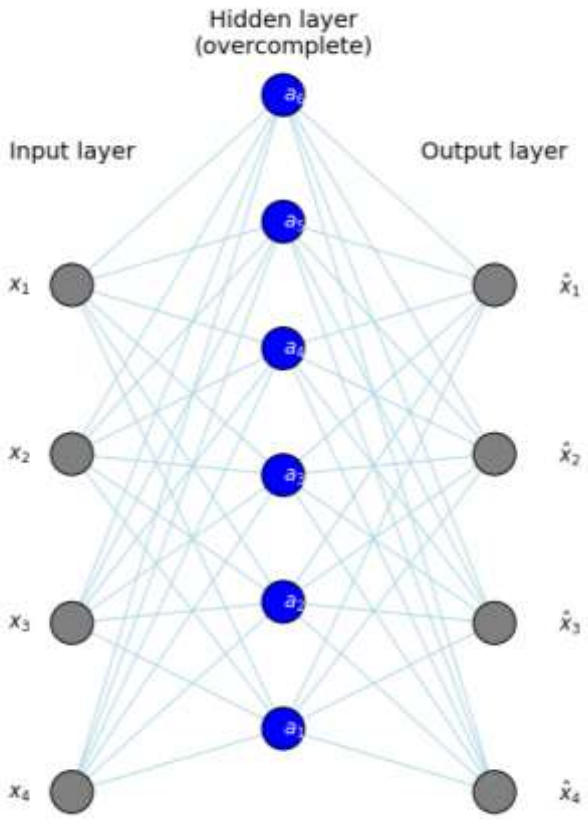
Autoencoder juga telah mengalami berbagai perkembangan dan melahirkan beragam varian seperti *Variational Autoencoder* (VAE), *Denoising Autoencoder* (DAE), *Sparse Autoencoder* (SAE), hingga *Adversarial Autoencoder* (AAE), yang dirancang untuk memenuhi kebutuhan spesifik dalam representasi data. Meski demikian, secara umum *autoencoder* dapat diklasifikasikan berdasarkan ukuran ruang latennya terhadap dimensi *input*, yaitu menjadi *undercomplete* dan *overcomplete autoencoder* [32].

Undercomplete autoencoder adalah arsitektur di mana jumlah neuron pada lapisan tersembunyi lebih sedikit dibandingkan jumlah fitur pada *input* ($p<d$). Tujuan dari desain ini adalah untuk memaksa model melakukan kompresi terhadap data *input*, sehingga hanya informasi paling esensial yang dapat ditransmisikan melalui representasi laten. Hal ini mendorong model untuk menemukan struktur tersembunyi dalam data tanpa hanya menyalin *input* ke *output* secara langsung. *Undercomplete autoencoder* yang menggunakan fungsi aktivasi linier pada konfigurasi *shallow* secara teoritis akan menghasilkan representasi yang identik dengan hasil dari *Principal Component Analysis* (PCA). Namun, ketika fungsi aktivasi non linear seperti ReLU atau sigmoid diterapkan, model dapat menangkap fitur non linear dari data yang tidak dapat dijangkau oleh metode linear seperti PCA. Oleh karena itu, *autoencoder* tipe ini sangat berguna dalam berbagai tugas, seperti reduksi dimensi, visualisasi fitur laten, dan deteksi anomali di mana data diperkirakan memiliki representasi tersembunyi yang lebih sederhana daripada bentuk aslinya [33].



Gambar 2.5 Gambar *Undercomplete Autoencoder*

Di sisi lain, *overcomplete autoencoder* memiliki dimensi ruang laten yang lebih besar atau sama dengan dimensi *input* ($p \geq d$). Pada pandangan awal, pendekatan ini mungkin terlihat lebih fleksibel karena memberikan model kapasitas yang lebih besar untuk merepresentasikan variasi dalam data. Namun, peningkatan kapasitas ini dapat menjadi pedang bermata dua. Jika tidak diimbangi dengan mekanisme regularisasi yang memadai, model *overcomplete* cenderung belajar solusi dangkal berupa *identity mapping*, yaitu hanya menyalin *input* ke output tanpa mempelajari fitur laten yang bermakna. Hal ini menyebabkan representasi yang dihasilkan menjadi kurang informatif dan tidak berguna untuk generalisasi ke data baru[33].



Gambar 2.6 *Overcomplete Autoencoder*

Untuk menghindari hal tersebut, pentingnya regularisasi eksplisit dalam desain *overcomplete autoencoder*. Beberapa pendekatan yang umum digunakan antara lain adalah *sparsity constraint* (*sparse autoencoder*), penalti kontraktif (*contractive autoencoder*), serta penggunaan *input* yang korup (*denoising autoencoder*). Dalam *sparse autoencoder*, misalnya, regularisasi tambahan pada fungsi loss digunakan untuk memaksa hanya sebagian kecil neuron pada ruang laten yang aktif. Dengan cara ini, meskipun ruang laten besar, hanya informasi penting yang disimpan secara efisien, mengurangi kecenderungan *overfitting* dan meningkatkan generalisasi model. Pendekatan ini memungkinkan *overcomplete autoencoder* digunakan secara efektif dalam aplikasi generatif, ekstraksi fitur kompleks, dan transfer *learning*, asalkan dirancang dengan kontrol regularisasi yang kuat [33].

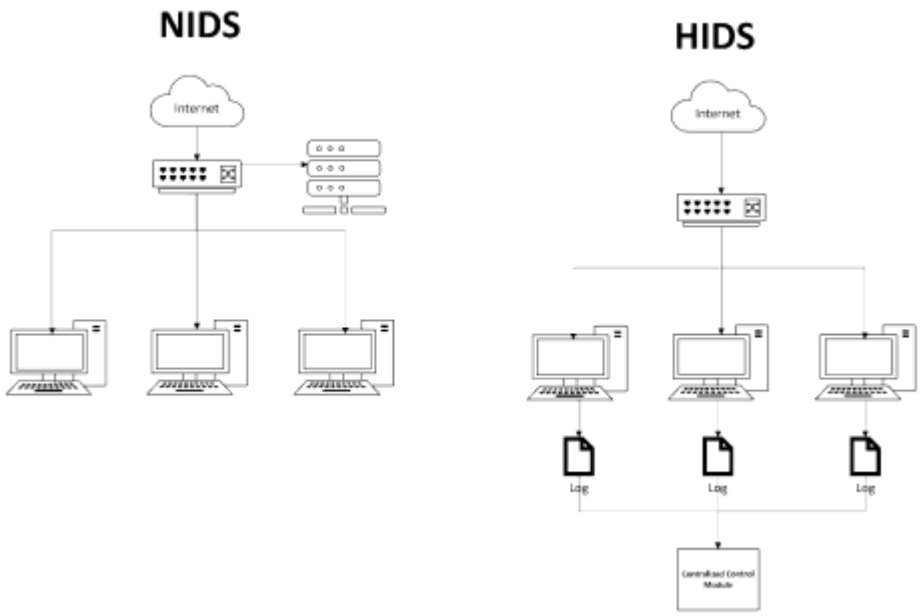
Dengan demikian, perbedaan antara *undercomplete* dan *overcomplete autoencoder* tidak hanya bersifat struktural, tetapi juga berkaitan erat dengan filosofi pembelajaran yang ingin

dicapai. *Undercomplete* cenderung ideal untuk pembelajaran kompresi dan generalisasi dalam kondisi data terbatas, sementara *overcomplete* dapat digunakan untuk representasi yang lebih kaya dan fleksibel selama regularisasi diterapkan secara ketat. Pemilihan antara kedua jenis ini harus mempertimbangkan tujuan akhir dari sistem, ketersediaan data, serta kebutuhan akan interpretabilitas dan efisiensi dalam pembelajaran.

Secara keseluruhan, *autoencoder* telah berkembang menjadi kerangka kerja penting dalam *unsupervised learning*. Kemampuannya dalam menangkap struktur data, baik secara linear maupun non linear, menjadikannya sangat berguna dalam berbagai tugas analitik, dari ekstraksi fitur hingga deteksi anomali. Ketika dikombinasikan dengan pendekatan lain seperti pembelajaran *semi-supervised* dan arsitektur berbasis *graph* seperti *Graph Neural Network* (GNN), *autoencoder* dapat diperluas untuk menangani data berskala besar dan dinamis, seperti yang ditemukan dalam konteks keamanan jaringan. Oleh karena itu, bagian selanjutnya akan membahas pendekatan *semi-supervised learning* yang memperkuat kapabilitas generalisasi model dalam lingkungan dengan ketersediaan label terbatas.

2.6 *Network Intrusion Detection System* (NIDS)

Network Intrusion Detection System (NIDS) adalah komponen penting dalam infrastruktur keamanan jaringan yang dirancang untuk memantau, menganalisis, dan mendeteksi trafik jaringan secara *real-time* guna mengidentifikasi aktivitas yang mencurigakan atau indikasi adanya serangan siber. Sistem ini bekerja dengan cara menginspeksi paket data yang melewati jaringan untuk mencari pola, *signature*, atau anomali yang menunjukkan adanya potensi ancaman, seperti upaya penyusupan, pemindaian *port*, atau transmisi data yang tidak sah. Tidak seperti *Host-based Intrusion Detection* System (HIDS) yang beroperasi langsung pada perangkat *endpoint* dan hanya memantau aktivitas lokal pada perangkat tersebut, NIDS bekerja pada level jaringan dan ditempatkan di titik-titik strategis seperti *gateway*, *router* utama, atau *switch* jaringan. Penempatan ini memungkinkan NIDS untuk mengamati dan menganalisis trafik data dari berbagai *host* sekaligus, sehingga cocok digunakan dalam lingkungan jaringan yang besar dan kompleks, seperti jaringan *enterprise* atau *data center*. NIDS dapat diimplementasikan menggunakan pendekatan berbasis *signature* maupun *anomaly-based*. Sistem berbasis *signature* menggunakan basis data berisi pola-pola serangan yang sudah dikenal, sedangkan sistem berbasis *anomaly* mendeteksi penyimpangan dari perilaku normal jaringan, sehingga lebih mampu mengenali serangan baru atau yang belum terdokumentasi. Beberapa NIDS modern bahkan menggabungkan kedua pendekatan ini dan didukung oleh teknologi *machine learning* untuk meningkatkan akurasi deteksi serta mengurangi *false positive* dan *false negative*. Keunggulan utama NIDS terletak pada kemampuannya melakukan deteksi secara pasif dan tidak mengganggu jalannya komunikasi jaringan, serta skalabilitasnya dalam mengawasi berbagai segmen jaringan secara simultan. Namun, tantangan yang dihadapi mencakup volume data yang besar, kebutuhan akan kecepatan analisis yang tinggi, serta kemampuan dalam menghadapi serangan yang terenkripsi atau bersifat *polymorphic*. Oleh karena itu, pengembangan NIDS yang adaptif, efisien, dan berbasis kecerdasan buatan menjadi fokus utama dalam penelitian keamanan jaringan saat ini [13].



Gambar 2.7 Arsitektur NIDS dan HIDS

Untuk mengatasi keterbatasan tersebut, dikembangkanlah pendekatan *adaptive* IDS, yaitu sistem deteksi intrusi yang mampu beradaptasi secara dinamis terhadap perubahan pola trafik dan perilaku pengguna. *Adaptive* IDS memanfaatkan *machine learning* dan *deep learning* untuk membangun model perilaku jaringan secara berkelanjutan, sehingga mampu mengenali serangan yang belum pernah terlihat sebelumnya. Dengan dukungan teknik seperti pembelajaran sekuensial, analisis perilaku, serta deteksi anomali berbasis data kontekstual, *adaptive* IDS dirancang untuk meningkatkan akurasi deteksi, mengurangi *false positive*, dan memberikan respons yang lebih fleksibel terhadap ancaman siber[13].

Dalam konteks jaringan modern yang bersifat dinamis dan terdistribusi, NIDS memiliki keunggulan dalam hal cakupan luas, tidak invasif, dan efisiensi pemantauan terhadap trafik antar *node*. Oleh karena itu, NIDS sering digunakan dalam infrastruktur skala besar seperti pusat data, jaringan perusahaan, serta sistem IoT. Dalam penelitian ini, fokus diberikan pada NIDS karena sifat datanya yang berbasis *network flow*, seperti pada *dataset* NF-UQ-NIDS-v2, yang secara struktural cocok untuk dianalisis menggunakan pendekatan *Graph Neural Network* (GNN). Pendekatan ini memungkinkan deteksi serangan tidak hanya berdasarkan pola lokal, tetapi juga berdasarkan relasi dan dinamika antar komponen dalam jaringan, yang menjadi elemen krusial dalam mendeteksi serangan yang tersebar[13].

2.7 PyTorch

PyTorch adalah *framework deep learning* yang dikembangkan oleh *Facebook AI Research* (FAIR), pertama kali diperkenalkan pada tahun 2017 setelah evolusi dari *framework* sebelumnya, Torch. PyTorch menonjol dengan sistem *autograd* yang kuat, yang memungkinkan diferensiasi otomatis dari graf komputasi, sangat memudahkan pengguna dalam membangun dan melatih model *deep learning*. PyTorch *framework deep learning* yang dioptimalkan berdasarkan *Python* dan *Torch*. *Framework* ini utamanya digunakan untuk aplikasi yang menggunakan GPU dan CPU [34].



Gambar 2.8 *PyTorch*

PyTorch bekerja dengan memanfaatkan dua komponen inti, yaitu tensor dan grafik komputasi. Tensor adalah tipe data dasar dalam PyTorch yang mirip dengan *array* multidimensi. Tensor digunakan untuk menyimpan dan memanipulasi *input* dan output model serta parameter model. Tensor memiliki kemiripan dengan *array* dalam NumPy, namun dengan kelebihan bahwa tensor dapat dijalankan di GPU untuk mempercepat komputasi, sehingga memungkinkan pengolahan data yang lebih efisien [34].

Grafik komputasi mengacu pada representasi *neural network* sebagai struktur grafik perhitungan. Dalam PyTorch, grafik komputasi ini bersifat dinamis, artinya grafik dibangun kembali pada saat *runtime* menggunakan kode yang sama yang digunakan saat melakukan komputasi untuk *forward pass*. Hal ini sangat penting untuk pembuatan struktur data yang diperlukan dalam *backpropagation*. Dengan grafik komputasi dinamis, PyTorch memungkinkan fleksibilitas yang lebih besar dalam mendesain dan melatih model *neural network*. Selain itu, PyTorch mendukung pemrograman dinamis, yang memungkinkan pengguna untuk mengubah model mereka secara *real-time* dan menyesuaikannya dengan data yang berbeda. Fitur ini sangat berguna dalam pengembangan model *Deep learning*, karena memungkinkan eksperimen dan iterasi yang cepat. Secara keseluruhan, kombinasi tensor dan grafik komputasi dinamis dalam PyTorch memberikan kerangka kerja yang kuat dan fleksibel untuk pengembangan model *Deep learning*, mendukung komputasi yang efisien dan adaptasi model yang cepat sesuai kebutuhan [34].

2.8 *Structemp*

Structemp merupakan singkatan dari Struktural dan Temporal, yaitu pendekatan analitis yang menggabungkan dua dimensi utama dalam representasi dan pemrosesan data graf, yakni aspek struktural dan aspek temporal. Integrasi keduanya memberikan kemampuan analisis yang lebih menyeluruh terhadap sistem yang kompleks, khususnya dalam konteks jaringan komputer yang memiliki karakteristik hubungan antar komponen serta dinamika waktu yang berubah secara kontinu. Pendekatan ini menjadi sangat penting dalam deteksi intrusi jaringan, karena serangan siber umumnya tidak hanya terjadi dalam satu waktu atau satu titik saja, melainkan melibatkan rangkaian aksi yang tersebar secara spasial dan temporal [9].

Pada sisi struktural, data jaringan direpresentasikan dalam bentuk graf di mana setiap *node* merepresentasikan komponen seperti *host*, perangkat, alamat IP, atau *flow*, sedangkan *edge* merepresentasikan hubungan antar *node* seperti koneksi, pertukaran paket, atau dependensi komunikasi. Dengan menggunakan *Graph Neural Network* (GNN), informasi dari *node-node* yang berdekatan dapat diagregasi melalui proses *message passing* untuk membentuk representasi *node* yang lebih informatif. Analisis struktural ini memungkinkan sistem untuk mengenali pola konektivitas, mendeteksi anomali topologis seperti *outlier* dalam hubungan jaringan, serta mengidentifikasi komunitas atau kelompok entitas yang memiliki frekuensi interaksi tinggi, yang sering kali menjadi indikator aktivitas mencurigakan [9].

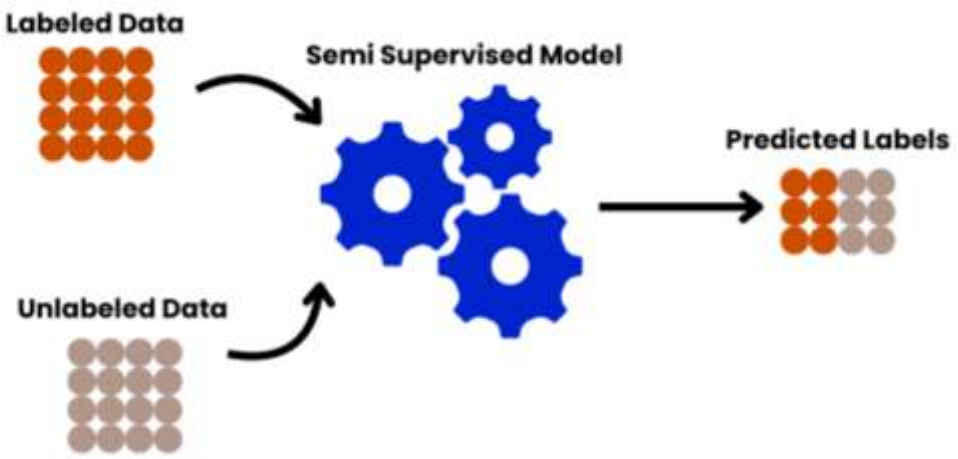
Sementara itu, analisis temporal menangkap bagaimana hubungan dan nilai atribut pada *node* atau *edge* berubah seiring waktu. Dalam skenario deteksi intrusi, dimensi temporal ini sangat krusial untuk mengenali serangan yang memiliki urutan aksi tertentu, seperti *scanning* yang diikuti dengan eksploitasi dan pencurian data. Selain itu, terdapat pula serangan yang memanfaatkan celah waktu tertentu atau menunjukkan pola temporal tidak linier, sehingga memerlukan model yang mampu mengidentifikasi temporal *dependencies* antar peristiwa. Dengan menangkap pola sekuensial dan fluktuasi nilai fitur dalam deret waktu, model dapat membedakan antara lonjakan trafik yang wajar dan anomali berbahaya [9].

Kombinasi dari analisis struktural dan temporal dalam pendekatan *Structemp* memungkinkan sistem untuk membangun model pembelajaran berbasis graf yang lebih kontekstual dan adaptif terhadap karakteristik data jaringan. Hal ini sangat relevan dalam deteksi intrusi modern yang menuntut pemahaman terhadap interaksi yang kompleks dan dinamis. Dengan menggunakan *Structemp*-GNN, model tidak hanya mengandalkan *snapshot* statis dari data jaringan, tetapi juga memperhitungkan evolusi topologi dan perilaku *node* dalam waktu, sehingga mampu memberikan deteksi yang lebih akurat terhadap serangan yang bersifat tersebar dan bertahap [9].

2.9 Semi Supervised Learning

Semi-supervised learning (SSL) merupakan paradigma dalam *machine learning* yang menggabungkan kekuatan antara *supervised learning* dan *unsupervised learning* dengan tujuan untuk meningkatkan performa model menggunakan kombinasi data berlabel dan data tidak berlabel. Pendekatan ini sangat berguna dalam situasi di mana proses pelabelan data membutuhkan biaya, waktu, atau keahlian khusus yang tidak selalu tersedia, sementara data tidak berlabel biasanya melimpah dan mudah diperoleh. Dengan memanfaatkan informasi dari kedua jenis data tersebut, SSL memungkinkan model untuk membangun representasi yang lebih umum dan *robust* terhadap distribusi data yang sebenarnya [20]

SSL bekerja berdasarkan asumsi bahwa data tidak berlabel tetap menyimpan informasi penting terkait struktur data atau distribusi kelas yang bisa dimanfaatkan selama pelatihan. Melalui teknik seperti *pseudo-labeling*, *consistency regularization*, dan pendekatan lainya, SSL dapat membantu model untuk mengidentifikasi pola-pola tersembunyi dan memperkuat batas klasifikasi meskipun hanya memiliki sedikit data berlabel. Dalam konteks deteksi intrusi jaringan (NIDS), paradigma ini sangat bermanfaat karena data serangan yang sudah diberi label sering kali langka, sedangkan trafik jaringan dalam bentuk data tidak berlabel tersedia dalam jumlah besar. Oleh karena itu, SSL menjadi solusi yang menjanjikan untuk meningkatkan akurasi sistem deteksi tanpa memerlukan pelabelan manual secara besar-besaran.[20].



Gambar 2.9 Semi-supervised learning

Dalam bidang *semi-supervised learning* (SSL), berbagai pendekatan telah dikembangkan untuk mengoptimalkan pembelajaran dari kombinasi data berlabel dan tidak berlabel. Menurut taksonomi yang dijelaskan oleh Yang et al. [20] pendekatan dalam *deep semi-supervised learning* (DSSL) dapat diklasifikasikan ke dalam lima kategori utama, yaitu metode generatif, regularisasi konsistensi, berbasis graf, *pseudo-labeling*, dan metode hibrida.

Metode generatif bertujuan untuk mempelajari distribusi laten dari data menggunakan model seperti *Variational Autoencoders* (VAE) dan *Generative Adversarial Networks* (GAN). Dengan memahami struktur distribusi data tidak berlabel, model dapat menghasilkan representasi fitur yang lebih informatif untuk mendukung klasifikasi. Salah satu pendekatan yang relevan adalah *TripleGAN*, yang menyatukan *generator*, *discriminator*, dan *classifier* dalam satu *pipeline* [20].

Metode regularisasi konsistensi didasarkan pada prinsip bahwa prediksi model seharusnya tetap stabil meskipun terjadi gangguan kecil pada *input*, seperti augmentasi. Arsitektur *Teacher-Student* umum digunakan dalam pendekatan ini, di mana model *student* dituntun untuk mengikuti *output* dari *teacher* yang dianggap lebih stabil [20].

Sementara itu, metode berbasis graf membentuk representasi data dalam struktur graf, di mana tiap *node* merepresentasikan sampel dan *edge* mengindikasikan kemiripan antar sampel. Label dari *node* yang berlabel disebarkan ke *node* tak berlabel melalui koneksi topologis dengan asumsi bahwa *node* yang terhubung cenderung memiliki label yang sama. Beberapa model terkenal yang termasuk dalam pendekatan ini adalah *Graph Convolutional Network* (GCN), *GraphSAGE*, dan *Variational Graph Autoencoder* (VGAE) [20].

Pada pendekatan *pseudo-labeling*, model menghasilkan label semu terhadap data tidak berlabel berdasarkan prediksi awal yang memiliki tingkat kepercayaan tinggi. Data tersebut kemudian digunakan kembali dalam proses pelatihan seolah-olah telah berlabel. Strategi ini bersifat iteratif dan memungkinkan model untuk memperluas cakupan data latih secara mandiri. Contoh metode yang mengadopsi pendekatan ini antara lain *Pseudo-Label*, *Tri-Net*, dan *Meta Pseudo Labels (MPL)* [20].

Metode hibrida menggabungkan beberapa strategi SSL ke dalam satu arsitektur untuk menggabungkan keunggulan dari masing-masing pendekatan. Teknik ini mencakup elemen

seperti *pseudo-labeling*, regularisasi konsistensi, augmentasi data, serta estimasi entropi. Metode seperti *MixMatch* dan *FixMatch* menjadi perwakilan efektif dari pendekatan ini, dan telah terbukti memberikan performa yang tinggi pada berbagai *benchmark semi-supervised learning* [20]. Pendekatan hibrid menjadi semakin populer dalam aplikasi dunia nyata karena kemampuannya menghadapi tantangan keterbatasan anotasi dan kompleksitas distribusi data secara bersamaan.

2.10 *Dataset*

Dataset adalah kumpulan data yang terorganisir dan digunakan untuk analisis atau pelatihan model *machine learning*. *Dataset* ini terdiri dari sekumpulan data yang berhubungan satu sama lain dan biasanya disusun dalam bentuk tabel, di mana baris-baris mewakili sampel data dan kolom-kolom mewakili fitur atau atribut dari data tersebut. Setiap baris dalam *dataset* mengandung informasi yang relevan untuk analisis dan dapat dikumpulkan dari berbagai sumber seperti eksperimen ilmiah, survei, sensor, atau log jaringan. *Dataset* dapat berbentuk data terstruktur, semi-terstruktur, atau tidak terstruktur. Dalam konteks NIDS, *dataset* yang digunakan biasanya berbasis dua tipe data utama yaitu *packet-based* dan *flow-based* data [35].

Packet-based data merupakan jenis data yang mencatat informasi setiap paket jaringan secara individual. Data ini biasanya dikumpulkan menggunakan alat seperti *Wireshark*, *tcpdump*, atau sistem *deep packet inspection* (DPI). Setiap entri dalam *dataset* ini mencakup informasi yang lebih detail seperti alamat IP sumber dan tujuan, *port* komunikasi, protokol yang digunakan (TCP, UDP, ICMP, dll.), ukuran paket, dan berbagai flag protokol (misalnya, SYN, ACK, FIN dalam TCP). Kelebihan dari *packet-based* data adalah tingkat detail yang tinggi sehingga memungkinkan analisis perilaku jaringan secara komprehensif hingga ke tingkat paket. Namun, kelemahannya adalah ukuran *dataset* yang sangat besar dan kebutuhan sumber daya komputasi yang tinggi, terutama dalam sistem *real-time* atau pada jaringan dengan trafik padat. Selain itu, analisis paket per paket memerlukan sistem klasifikasi yang sangat efisien agar tetap mampu menangkap serangan secara tepat waktu[35].

Sebaliknya, *flow-based* data menyederhanakan informasi jaringan dengan mengelompokkan beberapa paket yang memiliki atribut serupa menjadi satu aliran komunikasi yang disebut *flow*. Proses ini umumnya dilakukan oleh sistem seperti *NetFlow* (Cisco), IPFIX, atau sFlow. Setiap flow biasanya menyimpan informasi agregat seperti jumlah *byte* dan paket yang dikirim, durasi aliran, kecepatan rata-rata transfer, waktu mulai dan selesai komunikasi, serta atribut pengenal (alamat IP, *port*, protokol). Kelebihan utama dari *flow-based* data adalah efisiensinya dalam penyimpanan dan pemrosesan, karena ukuran data menjadi jauh lebih kecil dibanding *packet-based*. Hal ini menjadikannya pilihan yang lebih praktis untuk implementasi NIDS dalam skala besar atau pada perangkat dengan sumber daya terbatas, seperti dalam konteks IoT [35].

Penelitian ini menggunakan enam *dataset* yang bervariasi dalam jumlah data, jenis serangan, format data (*flow-based* dan *packet-based*), serta kompleksitas trafik jaringan. Seluruh *dataset* dipilih untuk merepresentasikan beragam skenario nyata dalam sistem deteksi intrusi jaringan (NIDS), dari lingkungan IoT hingga jaringan *enterprise*. Penjelasan rinci mengenai masing-masing *dataset* disajikan sebagai berikut.

Tabel 2.1 Klasifikasi *Dataset*

No	*Dataset*	Format	Sumber Data	Total Fitur	Jumlah Serangan	Jenis Serangan Utama	Jumlah Data
1	NF-UQ-NIDS-v2	Flow-based	gabungan NF-BoT-IoT-v2 (50%), CSE-CIC-IDS2018-v2 (25%), NF-ToN-IoT-v2 (22%), dan NF-UNSW-NB15-v2 (3%).	44	20	DDoS, Reconnaissance, Injection, DoS, Brute Force, Password, XSS, Infilteration, Exploits, Scanning, Fuzzers, Backdoor, Bot, Generic, Analysis, Theft, Shellcode, MITM, Worms, Ransomware	75987976
2	NF-ToN-IoT-v2	Flow-based	ToN-IoT	43	9	Backdoor, DoS, DDoS, Injection, MITM, Password, Ransomware, Scanning, XSS	16940496
3	NF-UNSW-NB15-v2	Flow-based	UNSW-NB15	43	9	Fuzzers, Analysis, Backdoor, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms	2390275
4	NF-ToN-IoT-v3	Flow-based	ToN-IoT	53	9	Backdoor, DoS, DDoS, Injection, MITM, Password, Ransomware, Scanning, XSS	27520260
5	UNSW-NB15	Packet-level	simulasi Cyber Range (ADFA)	49	9	Fuzzers, Analysis, Backdoors, DoS Exploits, Generic, Reconnaissance, Shellcode and Worms	2160176
6	HIKARI 2021	Flow-based	simulasi serangan terenkripsi.	88	5	Bruteforce-XML,Bruteforce,Background,Probing,XMRIGCC CryptoMiner	555278

NF-UQ-NIDS-v2 adalah sebuah *dataset* berskala besar yang dikembangkan oleh University of Queensland sebagai bagian dari upaya menyediakan *benchmark* komprehensif untuk penelitian sistem deteksi intrusi jaringan. *Dataset* ini terdiri atas 75.987.976 entri flow dan dibangun melalui proses penggabungan dari empat *dataset* sumber utama, yaitu NF-BoT-IoT-v2 (50%), CSE-CIC-IDS2018-v2 (25%), NF-ToN-IoT-v2 (22%), dan NF-UNSW-NB15-v2 (3%). Proses penggabungan ini menghasilkan kumpulan data yang sangat bervariasi dari segi jenis trafik, skenario jaringan, serta pola serangan, yang menjadikan NF-UQ-NIDS-v2 sebagai salah satu *dataset* dengan cakupan serangan terluas. Secara struktural, *dataset* ini terdiri dari 43 fitur, yang mencakup informasi statistik dari koneksi jaringan seperti durasi, jumlah

paket dan *byte* yang ditransmisikan, protokol yang digunakan, *port* sumber dan tujuan, serta atribut *flow* lainnya. Label serangan pada *dataset* ini terbagi ke dalam 20 kategori, meliputi serangan umum seperti DDoS, DoS, dan *Scanning*, hingga serangan yang lebih kompleks seperti MITM, *Ransomware*, *Shellcode*, XSS, *Injection*, dan *Password Attack*. Format data yang digunakan adalah CSV dengan struktur *NetFlow*, memudahkan proses integrasi ke berbagai platform pemrosesan data dan *Machine Learning*. NF-UQ-NIDS-v2 dirancang agar mampu merepresentasikan kondisi jaringan modern dengan berbagai jenis ancaman siber, serta memperluas cakupan representasi dari *dataset* individual yang menjadi sumbernya.[36]

NF-ToN-IoT-v2 adalah *dataset flow-based* yang dikembangkan dari ToN-IoT dan difokuskan pada lingkungan *Internet of Things* (IoT). *Dataset* ini terdiri dari 16.940.496 entri *flow* dengan 43 fitur yang merepresentasikan statistik koneksi jaringan seperti durasi, jumlah paket dan *byte*, *port* sumber dan tujuan, serta protokol yang digunakan. NF-ToN-IoT-v2 dirancang untuk mencerminkan trafik jaringan yang umum terjadi pada sistem IoT, termasuk komunikasi antar perangkat *edge* dan server pusat. Terdapat 9 kategori serangan dalam *dataset* ini, yaitu *Backdoor*, DoS, DDoS, *Injection*, MITM, *Password*, *Ransomware*, *Scanning*, dan XSS. Seluruh serangan disimulasikan secara realistis untuk meniru skenario ancaman yang mungkin terjadi dalam infrastruktur IoT. Format CSV yang digunakan membuat *dataset* ini mudah untuk diproses dan dianalisis lebih lanjut, serta cocok untuk digunakan dalam pengembangan dan pengujian sistem deteksi intrusi berbasis *machine learning*.[36]

NF-UNSW-NB15-v2 adalah versi *flow-based* dari *dataset* UNSW-NB15 yang telah diadaptasi untuk mendukung analisis berbasis statistik aliran jaringan. *Dataset* ini terdiri dari 2.390.275 entri flow dengan 43 fitur yang mencakup informasi seperti durasi koneksi, jumlah paket dan *byte*, protokol, serta atribut jaringan lainnya. Data ini berasal dari hasil simulasi trafik jaringan nyata dan serangan yang dilakukan di lingkungan *Cyber Range* oleh *Australian Centre for Cyber Security* (ACCS). NF-UNSW-NB15-v2 mencakup 9 kategori serangan, yaitu *Fuzzers, Analysis, Backdoor, DoS, Exploits, Generic, Reconnaissance, Shellcode*, dan *Worms*. Meskipun ukuran *dataset* ini lebih kecil dibandingkan *dataset* lain yang digunakan dalam penelitian ini, kualitas label yang akurat dan struktur fitur yang konsisten menjadikannya tetap relevan sebagai *benchmark* dalam pengujian sistem deteksi intrusi jaringan, terutama untuk validasi model dalam skenario data terbatas.[36]

NF-ToN-IoT-v3 adalah versi lanjutan dari NF-ToN-IoT-v2 yang disusun dalam format flow-based dengan jumlah entri sebesar 27.520.260 *flow*. *Dataset* ini memiliki 53 fitur, meningkat dari versi sebelumnya yang hanya memiliki 43 fitur, sehingga memberikan informasi yang lebih kaya terkait karakteristik koneksi jaringan. Seperti versi sebelumnya, *dataset* ini merepresentasikan trafik dalam lingkungan IoT dan mencakup 9 kategori serangan, yaitu *Backdoor, DoS, DDoS, Injection, MITM, Password, Ransomware, Scanning*, dan XSS. Penambahan fitur pada NF-ToN-IoT-v3 memungkinkan eksplorasi atribut yang lebih kompleks dalam mendeteksi anomali, baik dari sisi perilaku temporal maupun pola komunikasi antar perangkat. *Dataset* ini juga mempertahankan format CSV yang terstruktur dan kompatibel dengan berbagai metode pemrosesan data dan pelatihan model, menjadikannya sumber data yang bernilai tinggi untuk riset sistem deteksi intrusi dalam ekosistem IoT yang terus berkembang.[37]

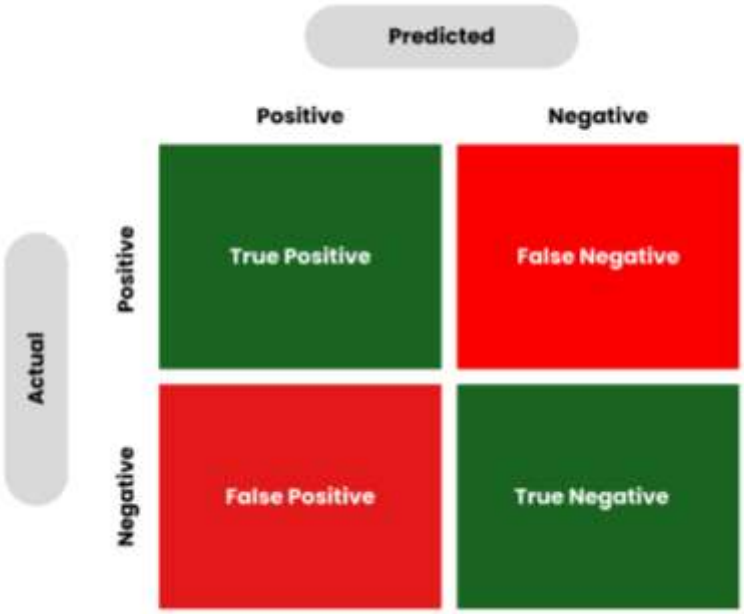
UNSW-NB15 adalah *dataset* packet-based yang dikembangkan oleh *Australian Centre for Cyber Security* (ACCS) di *University of New South Wales* (UNSW), menggunakan lingkungan

uji *Cyber Range* yang mereplikasi trafik jaringan nyata dan diserang oleh berbagai jenis serangan terencana. *Dataset* ini terdiri dari 540.044 entri, dengan 49 fitur yang mencakup atribut lengkap dari setiap paket jaringan, seperti alamat IP sumber dan tujuan, port, protokol, ukuran *payload*, *flag* TCP, serta informasi sesi dan layanan. Tidak seperti *dataset flow-based* yang merangkum koneksi menjadi agregat, *dataset* ini memberikan setiap paket secara individual, sehingga memungkinkan analisis yang sangat granular terhadap perilaku jaringan. UNSW-NB15 memuat 9 kategori serangan, yaitu *Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode*, dan *Worms*, yang mencerminkan kombinasi serangan berbasis aplikasi, eksploitasi protokol, serta teknik pengintaian. Setiap serangan diklasifikasikan berdasarkan profil ancaman nyata dan dijalankan dalam skenario terkontrol, menjamin kualitas anotasi dan representasi data yang realistis. *Dataset* ini telah menjadi *benchmark* dalam berbagai studi deteksi intrusi karena penyusunan fiturnya yang terstandar, label yang akurat, dan dokumentasi yang komprehensif, menjadikannya ideal untuk pengembangan serta evaluasi sistem IDS berbasis *machine learning*, terutama pada pendekatan klasifikasi berbasis paket secara langsung.[38]

HIKARI2021 adalah *dataset flow-based* yang dirancang untuk mencerminkan trafik jaringan nyata yang mengandung serangan terenkripsi dan aktivitas *crypto-mining*, menjadikannya salah satu *dataset* modern yang relevan dengan tantangan keamanan jaringan saat ini. Dataset ini terdiri dari 555.278 entri flow dengan 88 fitur, yang mencakup informasi statistik dari koneksi seperti durasi, jumlah *byte*, jumlah paket, port, protokol, serta berbagai metrik tambahan yang berguna untuk analisis perilaku jaringan. Salah satu karakteristik utama dari HIKARI2021 adalah inklusi serangan yang memanfaatkan enkripsi, di mana payload tidak lagi dapat dianalisis secara langsung, sehingga mendorong pendekatan deteksi berbasis metadata dan pola komunikasi. Terdapat 5 kategori serangan dalam *dataset* ini, yaitu *Bruteforce-XML, Bruteforce, Background, Probing,* dan *XMRIGCC CryptoMiner*, dengan sebagian besar berasal dari simulasi trafik pada sistem yang menjalankan layanan terenkripsi atau dikompromikan oleh *malware* penambang kripto. *Dataset* ini disusun dalam format CSV dan dilengkapi dengan dokumentasi yang memudahkan integrasi ke dalam *pipeline* analitik atau pelatihan model *machine learning*. HIKARI2021 menjadi penting dalam penelitian NIDS karena mampu mengevaluasi efektivitas sistem deteksi intrusi dalam konteks jaringan yang semakin banyak menggunakan enkripsi sebagai standar komunikasi.[39]

2.11 Evaluasi metrik

Evaluasi kinerja merupakan tahap penting dalam pengembangan sistem *Network Intrusion Detection System* (NIDS), karena memberikan gambaran sejauh mana sistem mampu mendeteksi dan mengklasifikasikan aktivitas jaringan dengan akurat. Untuk itu, digunakan berbagai metrik evaluasi yang dihitung berdasarkan hasil klasifikasi model terhadap data pengujian. Salah satu metode yang paling umum digunakan dalam mengukur performa model klasifikasi adalah *confusion matrix* [13].



Gambar 2.10 *Confusion Matrix*

Matriks ini menunjukkan perbandingan antara hasil prediksi model dan label aktual pada data, yang terdiri dari empat komponen utama: *True Positive* (TP) yaitu jumlah prediksi benar pada kelas positif, *True Negative* (TN) yaitu prediksi benar pada kelas negatif, *False Positive* (FP) yaitu prediksi salah ketika model mengklasifikasikan data sebagai positif padahal sebenarnya negatif, dan *False Negative* (FN) yaitu prediksi salah ketika model mengklasifikasikan data sebagai negatif padahal sebenarnya positif. Dari keempat komponen ini, sejumlah metrik evaluasi dapat dihitung. Akurasi mengukur proporsi prediksi benar terhadap seluruh data dan dirumuskan sebagai

$$Akurasi = \frac{TP + TN}{TP + TN + FP + FN}$$

Presisi mengukur ketepatan prediksi positif yang dilakukan model, yaitu

$$Presisi = \frac{TP}{TP + FP}$$

Recall atau sensitivitas menunjukkan kemampuan model dalam mendeteksi semua kejadian positif aktual, dengan rumus

$$Recall = \frac{TP}{TP + FN}$$

Sedangkan F1-score merupakan rata-rata harmonik dari presisi dan *recall*, yang memberikan keseimbangan antara keduanya, terutama dalam kondisi data tidak seimbang, dan dirumuskan sebagai

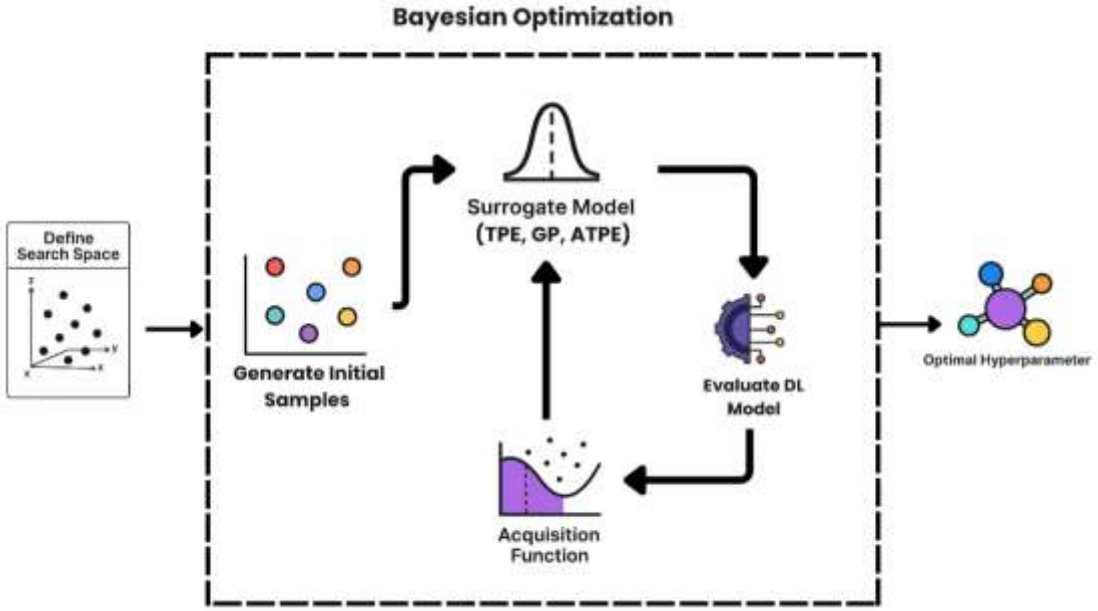
$$F1Score = 2 \times \frac{Recall \times Precision}{Recall + Precision}$$

Selain metrik klasifikasi tersebut, *inference time* juga menjadi parameter penting yang mengukur waktu yang dibutuhkan sistem untuk mendeteksi serangan setelah insiden terjadi.

Inference time yang rendah mencerminkan kemampuan sistem dalam memberikan respons cepat terhadap potensi ancaman, yang sangat penting dalam konteks NIDS *real-time*. Seluruh metrik ini memberikan dasar evaluasi yang komprehensif untuk menilai keandalan, ketepatan, dan kecepatan sistem deteksi intrusi dalam menghadapi berbagai skenario serangan jaringan[13].

2.12 *Bayesian Hyperparameter Tuning*

Hyperparameter tuning merupakan proses esensial dalam pengembangan model *machine learning* atau *deep learning* yang bertujuan untuk menemukan kombinasi nilai *hyperparameter* yang menghasilkan performa terbaik. Proses ini menjadi semakin penting ketika diterapkan pada model yang kompleks dan mahal secara komputasi, seperti *deep learning* dan *Graph Neural Networks* (GNN), di mana setiap kombinasi *hyperparameter* membutuhkan pelatihan penuh yang memakan waktu dan sumber daya signifikan. Pendekatan pencarian *hyperparameter* tradisional, seperti *grid search* dan *random search*, umumnya tidak efisien dalam menjelajahi ruang pencarian berdimensi tinggi karena mereka mengevaluasi konfigurasi secara acak tanpa mempertimbangkan hasil sebelumnya.



Gambar 2.11 Gambar *Hyperparameter Optimization*

Bayesian Hyperparameter Tuning menggunakan pendekatan yang lebih efisien dan adaptif dengan membangun sebuah model probabilistik yang disebut model *surrogate*. Model ini digunakan untuk memperkirakan nilai fungsi objektif seperti akurasi atau F1-score berdasarkan kombinasi *hyperparameter* yang telah diuji pada iterasi sebelumnya. Dengan menggunakan model *surrogate* ini, *Bayesian Optimization* melakukan proses *tuning* secara *iteratif*: pada setiap langkah, algoritma mengevaluasi satu kombinasi baru, memperbarui model *surrogate*, dan kemudian menggunakan informasi yang diperoleh untuk memandu pencarian berikutnya. Proses pemilihan konfigurasi berikutnya dikontrol oleh fungsi akuisisi, yang menyeimbangkan antara eksplorasi kombinasi baru dan eksploitasi konfigurasi yang sudah terbukti menghasilkan hasil baik. Pendekatan ini secara konsisten mampu mencapai performa optimal dengan jumlah evaluasi yang jauh lebih sedikit dibandingkan metode konvensional, dan sangat sesuai

diterapkan dalam *tuning hyperparameter model deep learning* dengan ruang pencarian yang kompleks [40].

Dalam proses *Bayesian Hyperparameter Tuning*, pemilihan titik berikutnya yang akan dievaluasi sangat bergantung pada fungsi akuisisi (*acquisition function*). Fungsi ini bertanggung jawab untuk menyeimbangkan dua strategi eksplorasi dalam proses optimasi eksploitasi, yaitu memilih kombinasi *hyperparameter* yang diprediksi memiliki performa tinggi berdasarkan model *surrogate* saat ini dan eksplorasi, yaitu mencoba kombinasi yang belum pernah dievaluasi sebelumnya untuk menghindari jebakan pada solusi lokal. Tanpa fungsi akuisisi yang efektif, proses *tuning* dapat mengalami stagnasi atau eksplorasi yang sia-sia pada area yang tidak menjanjikan[40].

Beberapa jenis fungsi akuisisi yang umum digunakan antara lain *Expected Improvement* (EI), *Probability of Improvement* (PI), dan *Upper Confidence Bound* (UCB). *Expected Improvement*, sebagai salah satu fungsi paling populer, menghitung nilai harapan dari peningkatan performa terhadap titik terbaik yang telah ditemukan sejauh ini. Fungsi ini tidak hanya mempertimbangkan nilai prediksi dari model *surrogate*, tetapi juga variansi atau ketidakpastian prediksi tersebut. Sementara itu, PI berfokus pada probabilitas bahwa konfigurasi baru akan mengungguli nilai terbaik yang telah ditemukan, dan UCB memadukan ekspektasi dan ketidakpastian untuk membentuk batas atas keyakinan. Ketiga fungsi ini memiliki keunggulan masing-masing dan dapat dipilih berdasarkan karakteristik ruang pencarian atau kompleksitas model. Pemilihan fungsi akuisisi yang sesuai sangat menentukan efisiensi konvergensi dalam *Bayesian Optimization*, khususnya dalam *tuning* model *machine learning* yang memerlukan evaluasi mahal seperti *deep learning* [40].

Model *surrogate* merupakan inti dari mekanisme *Bayesian Optimization* karena berfungsi sebagai pendekatan probabilistik untuk memperkirakan performa suatu kombinasi *hyperparameter* tanpa perlu melatih model secara langsung. Dalam konteks ini, fungsi objektif seperti akurasi atau F1-score dipandang sebagai fungsi *black-box* yang mahal secara komputasi, sehingga perlu didekati menggunakan model prediksi yang jauh lebih ringan. Model *surrogate* bertugas untuk membangun pemetaan dari ruang *hyperparameter* menuju nilai prediksi performa, berdasarkan hasil evaluasi dari kombinasi-kombinasi yang telah dicoba sebelumnya. Dengan kata lain, model ini digunakan untuk menebak kombinasi mana yang mungkin akan memberikan hasil terbaik di iterasi berikutnya[40].

Dalam praktiknya, berbagai jenis model dapat digunakan sebagai *surrogate*, seperti *Gaussian Process* (GP), *Random Forest*, dan *Tree-structured Parzen Estimator* (TPE). GP umum digunakan dalam ruang pencarian yang kecil dan kontinu karena kemampuannya dalam menghasilkan distribusi prediktif berbasis mean dan variansi. Sementara itu, TPE lebih sesuai untuk ruang pencarian yang besar, tidak kontinu, dan mengandung nilai kategorikal maupun diskrit. Model *surrogate* terus diperbarui secara iteratif dengan hasil eksperimen baru, sehingga prediksi yang dihasilkan menjadi semakin akurat seiring bertambahnya data. Keefektifan *Bayesian Optimization* sangat ditentukan oleh ketepatan model *surrogate* dalam merepresentasikan fungsi objektif yang tidak diketahui tersebut [40].

Tree-structured Parzen Estimator (TPE) adalah salah satu metode *surrogate* yang banyak digunakan dalam *Bayesian Optimization* modern, terutama dalam pustaka seperti Optuna. Berbeda dengan pendekatan lain yang langsung mencoba memprediksi performa suatu kombinasi *hyperparameter*, TPE justru mempelajari kemungkinan nilai *hyperparameter*

berdasarkan seberapa baik hasil yang pernah dicapai, sekaligus mempertimbangkan keseluruhan distribusi performa. Metode ini sangat fleksibel karena mampu menangani berbagai jenis nilai baik yang bersifat kontinu, diskrit, maupun kategorikal dan tetap efisien meskipun ruang pencariannya besar dan kompleks [40].

Secara praktik, TPE membagi hasil percobaan sebelumnya menjadi dua kelompok yaitu kelompok yang memberikan hasil terbaik dan kelompok sisanya. Dari kedua kelompok ini, TPE membentuk dua distribusi terpisah. Proses pencarian dilakukan dengan memilih kombinasi *hyperparameter* yang lebih mungkin berasal dari kelompok dengan performa tinggi. Dengan pendekatan ini, TPE mampu mempercepat proses *tuning* dan mengarahkan pencarian ke wilayah yang lebih menjanjikan dalam ruang parameter [40].

2.13 *Optimizer*

Dalam *deep learning*, proses pelatihan model memiliki tujuan utama untuk meminimalkan *loss function*, yaitu fungsi matematis yang mengukur seberapa jauh prediksi model dari nilai target yang sebenarnya. Proses ini memerlukan teknik optimasi numerik, dan itu merupakan peran *optimizer*. *Optimizer* adalah algoritma yang bertugas memperbarui bobot dan parameter model berdasarkan arah dan besarnya turunan gradien dari *loss function* terhadap parameter-parameter tersebut [41].

Tujuan utama dari *optimizer* adalah mempercepat konvergensi terhadap solusi optimal sambil menghindari jebakan lokal seperti *saddle point* dan *plateau*. *Optimizer* yang efektif tidak hanya mempercepat pelatihan, tetapi juga meningkatkan generalisasi model terhadap data yang belum pernah dilihat sebelumnya. Seiring meningkatnya kompleksitas model dan volume data, kebutuhan akan *optimizer* yang adaptif dan efisien menjadi semakin penting [41].

Optimizer memengaruhi performa akhir model secara signifikan. Meskipun tidak ada satu *optimizer* yang unggul secara mutlak untuk semua jenis permasalahan, pendekatan evaluasi empiris pada berbagai skenario telah memberikan wawasan praktis untuk pemilihan *optimizer*. Dalam sebuah studi *benchmarking* besar terhadap 15 *optimizer* populer dalam *deep learning*, ditemukan bahwa performa setiap *optimizer* sangat tergantung pada karakteristik masalah dan parameter awalnya. Namun, beberapa metode seperti Adam tetap menunjukkan performa yang kompetitif di banyak skenario tanpa perlu banyak penyetelan *hyperparameter* [41].

Sejumlah algoritma optimasi adaptif yang banyak digunakan dalam implementasi modern adalah AdamW, RAdam, dan Lion. Ketiganya merupakan pengembangan dari pendekatan dasar optimasi berbasis momen yaitu Adam, yang secara umum memanfaatkan kombinasi antara rata-rata gradien dan kuadrat gradien untuk menyesuaikan langkah pembaruan parameter secara individual. Adam dikenal luas karena kestabilannya dalam pelatihan model besar dan efisiensinya dalam menghadapi gradien yang tidak stabil atau bersifat *sparse* [41][42].

Meskipun demikian, pendekatan dasar tersebut memiliki sejumlah keterbatasan, seperti sensitivitas terhadap *hyperparameter*, regularisasi yang tidak eksplisit, dan ketidakstabilan pada tahap awal pelatihan. Untuk menjawab tantangan ini, berbagai varian telah dikembangkan. AdamW memperbaiki penerapan *weight decay* dengan memisahkannya secara eksplisit dari estimasi gradien. RAdam memperkenalkan mekanisme *rectification* untuk menstabilkan pembaruan pada fase awal pelatihan. Sementara itu, Lion menerapkan strategi pembaruan berbasis tanda (*sign-based update*) yang secara signifikan mengurangi konsumsi memori dan

komputasi, serta telah menunjukkan performa kompetitif dalam pelatihan model berskala besar seperti *Transformer* dan *Vision Transformer* [41].

Dengan mempertimbangkan karakteristik tersebut, ketiga algoritma AdamW, RAdam, dan Lion telah menjadi pilihan utama dalam berbagai implementasi *deep learning* modern. Masing-masing menawarkan pendekatan yang unik untuk mengatasi tantangan dalam proses optimasi, baik dari segi regularisasi, kestabilan pembaruan, maupun efisiensi komputasi. Untuk memahami keunggulan dan prinsip kerja dari masing-masing algoritma secara lebih mendalam, berikut dijelaskan ketiga metode tersebut secara terpisah.

AdamW merupakan salah satu algoritma optimasi adaptif yang dikembangkan untuk mengatasi kelemahan dari Adam, khususnya dalam hal penerapan regularisasi bobot melalui mekanisme *weight decay*. Pada Adam konvensional, *weight decay* diterapkan dengan menambahkan penalti langsung ke dalam gradien parameter, namun pendekatan ini terbukti dapat menyebabkan distorsi karena regularisasi menjadi terpengaruh oleh mekanisme penyesuaian langkah adaptif yang menjadi inti dari Adam itu sendiri. AdamW memperbaiki hal ini dengan memisahkan penerapan regularisasi dari proses pembaruan gradien, sehingga bobot parameter dikurangi secara eksplisit tanpa mengubah arah gradien. Pemisahan ini menghasilkan pembaruan parameter yang lebih stabil dan kontrol regularisasi yang lebih konsisten, yang pada akhirnya meningkatkan kemampuan generalisasi model terhadap data yang tidak terlihat [43].

Secara matematis, pembaruan parameter AdamW dilakukan melalui beberapa tahap. Pertama, estimasi momen pertama ($\boldsymbol{m_t}$) dihitung menggunakan *moving average* eksponensial gradien:

$$\boldsymbol{m_t} = \beta_1 \boldsymbol{m_{t-1}} + (1 - \beta_1)\boldsymbol{g_t}$$

di mana $\boldsymbol{g_t}$ adalah gradien saat ini dan β_1 adalah koefisien momentum. Kedua, estimasi momen kedua ($\boldsymbol{v_t}$) dihitung untuk mengukur variansi gradien:

$$\boldsymbol{v_t} = \beta_2 \boldsymbol{v_{t-1}} + (1 - \beta_2)\boldsymbol{g_t}^2$$

dengan β_2 sebagai koefisien *smoothing* untuk *second moment*. Karena mt dan vt bias terhadap nol pada langkah awal, dilakukan koreksi bias sebagai berikut:

$$\widehat{\boldsymbol{m}}_{\boldsymbol{t}} = \frac{\boldsymbol{m_t}}{1 - \beta_1{}^t}$$

$$\widehat{\boldsymbol{v}}_{\boldsymbol{t}} = \frac{\boldsymbol{v_t}}{1 - \beta_2{}^t}$$

Akhirnya, pembaruan parameter dilakukan dengan:

$$\boldsymbol{\theta_{t+1}} = \boldsymbol{\theta_t} - \eta\left(\frac{\widehat{\boldsymbol{m}}_{\boldsymbol{t}}}{\sqrt{\widehat{\boldsymbol{v}}_{\boldsymbol{t}}} + \epsilon} + \lambda \boldsymbol{\theta_t}\right)$$

di mana η adalah *learning rate*, ϵ adalah konstanta kecil untuk stabilitas numerik, dan λ adalah koefisien *weight decay* yang diterapkan secara eksplisit. Dengan pemisahan ini, AdamW memastikan bahwa penalti regularisasi tidak terpengaruh oleh penyesuaian adaptif pada arah gradien [43].

Dalam studi *benchmarking* terhadap lima belas optimizer populer yang diuji pada berbagai arsitektur dan konfigurasi pelatihan, AdamW secara konsisten menempati peringkat atas dalam hal *anytime performance*, yaitu kemampuannya mencapai akurasi tinggi dengan efisiensi waktu pelatihan yang baik. Selain itu, AdamW juga menunjukkan ketahanan terhadap *overfitting* dan sensitivitas yang lebih rendah terhadap perubahan *hyperparameter* [43].

RAdam atau *Rectified* Adam merupakan varian dari algoritma Adam yang dirancang untuk mengatasi ketidakstabilan pembaruan parameter yang sering terjadi pada tahap awal pelatihan. Pada Adam konvensional, proses penyesuaian langkah pembaruan adaptif didasarkan pada estimasi momen pertama dan kedua dari gradien. Namun, estimasi momen kedua pada awal pelatihan sering kali memiliki varians yang sangat tinggi atau belum stabil, yang dapat menyebabkan langkah pembaruan menjadi terlalu besar atau tidak terarah dengan baik. RAdam memperkenalkan komponen *rectification term* yang berfungsi menyesuaikan skala pembaruan berdasarkan keandalan estimasi momen kedua, sehingga pembaruan hanya dilakukan secara adaptif ketika jumlah iterasi cukup untuk menghasilkan estimasi yang stabil. Jika estimasi dianggap belum memadai, RAdam akan melakukan *fallback* ke bentuk pembaruan yang menyerupai SGD dengan momentum [44].

Secara matematis, estimasi momen pertama ($\boldsymbol{m_t}$) dan momen kedua ($\boldsymbol{v_t}$) pada RAdam tetap mengikuti formula standar Adam:

$$\boldsymbol{m_t} = \beta_1 \boldsymbol{m_{t-1}} + (1 - \beta_1)\boldsymbol{g_t}$$

$$\boldsymbol{v_t} = \beta_2 \boldsymbol{v_{t-1}} + (1 - \beta_2)\boldsymbol{g_t}^2$$

Dengan β_1 sebagai koefisien momentum dan β_2 sebagai koefisien *smoothing* untuk *second moment*. Karena $\boldsymbol{m_t}$ dan $\boldsymbol{v_t}$ bias terhadap nol pada langkah awal, dilakukan koreksi bias sebagai berikut:

$$\widehat{\boldsymbol{m}}_{\boldsymbol{t}} = \frac{\boldsymbol{m_t}}{1 - {\beta_1}^t}$$

$$\widehat{\boldsymbol{v}}_{\boldsymbol{t}} = \frac{\boldsymbol{v_t}}{1 - {\beta_2}^t}$$

Selanjutnya, RAdam menghitung nilai ρ_t untuk mengevaluasi keandalan estimasi variansi:

$$\rho_t = \rho_\infty - \frac{2t{\beta_2}^t}{1 - {\beta_2}^t}$$

Jika nilai ρ_t menunjukkan bahwa estimasi variansi belum stabil, langkah pembaruan bisa menjadi terlalu agresif. Oleh karena itu, RAdam memperkenalkan *rectification term* (rt) untuk menyesuaikan skala pembaruan agar tetap stabil:

$$r_t = \sqrt{\frac{\rho_t - 4}{\rho_\infty - 4} \cdot \frac{\rho_t - 2}{\rho_\infty - 2} \cdot \frac{\rho_\infty}{\rho_t}}$$

Apabila nilai ρ_t sudah memadai (>4), pembaruan parameter dilakukan sebagai berikut:

$$\boldsymbol{\theta_{t+1}} = \boldsymbol{\theta_t} - \eta r_t \left(\frac{\widehat{\boldsymbol{m}}_{\boldsymbol{t}}}{\sqrt{\widehat{\boldsymbol{v}}_{\boldsymbol{t}}} + \epsilon}\right)$$

Pendekatan ini memberikan keseimbangan antara adaptabilitas dan kestabilan pembaruan sparameter. Dalam evaluasi empiris terhadap berbagai *optimizer* adaptif, RAdam menunjukkan performa yang kompetitif, meskipun tidak sekuat AdamW dalam hal konsistensi lintas tugas (*across tasks*). RAdam secara khusus dinilai unggul dalam skenario pelatihan yang sensitif terhadap dinamika awal dan menjadi pilihan populer pada eksperimen yang memerlukan stabilisasi pada fase awal pelatihan model *neural network* [44].

Lion (*EvoLved Sign Momentum*) merupakan algoritma optimasi modern yang dirancang untuk menyederhanakan proses pembaruan parameter sekaligus meningkatkan efisiensi pelatihan model *deep learning*. Berbeda dengan *optimizer* adaptif konvensional seperti AdamW yang melacak momen pertama dan kedua dari gradien, Lion hanya menggunakan momentum dan fungsi tanda (*sign function*) untuk menentukan arah pembaruan parameter. Hal ini menghasilkan pembaruan dengan besar yang seragam dan struktur yang jauh lebih sederhana secara komputasi. Lion hanya memerlukan penyimpanan satu variabel momentum, sehingga mengurangi konsumsi memori secara signifikan. Proses pembaruan dalam Lion melibatkan dua interpolasi eksponensial yang terpisah untuk memadukan antara memori gradien historis dan kontribusi gradien saat ini. Fungsi tanda yang digunakan dalam pembaruan memberikan efek regularisasi implisit, yang membantu dalam meningkatkan kemampuan generalisasi model, meskipun terkadang menghasilkan nilai kesalahan pelatihan yang lebih tinggi. Selain itu, Lion menunjukkan ketahanan yang baik terhadap variasi *hyperparameter* dan memiliki keunggulan performa dalam pelatihan berskala besar. Dalam berbagai eksperimen, termasuk tugas klasifikasi citra dan pemodelan bahasa, Lion mampu mencapai akurasi yang tinggi sekaligus mempercepat proses pelatihan bila dibandingkan dengan AdamW dan Adafactor, menjadikannya pilihan yang efisien untuk pelatihan model dalam pengaturan komputasi terdistribusi [42].

Secara matematis, Lion memelihara hanya satu variabel momentum $\boldsymbol{m_t}$ yang diperbarui pada setiap langkah menggunakan interpolasi eksponensial antara momentum sebelumnya dan arah gradien saat ini:

$$\boldsymbol{m_t} = \beta_1 \boldsymbol{m_{t-1}} + (1 - \beta_1) sign(\boldsymbol{g_t})$$

di mana $\boldsymbol{g_t}$ adalah tensor gradien pada langkah ke-t, β_1 adalah koefisien momentum, dan fungsi sign () menghasilkan arah (+1 atau -1) untuk setiap elemen tensor gradien. Setelah momentum diperbarui, parameter model $\boldsymbol{\theta_t}$ dioptimalkan dengan langkah pembaruan yang sederhana:

$$\boldsymbol{\theta_{t+1}} = \boldsymbol{\theta_t} - \eta \boldsymbol{m_t}$$

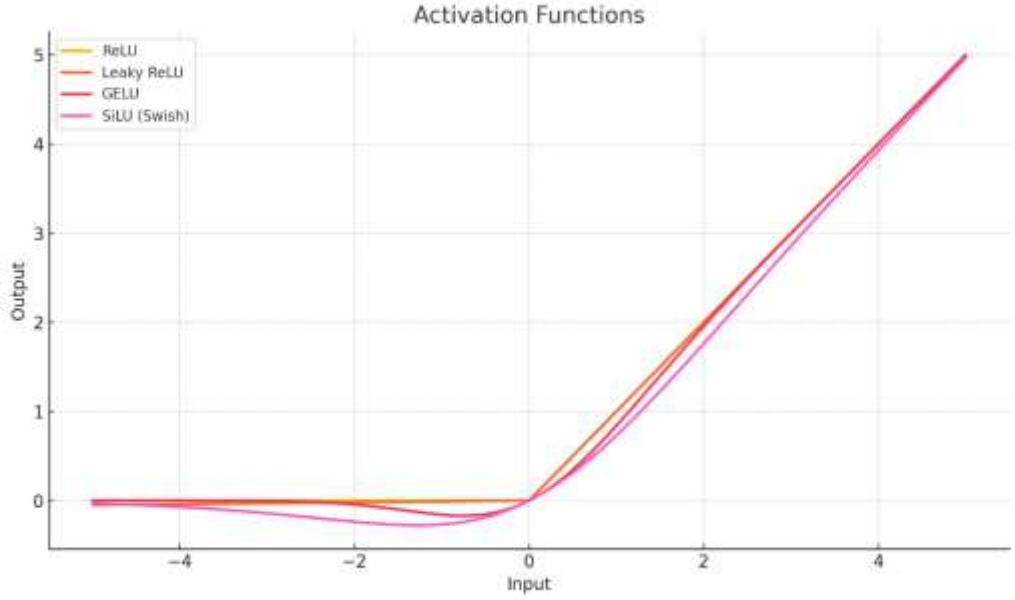
dengan η sebagai *learning rate* (skalar). Formula ini menunjukkan bahwa besar langkah pembaruan ditentukan sepenuhnya oleh η, sementara arah ditentukan oleh momentum ($\boldsymbol{m_t}$) yang sudah di kompensasi melalui fungsi *sign*. Pendekatan ini membuat Lion sangat ringan secara komputasi karena tidak ada operasi pembagian atau akar kuadrat yang diperlukan, seperti pada Adam atau RAdam. Sifatnya yang sederhana juga memungkinkan Lion untuk bekerja dengan baik pada sistem pelatihan berskala besar dengan keterbatasan memori [42].

Ketiga algoritma optimasi yang telah dibahas, yaitu AdamW, RAdam, dan Lion, masing-masing menawarkan pendekatan unik dalam menangani tantangan pelatihan model *deep learning* yang semakin kompleks. AdamW memberikan stabilitas dan kontrol regularisasi yang baik melalui pemisahan eksplisit *weight decay*, RAdam mengatasi ketidakstabilan pada fase awal pelatihan dengan pendekatan *rectified*, sedangkan Lion menghadirkan efisiensi tinggi

dengan struktur pembaruan parameter yang minimalis namun efektif. Pemahaman terhadap karakteristik dan prinsip kerja masing-masing *optimizer* sangat penting dalam proses pemilihan algoritma yang sesuai dengan kebutuhan arsitektur dan skenario pelatihan yang digunakan. Dalam praktiknya, efektivitas setiap *optimizer* sangat bergantung pada konteks penggunaannya, termasuk struktur model, ukuran *batch*, jenis data, serta strategi *tuning* yang diterapkan. Oleh karena itu, pemilihan *optimizer* yang tepat tidak hanya menjadi aspek teknis, tetapi juga bagian integral dari desain sistem pembelajaran yang adaptif dan efisien.

2.14 Fungsi Aktivasi

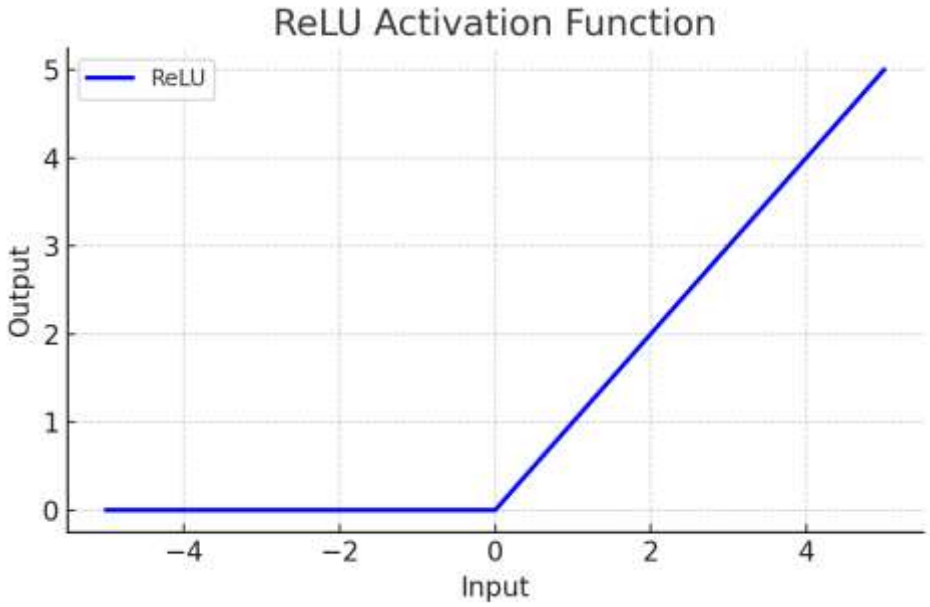
Fungsi aktivasi merupakan komponen esensial pada *machine learning* dan *deep learning*. Fungsinya adalah memperkenalkan sifat non-linear ke dalam sistem, yang memungkinkan model untuk mempelajari dan mengekspresikan hubungan kompleks antar fitur dalam data. Tanpa fungsi aktivasi, terlepas dari jumlah lapisan dalam jaringan, seluruh proses transformasi *input* ke *output* hanya akan bersifat linier. Hal ini membatasi kapasitas model dalam menyelesaikan permasalahan dunia nyata yang umumnya bersifat non-linear, seperti klasifikasi gambar, deteksi intrusi jaringan, atau pengenalan kata-kata. Secara matematis, fungsi aktivasi bekerja dengan memetakan *output* dari sebuah neuron ke dalam bentuk baru, sering kali melalui transformasi non-linier yang ditentukan oleh jenis fungsi yang digunakan. Transformasi ini memungkinkan jaringan untuk membentuk permukaan *loss* yang kompleks, sehingga proses optimasi dapat mengeksplorasi ruang solusi yang lebih kaya. Selain itu, fungsi aktivasi juga membantu menjaga stabilitas propagasi gradien selama pelatihan. Jika tidak ada fungsi aktivasi yang tepat, jaringan akan lebih rentan terhadap fenomena *vanishing gradient*, di mana nilai gradien mengecil secara drastis saat di propagasi ke lapisan awal, menghambat proses pembelajaran parameter pada lapisan-lapisan tersebut. Karakteristik ideal dari fungsi aktivasi meliputi kemampuan memperkenalkan non-linearitas tajam tanpa menyebabkan hilangnya informasi gradien, efisiensi komputasi, serta fleksibilitas dalam menyesuaikan bentuk aktivasi dengan karakteristik data. Aktivasi juga sebaiknya memiliki properti seperti kelicinan (*smoothness*) agar memudahkan optimasi berbasis gradien, serta distribusi *output* yang mendekati nol-mean untuk mempercepat pelatihan [45].



Gambar 2.12 Perbandingan Fungsi Aktivasi

Fungsi aktivasi yang awalnya hanya terbatas pada *Logistic Sigmoid* dan *Tanh* kini telah berkembang ke arah fungsi yang lebih stabil dan efisien, seperti ReLU dan turunannya. Evolusi ini didorong oleh kebutuhan untuk mengatasi berbagai keterbatasan fungsi lama, termasuk saturasi *output* dan komputasi eksponensial yang mahal. Studi komprehensif yang dilakukan dalam menunjukkan bahwa fungsi-fungsi aktivasi modern tidak hanya mempercepat proses pelatihan, tetapi juga memberikan keunggulan dalam hal akurasi, stabilitas, dan kemampuan generalisasi pada berbagai jenis data dan arsitektur jaringan [45].

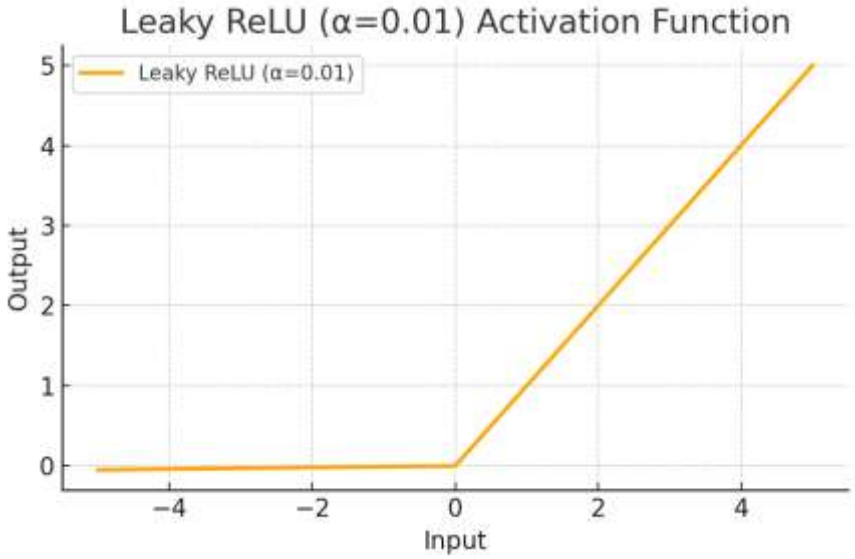
Dalam penelitian ini, empat fungsi aktivasi dipilih untuk dianalisis dan digunakan dalam konteks *deep learning*, yaitu *Rectified Linear Unit* (ReLU), *Leaky* ReLU, *Gaussian Error Linear Unit* (GELU), dan *Sigmoid-weighted Linear Unit* (SiLU). Keempat fungsi ini dipilih karena telah terbukti mampu mengatasi berbagai keterbatasan dari fungsi-fungsi aktivasi konvensional serta menunjukkan performa unggul dalam berbagai tugas pemodelan pada *neural network* berskala besar. Setiap fungsi aktivasi memiliki karakteristik unik dalam hal linearitas, utilisasi nilai negatif, kelicinan fungsi, dan kemampuan regularisasi implisit. Oleh karena itu, bagian berikutnya akan membahas secara terpisah masing-masing fungsi tersebut, mencakup formulasi dasar, kelebihan, serta relevansinya dalam praktik deep learning saat ini [45].



Gambar 2.13 Fungsi Aktivasi ReLu

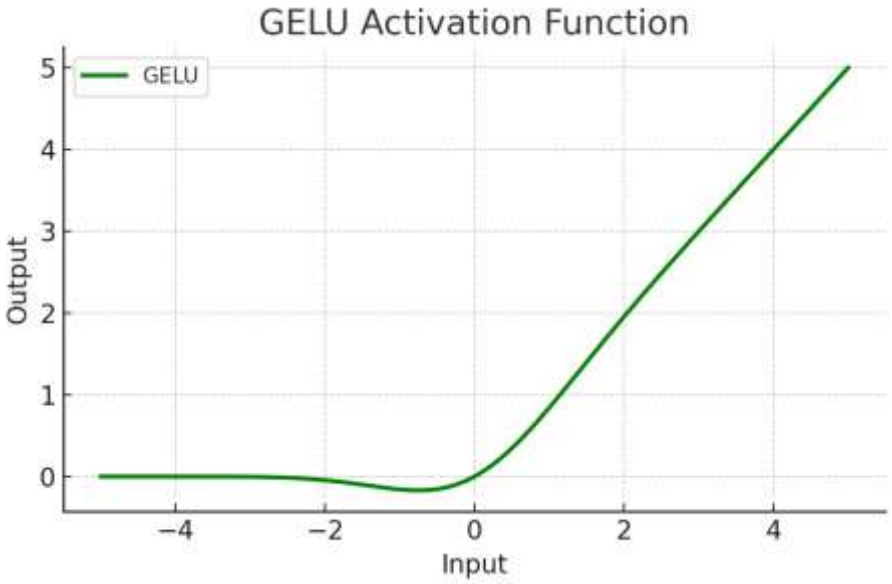
Rectified Linear Unit (ReLU) adalah fungsi aktivasi yang paling umum digunakan dalam *deep learning* karena kesederhanaannya dan efisiensi komputasinya. Fungsi ini didefinisikan sebagai f(x)=max(0,x), yang berarti hanya nilai *input* positif yang diteruskan, sementara *input* negatif diubah menjadi nol. Keunggulan utama ReLU terletak pada kemampuannya mengatasi masalah *vanishing gradient* yang umum terjadi pada fungsi aktivasi seperti *sigmoid* dan tanh, sehingga memungkinkan pelatihan neural *network* yang lebih dalam dan cepat. Selain itu, ReLU mendorong sparsitas aktivasi, yang dapat meningkatkan efisiensi representasi dan mempercepat konvergensi selama pelatihan. Namun, ReLU memiliki kelemahan dalam bentuk fenomena "*dying* ReLU", yaitu ketika neuron terus-menerus menerima *input* negatif sehingga

tidak pernah aktif dan berhenti belajar. Meskipun demikian, ReLU tetap menjadi fungsi aktivasi *default* dalam berbagai arsitektur karena memberikan keseimbangan yang baik antara akurasi, kecepatan, dan stabilitas pelatihan [45].



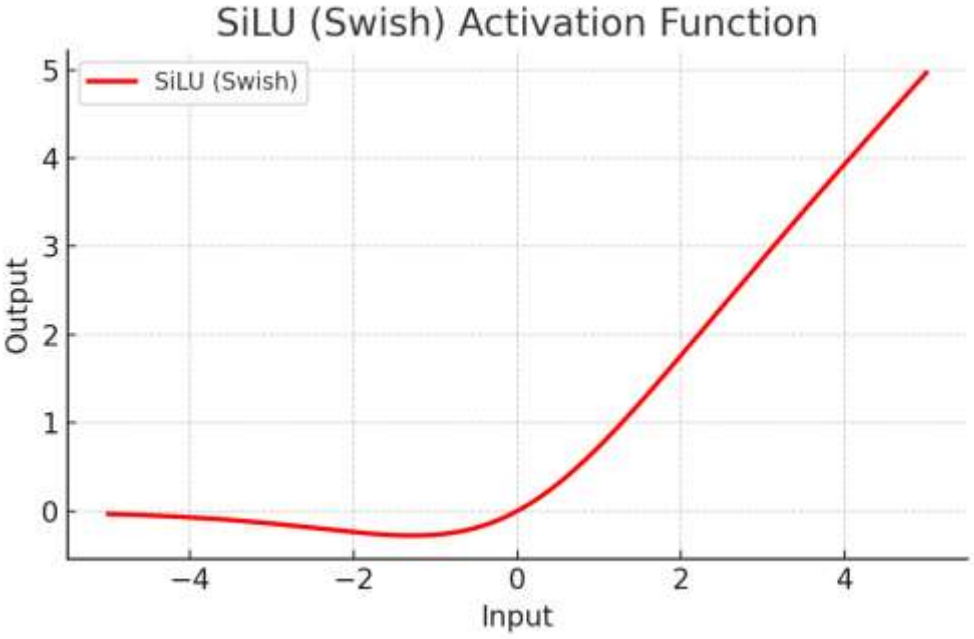
Gambar 2.14 Fungsi Aktivasi *Leaky ReLu*

Leaky Rectified Linear Unit (*Leaky* ReLU) merupakan pengembangan dari ReLU yang dirancang untuk mengatasi masalah *dying* ReLU, yaitu kondisi di mana neuron menjadi tidak aktif secara permanen akibat menerima *input* negatif secara terus-menerus. Leaky ReLU memperkenankan nilai negatif kecil untuk dilewatkan dengan mendefinisikan fungsi sebagai $f(x)=x$ jika $x>0$, dan $f(x)=\alpha x$ jika $x\leq 0$, dengan α biasanya bernilai kecil seperti 0.01. Dengan demikian, gradien tetap dapat mengalir meskipun *input* bernilai negatif, sehingga mengurangi risiko neuron berhenti belajar. Fungsi ini mempertahankan keunggulan ReLU dalam hal efisiensi komputasi dan kecepatan konvergensi, namun memberikan fleksibilitas tambahan pada domain negatif. Meskipun Leaky ReLU tidak memiliki parameter yang dapat disesuaikan secara otomatis, implementasinya yang sederhana dan performanya yang stabil menjadikannya alternatif populer dalam banyak arsitektur *deep learning*, khususnya ketika digunakan dalam jaringan dengan kedalaman besar atau data dengan distribusi yang tidak seimbang [45].



Gambar 2.15 Fungsi Aktivasi GeLu

Gaussian Error Linear Unit (GELU) adalah fungsi aktivasi yang menggabungkan pendekatan non-linearitas dengan karakteristik probabilistik. Berbeda dengan ReLU yang secara tegas memisahkan nilai negatif dan positif, GELU secara halus memodulasi output berdasarkan probabilitas bahwa *input* merupakan bagian dari distribusi Gaussian, sehingga menghasilkan transisi yang lebih lembut dan kontinu. Karakteristik ini membuat GELU sangat efektif dalam menjaga kestabilan gradien dan menghindari saturasi yang terlalu tajam. Fungsi ini juga bersifat halus (*smooth*) dan non-monotonik, memungkinkan jaringan untuk menyesuaikan tingkat aktivasi secara lebih fleksibel terhadap distribusi *input*. GELU telah digunakan secara luas dalam arsitektur *transformer* modern seperti BERT dan GPT, karena terbukti mampu meningkatkan akurasi sekaligus menjaga efisiensi pelatihan dalam skenario pembelajaran skala besar [45].



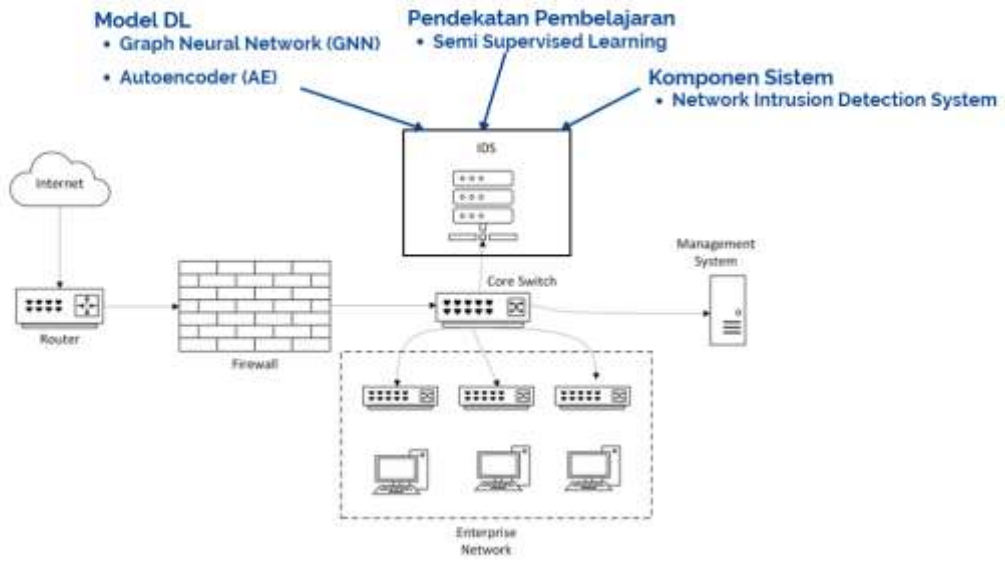
Gambar 2.16 Fungsi Aktivasi SiLU

Sigmoid-weighted Linear Unit (SiLU), juga dikenal sebagai Swish, merupakan fungsi aktivasi yang menggabungkan linearitas dan non-linearitas. Tidak seperti ReLU yang memotong semua nilai negatif, SiLU tetap mempertahankan kontribusi nilai negatif dengan bobot yang dilemahkan oleh sigmoid, menghasilkan kurva aktivasi yang halus dan dapat di diferensiasikan di seluruh domain. Fungsi ini bersifat non-monotonik dan *smooth*, yang memungkinkan propagasi gradien yang lebih stabil dan ekspresivitas jaringan yang lebih tinggi, terutama dalam model berskala besar. Keunggulan utama SiLU terletak pada kemampuannya menyeimbangkan antara efek linear dan non-linear secara dinamis, tanpa memerlukan parameter tambahan. Dalam berbagai eksperimen pada klasifikasi citra dan pemodelan bahasa, SiLU terbukti mampu mengungguli ReLU dan variannya dalam hal akurasi dan kecepatan konvergensi, serta memberikan regularisasi implisit yang membantu meningkatkan generalisasi model [45].

BAB 3 METODOLOGI

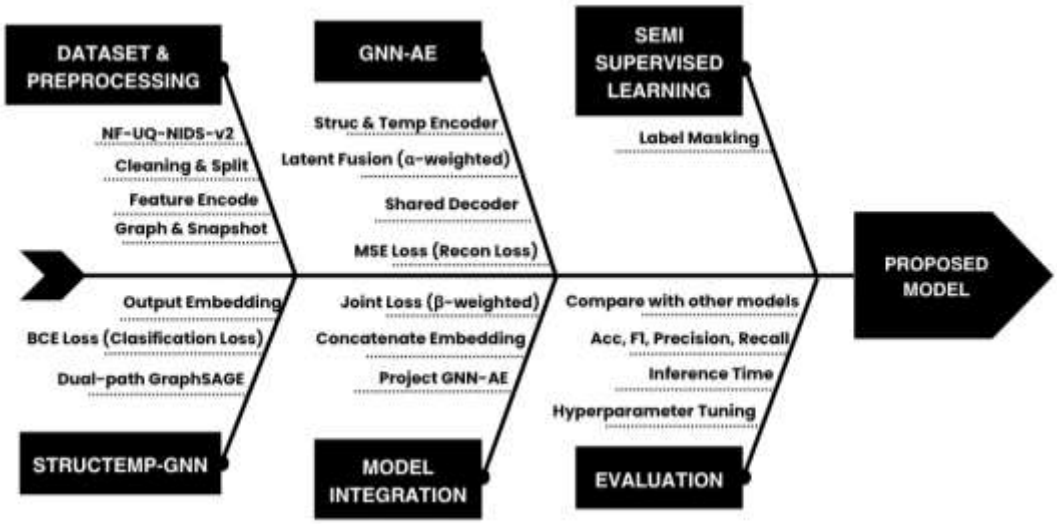
3.1 Metode yang digunakan

Penelitian ini mengembangkan sistem deteksi intrusi jaringan real-time berbasis *Graph Neural Network* (GNN) dengan menggunakan pendekatan *semi-supervised learning* dan aristektur *Structemp* untuk meningkatkan akurasi deteksi anomali pada jaringan yang dinamis. Metodologi ini dirancang untuk mengatasi tantangan utama dalam deteksi intrusi, seperti kompleksitas hubungan antar komponen jaringan, dinamika temporal yang tinggi, serta keterbatasan data berlabel.



Gambar 3.1 Arsitektur Penempatan IDS dalam Jaringan *Enterprise*

Rancangan sistem deteksi intrusi ini mempertimbangkan integrasi dengan infrastruktur jaringan yang ada serta alur metodologi yang sistematis untuk memastikan setiap tahap pengembangan mendukung pencapaian performa optimal. Pendekatan ini mencakup desain arsitektur jaringan hingga perumusan tahapan proses yang terstruktur dari awal hingga evaluasi.



Gambar 3.2 Gambar *Fishbone Diagram*

Tahapan awal dimulai dengan *preprocessing* data dari *dataset* untuk memastikan kualitas dan kesiapan *input*. Nilai-nilai tidak valid, seperti *NaN* dan tak hingga, diimputasi menggunakan metode substitusi dengan mengganti nilai tersebut menjadi nol lalu *row* tersebut didrop. Fitur kategorikal seperti jenis protokol dan *state* jaringan dikonversi ke bentuk numerik menggunakan teknik ordinal *encoding*. Sementara itu, fitur numerik dinormalisasi ke dalam rentang 0 sampai dengan 1 menggunakan *MinMaxScaler* agar tidak mendominasi model akibat skala besar.

Untuk membentuk *node* jaringan, kombinasi alamat IP dan *port* dari sumber serta tujuan digabung menjadi satu fitur lalu dihapus dari set fitur utama untuk mengurangi redundansi. Selanjutnya, digunakan mekanisme *sliding window* dengan ukuran *wind*ow 512 dan *overla*p 50%, sebagaimana diadaptasi dari studi Boukari et al. [9] Tujuan dari pemilihan parameter ini adalah untuk mengoptimalkan deteksi pola serangan yang bersifat temporal tanpa mengorbankan efisiensi komputasi.

Setiap *window* kemudian dikonversi menjadi graf dinamis yang merepresentasikan dua jenis hubungan struktural, yang menghubungkan *node* dengan *endpoi*nt yang sama, dan hubungan temporal, yang menghubungkan *node* dengan 50 aliran sebelumnya. Pembentukan *edge* temporal ini memungkinkan model mempelajari konteks historis, seperti lonjakan dalam jumlah paket atau perubahan frekuensi koneksi.

Model pada penelitian ini mengintegrasikan dua arsitektur utama, yaitu *Structemp*-GNN dan GNN-AE, dalam sebuah pendekatan *semi-supervised*. *Structemp*-GNN terdiri dari dua jalur pemrosesan paralel, yaitu jalur struktural yang mengekstraksi informasi topologi jaringan menggunakan tiga lapisan SAGEConv, dan jalur temporal yang menangkap dinamika perilaku jaringan dari waktu ke waktu menggunakan tiga lapisan SAGEConv tambahan. *Embedding* dari kedua jalur digabungkan untuk membentuk representasi struktural-temporal.

Untuk memanfaatkan data tak berlabel, model juga mengintegrasikan GNN *Autoencoder* (GNN-AE). *Encoder* terdiri atas dua jalur yaitu struktural dan temporal, masing-masing menggunakan dua lapisan SAGEConv yang memproyeksikan fitur *input* berdimensi 39 ke dalam ruang laten berdimensi 128. *Decoder* kemudian merekonstruksi kembali fitur *input* dari representasi laten tersebut menggunakan dua lapisan *SAGEConv* tambahan. Proses rekonstruksi ini menggunakan dua jalur struktur dan temporal, dan hasil akhirnya dibandingkan dengan *input* asli menggunakan *Mean Squared Error* (MSE). Deviasi yang tinggi pada hasil rekonstruksi dapat menandakan adanya anomali.

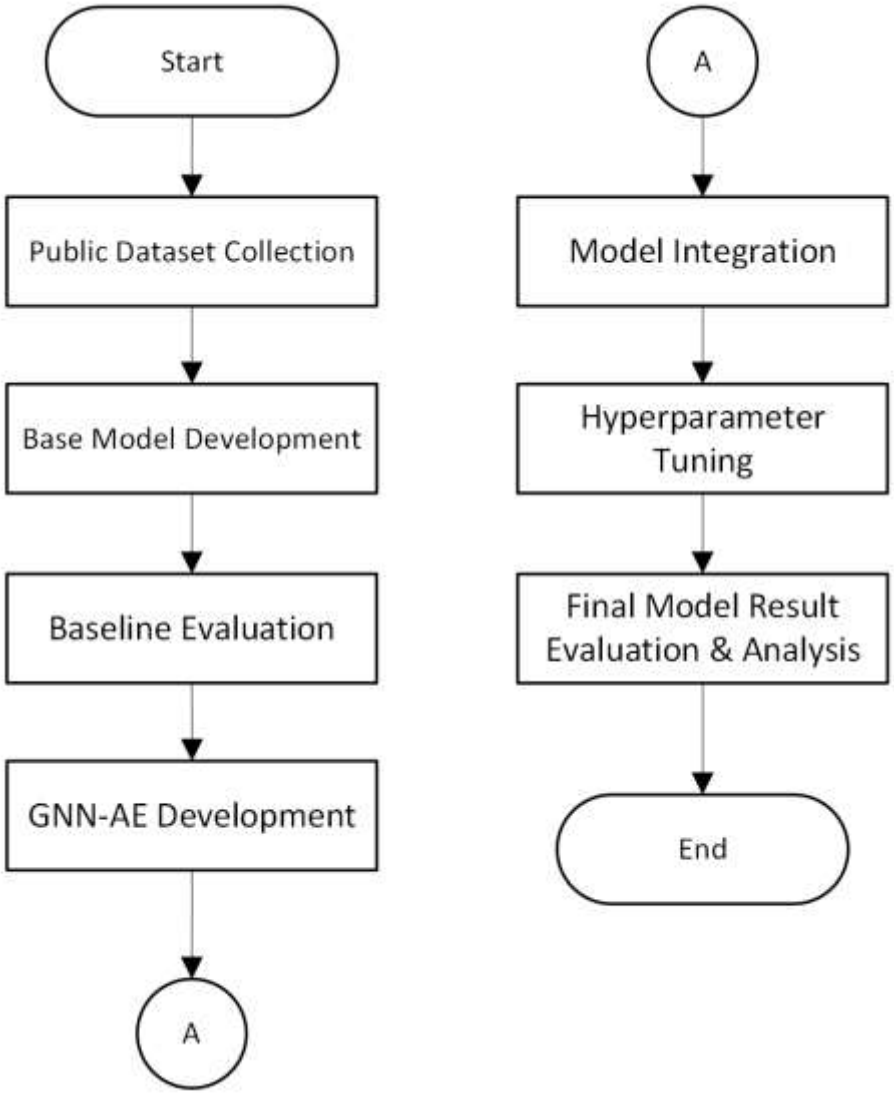
Representasi dari *StructTemp*-GNN dan GNN-AE diproyeksikan, digabungkan, lalu diklasifikasikan menggunakan dua lapisan *Feedforward Neural Network* (FNN) dengan aktivasi *ReLU*. Model dilatih secara *semi-supervised*, di mana 50% data digunakan sebagai data berlabel dan sisanya digunakan sebagai data tidak berlabel. *Binary Cross-Entropy Loss* digunakan untuk klasifikasi pada data berlabel, sedangkan MSE *Los*s digunakan untuk proses rekonstruksi *Autoencoder*. Kedua *loss* digabungkan menggunakan koefisien β.

Evaluasi dilakukan menggunakan metrik akurasi, presisi, recall, F1-score, serta waktu deteksi (*inference time*) untuk menilai efektivitas sistem dalam skenario real-time. Selain itu, *hyperparameter* penting seperti hidden_dim, latent_dim, num_layers, α (alpha), β (beta), *learning rate* (lr), *weight decay* (wd), fungsi aktivasi (*activation*), dan algoritma optimasi

(*optimizer*) dioptimalkan menggunakan *Bayesian Optimization* untuk memastikan performa terbaik dari model. Untuk melengkapi evaluasi, penelitian ini juga melakukan komparasi performa dengan dua model *baseline*, yaitu arsitektur *Structemp*-GNN berdasarkan studi sebelumnya, serta model *self-supervised* NEGSC. Hal ini bertujuan untuk menilai secara objektif kontribusi integrasi GNN-AE terhadap peningkatan performa sistem deteksi intrusi.

3.2 Urutan pelaksanaan penelitian

Alur penelitian yang digunakan dalam penelitian ini adalah sebagai berikut:



Gambar 3.3 Diagram Alir Penelitian

3.2.1 Pemilihan Data Publik

Dalam pengembangan sistem deteksi intrusi jaringan berbasis pembelajaran *semi-supervised*, pemilihan *dataset* berperan penting dalam memastikan bahwa model diuji secara menyeluruh dalam berbagai kondisi operasional. Penelitian ini menggunakan enam *dataset* publik yang telah dijelaskan secara rinci pada Bab 2, yaitu NF-UQ-NIDS-v2, NF-ToN-IoT-v2, NF-UNSW-NB15-v2, NF-ToN-IoT-v3, UNSW-NB15, dan HIKARI2021. Keenam *dataset* ini dipilih secara strategis karena mewakili variasi konteks jaringan yang luas, mencakup lingkungan *enterprise*, infrastruktur *Internet of Things* (IoT), serta trafik terenkripsi dan realistis.

Dataset NF-UQ-NIDS-v2 digunakan sebagai *dataset* utama dalam pelatihan dan validasi model. Hal ini didasarkan pada karakteristiknya yang menggabungkan beberapa sumber terpercaya dan memuat 20 jenis serangan yang berbeda, menjadikannya sangat relevan untuk pengujian model berbasis *Graph Neural Network* (GNN). Sementara itu, lima *dataset* lainnya digunakan sebagai sumber pengujian lanjutan untuk mengevaluasi kemampuan generalisasi model terhadap domain yang berbeda serta distribusi data yang tidak identik dengan data pelatihan.

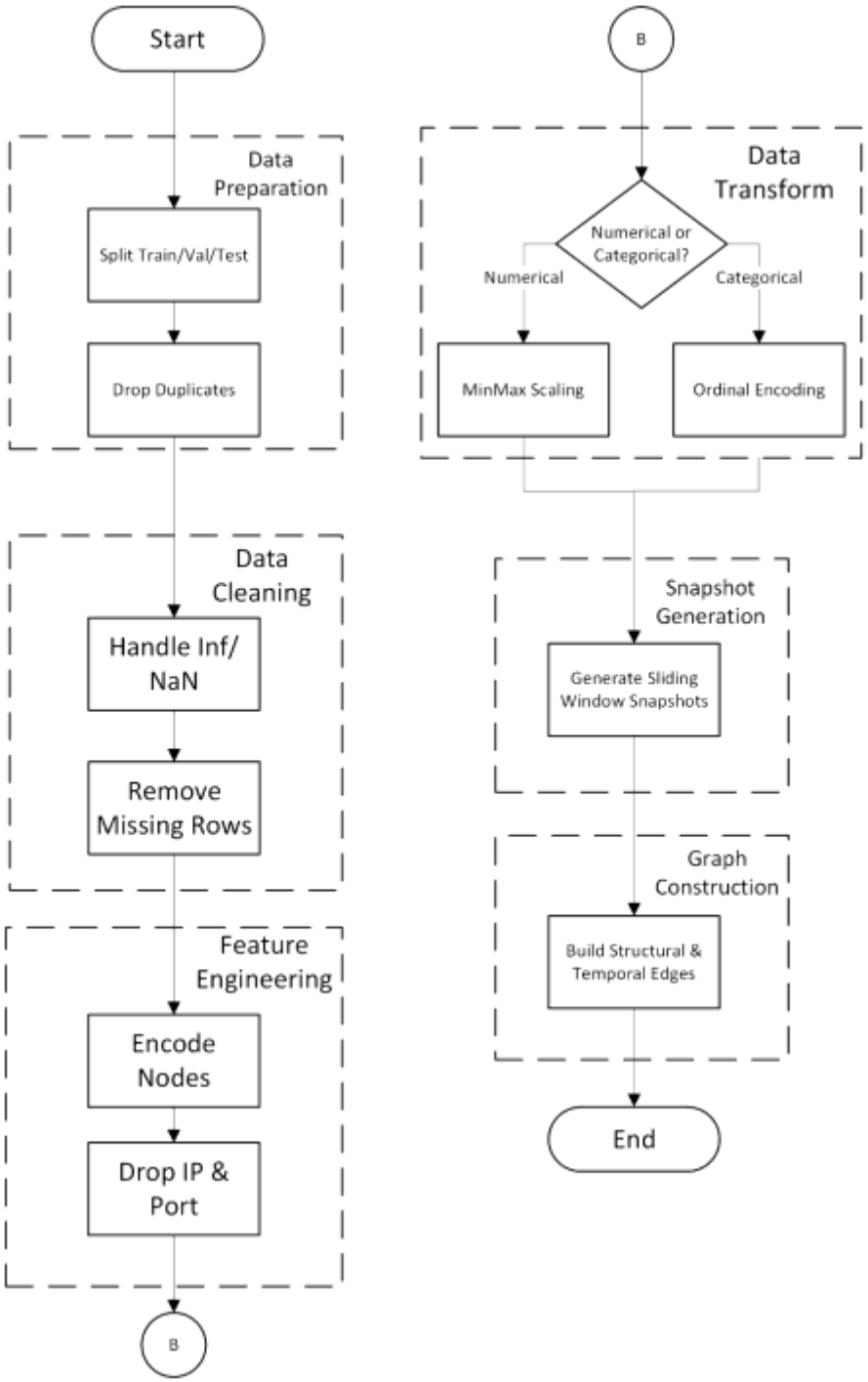
Namun, dalam pelaksanaan eksperimen, seluruh proses dilakukan menggunakan laptop pribadi yang dilengkapi dengan GPU NVIDIA RTX 3050 Laptop dengan VRAM sebesar 4 GB. Untuk menjaga efisiensi proses pelatihan dan menghindari waktu komputasi yang terlalu panjang, diterapkan kebijakan pembatasan jumlah data dari masing-masing *dataset*. Secara spesifik, digunakan maksimal 1.600.000 baris dari *dataset* yang memiliki jumlah entri lebih besar dari batas tersebut. Angka ini ditentukan berdasarkan pertimbangan praktis terhadap durasi pelatihan, efisiensi penggunaan memori, serta kestabilan sistem selama proses eksekusi model *semi-supervised*.

Pemilihan subset data dilakukan menggunakan teknik *stratified sampling*, yaitu metode pengambilan sampel terstratifikasi yang menjaga proporsi distribusi kelas (label serangan) agar tetap sebanding dengan data asli. Pendekatan ini dipilih untuk memastikan bahwa hasil evaluasi model tetap valid dan representatif, meskipun dilakukan pada skala data yang lebih kecil. Dengan demikian, strategi seleksi data dalam penelitian ini tidak hanya menyesuaikan dengan keterbatasan perangkat keras, tetapi juga memastikan kualitas eksperimen tetap terjaga dari segi akurasi, generalisasi, dan efisiensi.

Secara keseluruhan, strategi pemilihan dan pengelolaan data dalam penelitian ini dirancang untuk menjembatani kebutuhan metodologis terhadap cakupan skenario dunia nyata dengan keterbatasan sumber daya komputasi yang tersedia. Hal ini sekaligus mencerminkan tantangan riil dalam pengembangan sistem deteksi intrusi yang dapat diimplementasikan secara efisien pada perangkat dengan spesifikasi menengah, tanpa mengorbankan akurasi maupun ketahanan model terhadap variasi serangan.

3.2.2 Data *Preprocessing*

Pada tahapan data *preprocessing* dan konstruksi graf, dilakukan serangkaian prosedur yang bertujuan untuk mentransformasi data mentah menjadi representasi graf yang kompatibel dengan arsitektur *Graph Neural Network* (GNN). Secara umum, proses ini terdiri dari enam tahap utama: persiapan data (data *preparation*), pembersihan data (data *cleaning*), rekayasa fitur (*feature engineering*), transformasi data (data *transformation*), konstruksi graf (*graph construction*), dan pembuatan *snapshot* (*snapshot generation*). Setiap tahap saling berkesinambungan dalam memastikan bahwa data memiliki kualitas, struktur, dan skala yang sesuai untuk kebutuhan pemodelan berbasis graf.



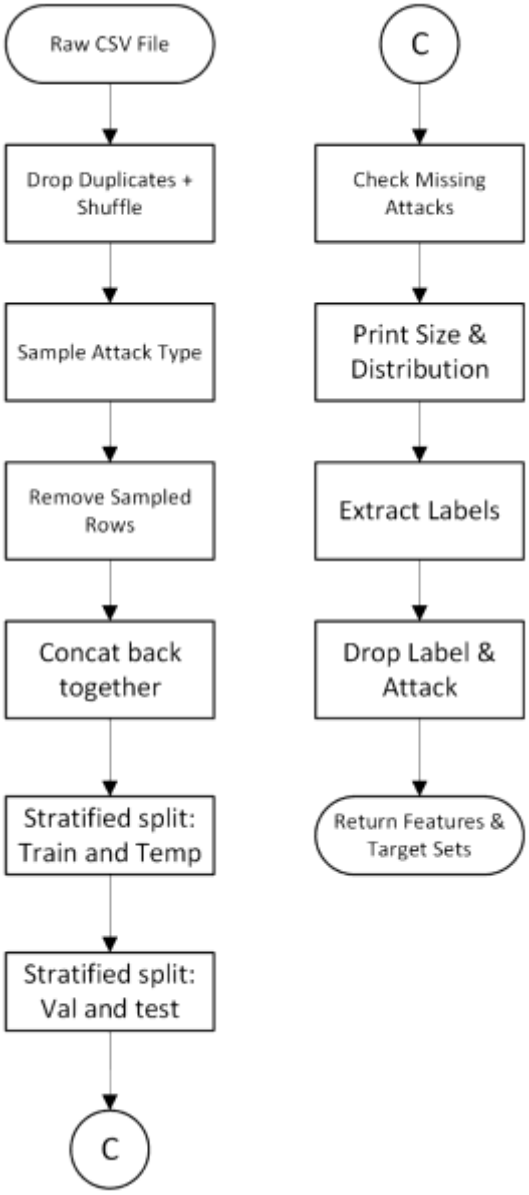
Gambar 3.4 Diagram Alur Global Data *preprocessing*

Rangkaian proses ini dirancang untuk mengatasi berbagai isu yang umum ditemukan pada data jaringan mentah, serta untuk membentuk struktur graf yang mampu merepresentasikan hubungan antar komponen dalam konteks spasial maupun temporal.

3.2.2.1 Data *Preparation*

Tahap data *preparation* merupakan langkah awal yang sangat krusial dalam proses data *preprocessing*, karena menentukan kualitas dan representasi data yang akan digunakan pada tahapan pemodelan selanjutnya. Proses ini berfungsi untuk menyusun dan menata *dataset*

mentah agar siap diproses lebih lanjut, dengan memastikan distribusi label yang seimbang, menghindari duplikasi data yang dapat menyebabkan data *leakage*, serta membagi *dataset* ke dalam subset yang sesuai untuk pelatihan, validasi, dan pengujian model. Pada tahap ini juga dilakukan langkah-langkah khusus untuk menjamin keterwakilan seluruh jenis serangan dalam data latih, sehingga model mampu mengenali setiap pola serangan yang ada. Penjelasan lengkap mengenai alur dan proses dalam tahap data *preparation* ditunjukkan pada uraian dan diagram berikut.



Gambar 3.5 Diagram Alur Data *Preparation*

Pada tahap data *preparation*, proses awal yang dilakukan bertujuan untuk menyiapkan *dataset* mentah agar berada dalam kondisi yang layak digunakan dalam proses pembelajaran model. *Dataset* yang digunakan berasal dari *file* CSV yang berisi jutaan entri trafik jaringan, termasuk label dan atribut-atribut penting seperti alamat IP, *port*, dan jenis serangan. Setelah data dimuat, langkah pertama yang dilakukan adalah menghilangkan data duplikat. Duplikasi dapat menyebabkan bias serta kebocoran informasi antar subset data, khususnya jika data yang

sama muncul di data latih dan data uji, yang akan memberikan estimasi performa model yang terlalu optimis.

Setelah data duplikat dihapus, seluruh baris data diacak menggunakan *seed* yang tetap untuk menjaga kemampuan pengulangan hasil. Proses ini penting untuk memastikan bahwa distribusi data tidak terpengaruh oleh urutan aslinya, terutama karena urutan dalam data trafik jaringan bisa berkaitan erat dengan waktu atau sesi tertentu yang dapat menyebabkan bias jika tidak diacak. Selanjutnya, dilakukan proses *stratified sampling* dengan mengambil satu entri secara acak dari masing-masing jenis serangan. Hal ini dilakukan untuk menjamin bahwa semua jenis serangan memiliki representasi minimal dalam data latih, sehingga model tidak kehilangan eksposur terhadap kelas-kelas minoritas yang mungkin jarang muncul. Setelah entri-entri sampel dikumpulkan, data yang tersisa dipisahkan dengan menghapus baris-baris yang telah diambil tadi, kemudian kembali diacak untuk meratakan distribusi.

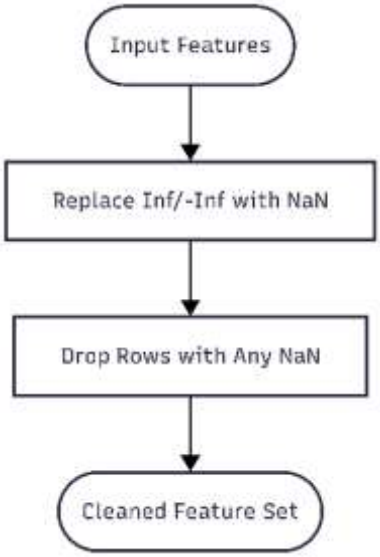
Data hasil penggabungan antara entri sampel dan data yang tersisa selanjutnya digunakan sebagai basis untuk proses *stratified splitting* ke dalam tiga subset: data latih (*training set*), validasi (*validation set*), dan pengujian (*test set*). Pemisahan dilakukan dalam dua tahap: pertama memisahkan data latih dari data sementara, lalu memisahkan data sementara menjadi data validasi dan data uji. Teknik stratifikasi digunakan untuk memastikan distribusi label (jenis serangan) tetap seimbang di seluruh subset.

Setelah proses pembagian data, dilakukan pengecekan untuk memastikan bahwa semua jenis serangan tetap terdistribusi secara merata, khususnya dalam data latih. Jika terdapat jenis serangan yang hilang, sistem akan mengeluarkan peringatan untuk menandai potensi ketidakseimbangan kelas yang dapat memengaruhi proses pelatihan. Selanjutnya, ukuran serta distribusi label dari setiap subset dicetak sebagai dokumentasi dan validasi proses pemisahan.

Sebagai tahap akhir, dilakukan pemisahan antara fitur dan target dari masing-masing subset. Kolom-kolom yang tidak diperlukan dalam proses pemodelan seperti Label, *Attack*, dan *Dataset* dihapus dari data fitur untuk memastikan bahwa tidak ada kebocoran informasi label selama pelatihan. Data yang telah disiapkan ini kemudian akan digunakan dalam proses *preprocessing* lanjutan sebelum dibentuk menjadi graf.

3.2.2.2 Data *Cleaning*

Tahap data *cleaning* merupakan bagian krusial dalam proses data *preprocessing* yang bertujuan untuk memastikan bahwa *dataset* yang akan digunakan dalam proses pelatihan model tidak mengandung elemen-elemen yang dapat mengganggu proses pembelajaran atau menghasilkan bias. Dalam konteks data jaringan, *dataset* mentah yang telah dipisahkan dan disiapkan masih sangat mungkin mengandung nilai-nilai yang tidak valid, seperti nilai tak terhingga, nilai hilang, maupun nilai numerik ekstrem yang berasal dari kesalahan pencatatan atau gangguan dalam proses pengumpulan data. Jika tidak ditangani secara sistematis, nilai-nilai ini dapat menyebabkan ketidakstabilan pada proses pelatihan model, memunculkan *NaN loss*, serta menurunkan performa dan generalisasi model terhadap data baru.



Gambar 3.6 Diagram Alur Data *Cleaning*

Langkah pertama dalam proses pembersihan data adalah identifikasi dan penanganan nilai tak hingga, baik positif (inf) maupun negatif (-inf). Nilai ini dapat muncul akibat anomali dalam data mentah atau sebagai hasil dari operasi matematis yang tidak terdefinisi, seperti pembagian oleh nol atau konversi logaritma terhadap nilai nol. Dalam banyak pustaka *deep learning*, termasuk PyTorch dan scikit-learn, keberadaan nilai tak hingga secara langsung akan menyebabkan fungsi-fungsi pemodelan gagal berjalan atau menghasilkan *runtime error*. Untuk itu, seluruh nilai inf dan -inf disubstitusi dengan nilai NaN agar dapat ditangani secara konsisten dalam proses selanjutnya.

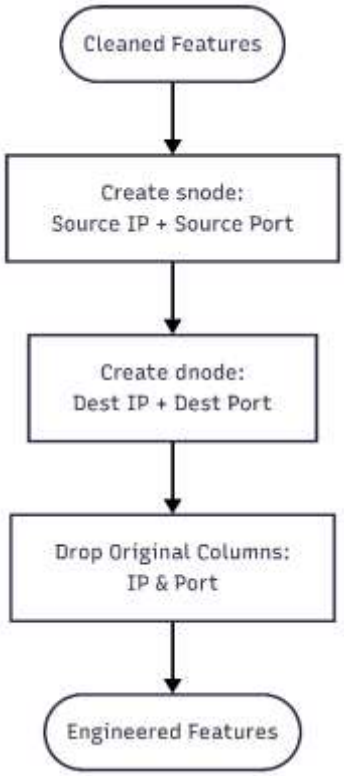
Setelah proses substitusi, seluruh baris data yang mengandung nilai NaN pada kolom manapun dihapus secara menyeluruh. Penghapusan ini menggunakan pendekatan *row-wise deletion*, yang lebih aman dibandingkan dengan *imputation*, mengingat sifat data jaringan yang kompleks dan tidak selalu memiliki distribusi yang dapat diestimasi secara akurat untuk pengisian nilai. Metode imputasi seperti mean atau median dapat memperkenalkan *noise* atau bias yang tidak dapat dikontrol dan berdampak langsung terhadap performa model pada skenario dunia nyata. Oleh karena itu, penghapusan nilai hilang merupakan kompromi terbaik antara kualitas data dan kompleksitas pemrosesan, khususnya karena ukuran *dataset* yang digunakan dalam penelitian ini masih mencukupi meskipun sebagian baris dihapus.

Selain itu, pembersihan data juga secara implisit meningkatkan efisiensi proses pembelajaran karena model tidak perlu melakukan pengecekan atau kompensasi terhadap nilai anomali selama proses *forward* maupun *backward propagation*. *Dataset* hasil proses *cleaning* ini memiliki integritas data yang lebih baik, tidak mengandung gangguan numerik, dan lebih stabil digunakan untuk tahap-tahap selanjutnya, seperti *feature engineering* dan transformasi ke dalam representasi graf. Dengan demikian, data *cleaning* menjadi fondasi yang sangat penting dalam memastikan kualitas hasil pelatihan model dan keandalan sistem deteksi intrusi yang dikembangkan..

3.2.2.3 *Feature Engineering*

Tahap *feature engineering* berperan penting dalam proses konversi data mentah menjadi bentuk representasi yang lebih informatif dan relevan untuk kebutuhan pemodelan berbasis

Graph Neural Network (GNN). Dalam konteks data jaringan, fitur-fitur seperti alamat IP dan nomor *port* tidak hanya bersifat numerik biasa, tetapi merepresentasikan komponen dan hubungan komunikasi antar *node* dalam jaringan. Oleh karena itu, perlu dilakukan proses ekstraksi fitur yang mampu menangkap struktur hubungan tersebut secara eksplisit. Tujuan utama dari tahap ini adalah membentuk representasi *node* yang dapat dipetakan dalam struktur graf melalui kombinasi atribut sumber dan tujuan, serta menghilangkan atribut mentah yang tidak lagi dibutuhkan setelah proses konstruksi identitas *node* selesai dilakukan.



Gambar 3.7 Diagram Alur *Feature Engineering*

Proses *feature engineering* dimulai dari himpunan fitur yang telah melewati proses pembersihan data sebelumnya, sehingga seluruh nilai yang tidak valid telah dieliminasi. Pada tahap ini, dilakukan pembentukan dua atribut baru yang merepresentasikan komponen sumber dan tujuan dalam komunikasi jaringan, yaitu s*node* (*source node*) dan d*node* (*destination node*). Atribut s*node* dibentuk melalui penggabungan antara alamat IP sumber (IPV4_SRC_ADDR) dan nomor *port* sumber (L4_SRC_PORT) menjadi sebuah *string* unik. Penggabungan ini bersifat deterministik dan konsisten untuk setiap komponen jaringan yang berperan sebagai pengirim dalam sesi komunikasi. Proses yang sama juga dilakukan terhadap alamat IP dan port tujuan untuk menghasilkan atribut d*node*, yang merepresentasikan penerima komunikasi.

Tujuan dari proses penggabungan ini adalah untuk membentuk identitas *node* yang unik berdasarkan pasangan alamat dan *port*, sehingga setiap simpul dalam graf dapat dikenali sebagai representasi dari satu komponen komunikasi yang spesifik. Dalam konteks data jaringan, kombinasi IP dan *port* menjadi elemen yang merepresentasikan tidak hanya identitas logis suatu perangkat, tetapi juga konteks sesi komunikasi yang sedang berlangsung. Oleh karena itu, pemisahan komponen berdasarkan kombinasi tersebut penting untuk memastikan

bahwa struktur graf yang dibentuk nantinya mencerminkan realitas hubungan dan arus data dalam jaringan.

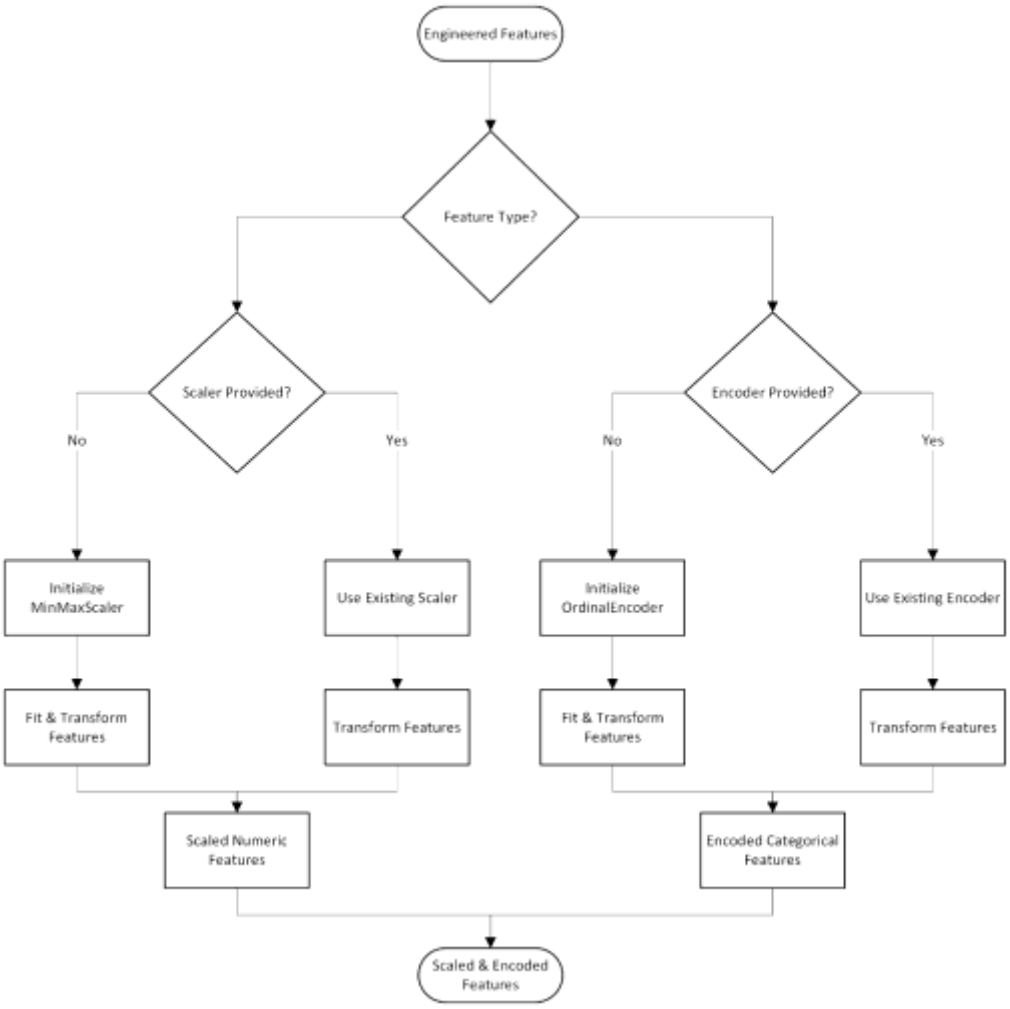
Setelah atribut s*node* dan d*node* terbentuk, atribut-atribut mentah yang digunakan dalam proses penggabungan tersebut dihapus dari *dataset*. Kolom-kolom seperti IPV4_SRC_ADDR, L4_SRC_PORT, IPV4_DST_ADDR, dan L4_DST_PORT tidak lagi diperlukan karena informasinya telah dikonversi ke dalam bentuk representasi *node* yang lebih terstruktur. Penghapusan ini juga membantu menghindari pengulangan data dan mengurangi risiko kebocoran semantik, di mana model dapat mengakses informasi yang tidak seharusnya tersedia secara eksplisit pada tahap pelatihan. Selain itu, penghapusan atribut ini juga bertujuan untuk menyederhanakan dimensi data yang akan diproses oleh model.

Hasil akhir dari tahap *feature engineering* adalah himpunan fitur yang telah diperkaya dengan struktur identitas *node* dan disederhanakan dari atribut-atribut mentah yang tidak lagi dibutuhkan. Representasi ini sangat penting sebagai fondasi untuk membentuk *edge* struktural dan temporal pada tahap *graph construction*. Dengan memastikan bahwa setiap komponen jaringan direpresentasikan sebagai *node* yang unik, model *Graph Neural Network* yang dibangun akan mampu memanfaatkan relasi antar *node* dengan lebih efektif, baik untuk menangkap pola komunikasi reguler maupun untuk mendeteksi anomali dalam bentuk intrusi jaringan. Tahap ini secara langsung berkontribusi pada keberhasilan proses konstruksi graf serta pada performa akhir dari sistem deteksi intrusi yang dikembangkan.

3.2.2.4 Data *Transformation*

Tahapan data *transformation* berfokus pada penyesuaian skala nilai numerik serta pengkodean nilai kategorikal dari fitur-fitur yang telah direkayasa sebelumnya agar sesuai dengan kebutuhan *input* model Graph Neural Network (GNN). Proses ini penting dilakukan untuk menghindari dominasi fitur dengan nilai besar terhadap proses pembelajaran dan untuk memastikan bahwa semua tipe data dapat diproses secara konsisten dalam arsitektur GNN. Normalisasi nilai numerik menjadi langkah penting dalam konteks *deep learning* karena banyak algoritma berbasis gradien sangat sensitif terhadap perbedaan skala antar fitur. Sementara itu, pengkodean nilai kategorikal diperlukan agar data diskrit dapat direpresentasikan dalam bentuk numerik yang dapat dipahami oleh model.

Pada tahap ini, fitur yang telah direkayasa pada tahap sebelumnya dipersiapkan untuk digunakan dalam pelatihan model melalui dua jalur transformasi utama, yaitu untuk fitur numerik dan fitur kategorikal. Jalur pertama menangani fitur numerik dengan teknik normalisasi MinMax Scaling, sedangkan jalur kedua menangani fitur kategorikal dengan teknik pengkodean OrdinalEncoder. Tujuannya adalah untuk memastikan bahwa semua fitur, baik numerik maupun kategorikal, memiliki kontribusi yang seimbang dalam proses propagasi informasi pada GNN. Ketidakseimbangan dalam skala atau representasi dapat menyebabkan ketidakstabilan gradien atau ketidakseimbangan pembobotan antar fitur dalam pembelajaran *node embedding*, yang pada akhirnya menurunkan performa model secara keseluruhan



Gambar 3.8 Diagram Alur Data *Transformation*

Secara teknis, proses ini dibagi menjadi dua alur berdasarkan apakah *scaler* telah disediakan dari proses sebelumnya. Jika belum (*branch* "No"), artinya transformasi dilakukan pertama kali, biasanya pada data pelatihan. Dalam hal ini, dilakukan inisialisasi objek MinMaxScaler dari pustaka sklearn.preprocessing, yang kemudian *fit* dan *transform* seluruh data fitur. Proses fit berguna untuk menghitung nilai minimum dan maksimum dari setiap fitur, sementara *transform* mengubah nilai-nilai fitur berdasarkan rumus normalisasi:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Hasil transformasi ini adalah fitur numerik dengan nilai berada pada rentang [0, 1], yang secara signifikan meningkatkan kecepatan konvergensi serta stabilitas komputasi saat menggunakan fungsi aktivasi non-linear seperti ReLU atau Sigmoid pada lapisan GNN. Jika *scaler* sudah disediakan, diasumsikan bahwa proses normalisasi telah dilakukan pada data pelatihan, dan informasi minimum serta maksimum fitur sudah tersimpan dalam objek *scaler*. Dalam kasus ini, proses hanya menerapkan *transform* pada subset data lainnya seperti *validation* set dan testing set. Pendekatan ini mencegah kebocoran data (*data leakage*) dan memastikan konsistensi skala di seluruh fase pelatihan dan evaluasi.

Sementara itu, pada jalur fitur kategorikal, proses serupa diterapkan menggunakan *OrdinalEncoder*. Jika objek encoder belum tersedia, dilakukan inisialisasi *OrdinalEncoder* yang kemudian fit dan *transform* seluruh data kategorikal untuk mengonversi nilai diskrit menjadi bilangan bulat yang merepresentasikan urutan kategori. Apabila encoder sudah tersedia, proses hanya menjalankan *transform* pada data baru menggunakan *mapping* yang sama untuk menjaga konsistensi representasi kategori di seluruh subset data. Konsistensi ini penting karena perbedaan antara *training* set dan testing set dapat menyebabkan model gagal mengenali kategori tertentu, sehingga menurunkan akurasi deteksi anomali.

Hasil dari tahap transformasi ini adalah dua komponen penting: data fitur yang telah dinormalisasi dan dikodekan, serta objek *scaler* dan *encoder* itu sendiri. Kedua objek ini akan disimpan dan digunakan kembali pada tahap evaluasi atau saat melakukan inferensi pada data baru, sehingga memastikan proses yang deterministik dan dapat direplikasi. Dengan skema seperti ini, *pipeline* data pada sistem deteksi intrusi berbasis GNN tetap terjaga integritasnya dari tahap *preprocessing* hingga tahap *deployment*, mendukung implementasi sistem yang handal dan adaptif terhadap data trafik jaringan yang terus berubah..

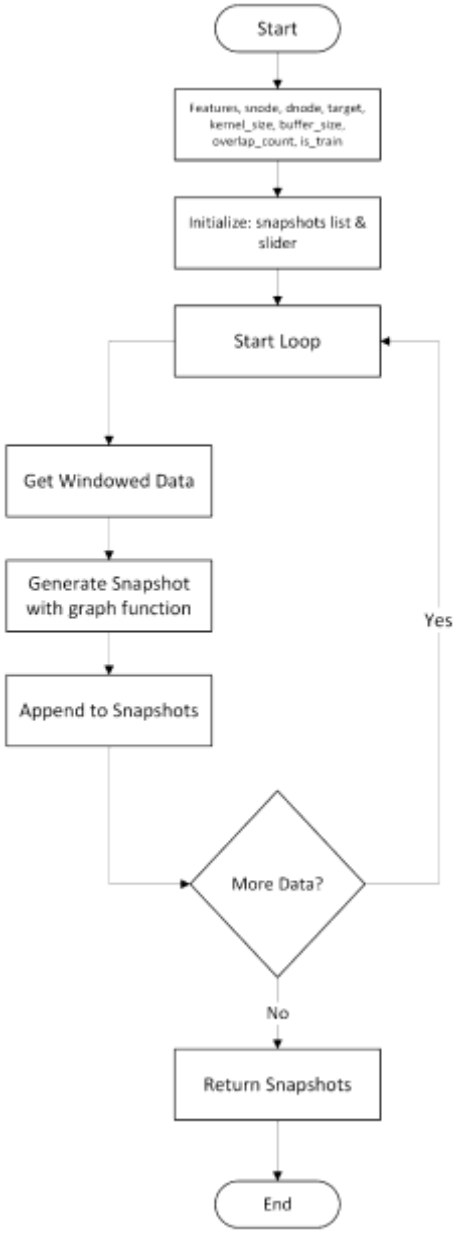
3.2.2.5 Snapshot Generation

Salah satu tahap penting dalam *preprocessing* data sebelum proses *graph construction* adalah pembentukan *snapshot*, yang bertujuan untuk membagi data flow hasil ekstraksi fitur ke dalam unit-unit lebih kecil berdasarkan jendela waktu (*time window*). Tujuan utama dari proses ini adalah untuk memungkinkan model *Graph Neural Network* (GNN) mempelajari dinamika temporal secara bertahap, tanpa harus memproses keseluruhan data sekaligus. Hal ini sangat relevan dalam konteks deteksi intrusi, karena pola serangan jaringan umumnya berlangsung secara bertahap dan memiliki karakteristik temporal yang tidak langsung muncul secara instan.

Proses dimulai dengan menerima *input* berupa fitur *node* (features), indeks *node* sumber dan tujuan (s*node*, d*node*), label target (target), serta parameter jendela seperti kernel_size, buffer_size, overlap_count, dan flag is_train. Kemudian dilakukan inisialisasi variabel penting, yaitu list kosong untuk menyimpan snapshot dan pengatur posisi jendela (*slider*). Sistem kemudian memasuki proses iteratif untuk menelusuri data berdasarkan ukuran dan pergeseran jendela waktu yang ditentukan.

Dalam setiap iterasi, data dipotong menggunakan metode *sliding window* untuk menghasilkan potongan data sesuai ukuran kernel_size dan buffer_size. Data yang terpotong ini mencakup fitur *node* dan informasi konektivitas antar *node* berdasarkan urutan waktu. Selanjutnya, fungsi graph() digunakan untuk membentuk graf berdasarkan potongan tersebut, dengan membangun kembali sisi-sisi struktural dan temporal yang relevan. Graf hasil dari tiap jendela kemudian ditambahkan ke dalam *list snapshot*.

Proses ini terus berjalan selama masih terdapat data yang dapat diproses oleh jendela waktu berikutnya. Setiap langkah menjaga kesinambungan temporal dengan menggunakan overlap_count, sehingga *snapshot* yang dihasilkan memiliki sebagian informasi yang tumpang tindih antar jendela, memungkinkan model untuk mengenali transisi waktu secara halus. Setelah seluruh data selesai diproses, *list* yang berisi semua *snapshot* dikembalikan sebagai *output* dan akan digunakan pada tahap berikutnya, yaitu konstruksi graf dan pelatihan model.

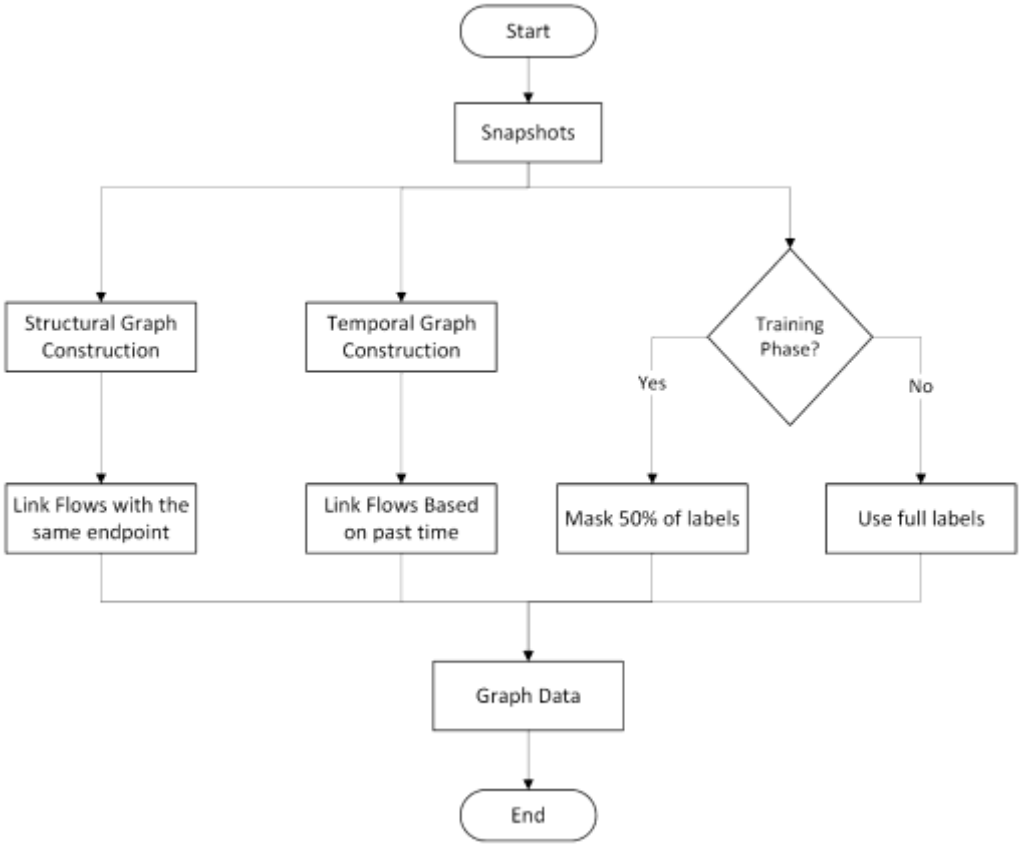


Gambar 3.9 Diagram Alur *Snapshot Generation*

Pendekatan ini mengacu pada *framework Structemp*-GNN yang diperkenalkan oleh Boukari et al. [9], di mana *snapshot* digunakan untuk mengadaptasi dinamika jaringan ke dalam bentuk graf heterogen dinamis. Dengan menyisipkan konteks waktu dalam pembentukan unit *input*, model deteksi intrusi menjadi lebih responsif terhadap serangan bertahap seperti infiltrasi lambat, serangan berulang, maupun anomali yang tersembunyi secara temporal.

3.2.2.6 Graph Construction

Pada tahapan ini, data fitur yang telah diproses sebelumnya akan diubah ke dalam representasi graf yang dapat diproses oleh model *Graph Neural Network* (GNN). Proses ini penting karena GNN membutuhkan struktur hubungan antar *node* untuk dapat mempelajari konteks dan interaksi secara efektif. Representasi graf yang digunakan mencakup dua jenis ketergantungan utama yaitu struktural dan temporal, yang masing-masing dibangun melalui proses berbeda.



Gambar 3.10 Diagram Alur *Graph Construction*

Proses konstruksi graf dalam sistem ini dilakukan dengan pendekatan paralel untuk membentuk dua jenis hubungan utama antar flow, yaitu graf struktural dan graf temporal. Pada konstruksi graf struktural, setiap flow yang terekam dalam data jaringan dikonversi menjadi sebuah *node*. *Edge* dibentuk dengan menghubungkan *node-node* yang memiliki kesamaan pada *endpoint*, seperti alamat IP sumber atau tujuan yang identik. Relasi ini menggambarkan keterkaitan topologis antar komponen dalam jaringan yang berpotensi saling berinteraksi, dan penting untuk menangkap pola komunikasi yang tidak biasa dalam konteks serangan lateral maupun internal.

Secara bersamaan, dilakukan konstruksi graf temporal untuk menangkap dinamika waktu dari trafik jaringan. *Flow* yang terjadi lebih awal dalam jendela waktu tertentu akan dihubungkan secara berarah ke *flow* yang muncul setelahnya. Proses ini menghasilkan *edge* temporal yang menyimpan informasi urutan kronologis antar aktivitas jaringan. Dengan membentuk koneksi berdasarkan waktu, model dapat mengidentifikasi pola sekuensial seperti serangan bertahap, aktivitas anomali yang meningkat, atau propagasi trafik yang mencurigakan dari satu titik ke titik lainnya dalam kurun waktu tertentu.

Setelah kedua jenis *edge* selesai dibentuk, sistem akan mengevaluasi konteks eksekusi model. Apabila sistem berada dalam fase pelatihan, maka sebagian label akan disembunyikan melalui teknik *masking* sebesar 50% guna menyimulasikan kondisi *semi-supervised learning*. Hal ini mendorong model untuk belajar dari struktur data dan hubungan antar *node*, meskipun informasi label tidak tersedia secara penuh. Sebaliknya, apabila proses berada pada tahap

validasi atau pengujian, maka seluruh label akan digunakan tanpa *masking* untuk keperluan evaluasi performa klasifikasi model secara menyeluruh.

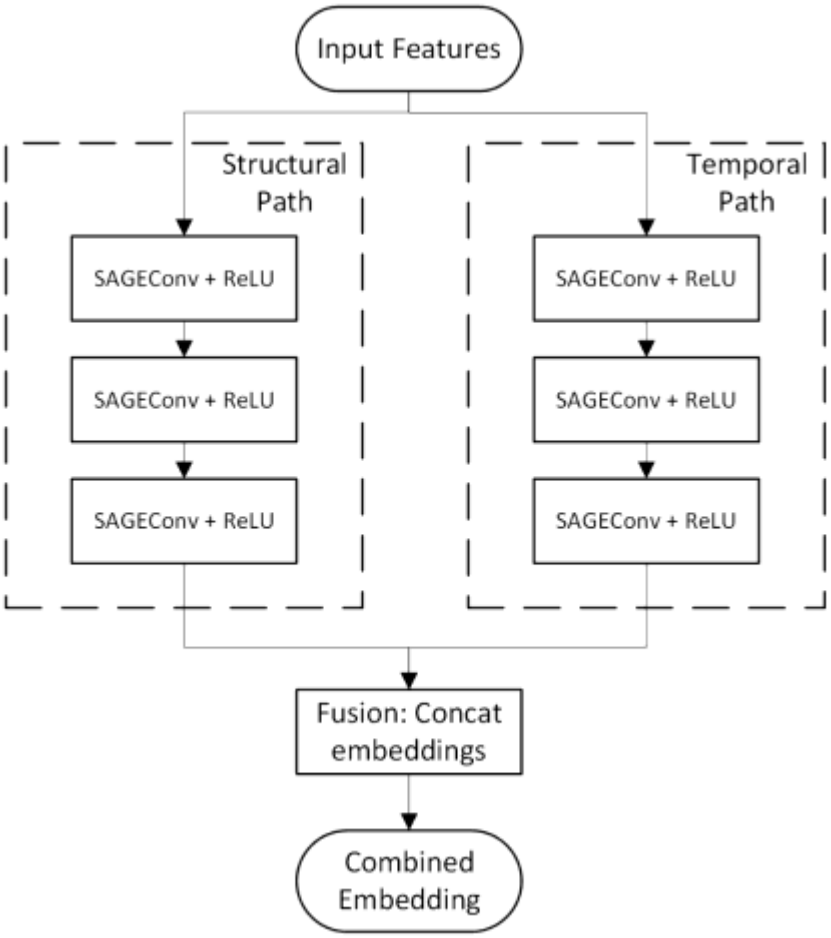
Semua komponen hasil dari tahap ini termasuk *node* features, *edge* index untuk koneksi struktural dan temporal, serta label (atau *label mask*) akan dikonsolidasikan menjadi satu objek data graf. Objek inilah yang akan menjadi *input* utama untuk model *Graph Neural Network* pada tahap selanjutnya, memungkinkan proses *message passing* dan pembelajaran representasi yang menggabungkan informasi topologis dan temporal secara efisien.

3.2.3 Pengembangan Model Dasar

Tahapan ini difokuskan pada pembangunan model awal dengan menggunakan arsitektur dasar *Structemp* GNN, yang menjadi fondasi dalam proses pengembangan sistem deteksi intrusi berbasis *Graph Neural Network*. Pemilihan arsitektur ini dilandaskan pada kemampuannya dalam memproses informasi spasial dan temporal secara paralel, memungkinkan model untuk memahami pola hubungan antar entitas jaringan sekaligus perubahan perilaku lalu lintas seiring waktu.

Proses pelatihan dimulai dengan masukan berupa fitur hasil ekstraksi dari data jaringan. Arsitektur *Structemp* GNN terdiri atas dua jalur utama, yaitu *Structural Path* dan *Temporal Path*. Jalur struktural bertugas mengolah informasi topologi jaringan menggunakan tiga lapisan *convolutional* (*SAGEConv*) berturut-turut yang masing-masing diikuti oleh fungsi aktivasi ReLU. Jalur ini memungkinkan model menangkap hubungan spasial antar node dalam satu snapshot graf. Secara paralel, *Temporal Path* memproses data temporal menggunakan tiga lapisan *convolutional* (SAGEConv), yang juga masing-masing dilengkapi dengan aktivasi ReLU. Jalur ini penting untuk mengenali pola temporal yang muncul antar *snapshot* graf secara berurutan.

Setelah kedua jalur selesai melakukan proses propagasi informasi, hasil dari kedua *path* tersebut digabungkan melalui proses *concatenation* untuk menghasilkan *combined embedding*. Representasi gabungan ini menjadi fitur akhir yang merepresentasikan baik konteks struktural maupun temporal dari data jaringan, dan selanjutnya digunakan oleh layer klasifikasi untuk melakukan identifikasi terhadap aktivitas anomali atau normal.



Gambar 3.11 Arsitektur *Baseline* Model

Dengan struktur arsitektur paralel ini, model dapat memproses data secara efisien tanpa mengorbankan konteks penting dari dua dimensi utama dalam data jaringan. Pengembangan model dasar ini juga menjadi acuan dalam tahap berikutnya yang akan melibatkan penambahan *autoencoder* dan pendekatan *semi-supervised learning*.

3.2.4 Evaluasi *Baseline*

Evaluasi *baseline* dilakukan untuk mengukur performa awal model *Structemp*-GNN sebelum dilakukan pengembangan lebih lanjut dengan integrasi modul *Graph Neural Network Autoencoder* (GNN-AE). Tahapan ini bertujuan untuk menyediakan tolok ukur kinerja yang dapat digunakan sebagai acuan perbandingan terhadap hasil dari model akhir yang telah di optimasi.

Pada tahap ini, *dataset* NF-UQ-NIDS-v2 digunakan sebagai data uji coba utama. *Dataset* tersebut telah melalui proses normalisasi fitur ke rentang [0,1] menggunakan MinMaxScaler, pembuatan *snapshot* dengan teknik *sliding window* (buffer size 512 dan overlap 256), serta konstruksi *graph* yang menggabungkan *edge* struktural dan temporal untuk menangkap relasi antar aliran trafik jaringan. Setelah *preprocessing*, *dataset* dipisahkan menjadi tiga subset dengan proporsi 60% untuk *training* set, 20% untuk *validation* set, dan 20% untuk *testing* set. Pembagian dilakukan secara *stratified* untuk memastikan distribusi kelas yang seimbang pada setiap subset.

Model *Structemp*-GNN dilatih dengan konfigurasi *hyperparameter default* sebagaimana diadopsi dari penelitian Boukari et al. [9]. Konfigurasi ini mencakup jumlah neuron per layer sebanyak 64, ukuran *embedding* sebesar 32, serta kombinasi filter SAGEConv pada setiap blok GNN. Pelatihan dilakukan selama 15 *epoch* menggunakan *optimizer* Adam dengan *variable learning rate* yang memiliki nilai awal 0.001 dan penurunan secara eksponensial. Proses pelatihan dilaksanakan pada lingkungan eksekusi lokal yang dilengkapi GPU NVIDIA RTX 3050 untuk mempercepat komputasi.

Evaluasi kinerja model dilakukan setelah proses pelatihan selesai, dengan menguji performa model pada *validation* set untuk memantau *overfitting* dan kemudian pada testing set untuk memperoleh hasil akhir. Metrik yang digunakan untuk menilai kinerja model meliputi akurasi, presisi, recall, F1-Score, serta waktu inferensi per snapshot dan per flow untuk menilai kelayakan model pada skenario real-time. Perhitungan metrik dilakukan menggunakan pustaka Scikit-Learn yang menyediakan fungsi-fungsi evaluasi standar seperti accuracy_score, precision_score, recall_score, dan f1_score.

3.2.5 Pengembangan *Autoencoder* berbasis GNN (GNN_AE)

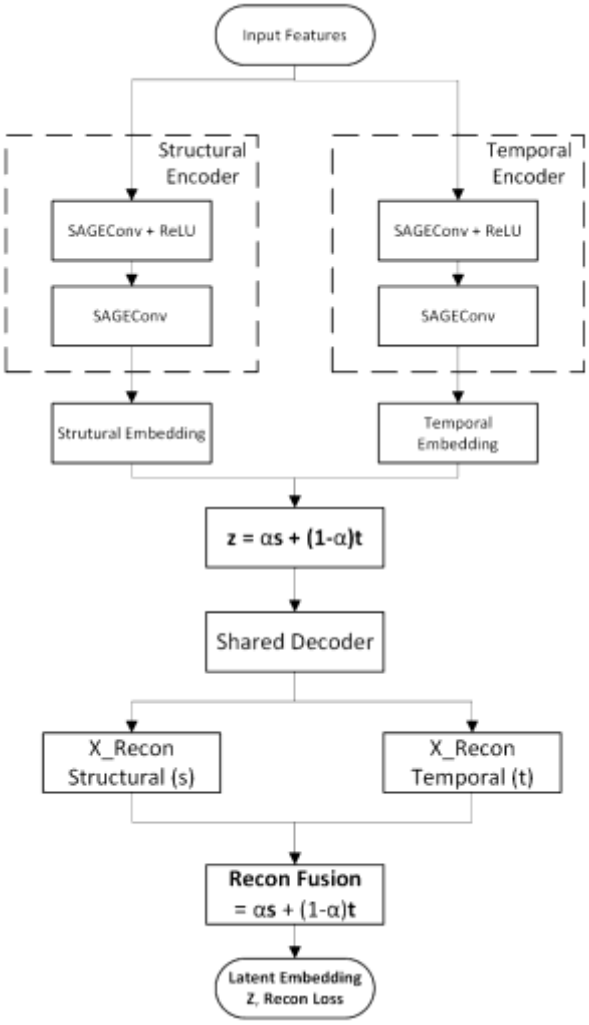
Pada tahap ini, dikembangkan arsitektur *Autoencoder* berbasis *Graph Neural Network* (GNN-AE) sebagai salah satu komponen utama dalam sistem *semi-supervised learning* untuk deteksi intrusi jaringan. Model ini dirancang untuk melakukan *encoding* dan *decoding* terhadap data jaringan berbentuk graf, dengan tujuan menghasilkan representasi laten yang kaya informasi untuk mendukung proses klasifikasi pada arsitektur keseluruhan.

GNN-AE menerima *input* berupa fitur node (x), indeks *edge* struktural, dan indeks *edge* temporal, yang masing-masing mencerminkan hubungan topologis dan kronologis antar komponen jaringan. Proses *encoding* dilakukan secara paralel melalui dua jalur, yaitu *encoder* struktural dan temporal, yang masing-masing terdiri atas dua lapisan SAGEConv dengan aktivasi ReLU untuk menangkap non-linearitas. Hasil *embedding* dari kedua jalur digabungkan menggunakan mekanisme fusi linier berbobot, menghasilkan vektor laten z yang selanjutnya digunakan untuk proses *decoding*.

Decoder memproyeksikan kembali *embedding* ke ruang fitur asli dengan menggunakan parameter bersama (*shared weights*) dan menghasilkan dua hasil rekonstruksi: x_recon_s (berdasarkan koneksi struktural) dan x_recon_t (berdasarkan koneksi temporal). Kedua hasil rekonstruksi ini kemudian difusikan kembali untuk memperoleh prediksi akhir x_recon. Selisih antara *input* awal x dan hasil rekonstruksi x_recon dihitung sebagai *reconstruction loss* menggunakan *Mean Squared Error* (MSE), yang berfungsi untuk memperkaya representasi laten agar lebih menyeluruh.

Arsitektur GNN-AE memulai proses *encoding* dengan dua jalur paralel: jalur encoder struktural dan jalur encoder temporal. Jalur struktural menggunakan dua lapisan konvolusi SAGEConv secara berturut-turut. Lapisan pertama mengubah dimensi *input* ke representasi laten sementara dan mengaktifkannya menggunakan fungsi aktivasi ReLU untuk menangkap non-linearitas dalam pola konektivitas. Hasilnya kemudian diteruskan ke lapisan konvolusi kedua yang memproyeksikan informasi ke dalam ruang laten berdimensi lebih rendah, menghasilkan *embedding* yang merepresentasikan hubungan struktural antar node berdasarkan koneksi yang ditentukan oleh *struct_edge_index*. Secara paralel, jalur temporal memproses *input* yang sama namun dengan *temp_edge_index*. Arsitektur dan prosesnya serupa dengan

jalur struktural, tetapi fokus pada urutan kronologis kejadian antar *flow*. Jalur ini menangkap dinamika waktu dalam trafik jaringan, seperti hubungan sebab-akibat antar paket atau jejak aktivitas pengguna yang menyebar dalam durasi tertentu. Dengan memisahkan pemrosesan struktural dan temporal, model dapat secara eksplisit mempelajari dua aspek berbeda dari data jaringan yang saling melengkapi untuk memperkaya representasi laten.



Gambar 3.12 Arsitektur Model GNN-AE

Hasil akhir dari kedua jalur *encoder* tersebut adalah dua vektor *embedding*, satu dari sisi struktural dan satu dari sisi temporal. Kedua *embedding* ini kemudian digabungkan melalui mekanisme fusi linier berbobot sebagai berikut:

$$\boldsymbol{z} = \alpha \boldsymbol{s} + (1 - \alpha)\boldsymbol{t},$$

di mana α adalah *hyperparameter* fusi yang menentukan seberapa besar kontribusi masing-masing *embedding*.

Setelah *embedding* gabungan z diperoleh, proses *decoding* dilakukan melalui dua jalur lagi: satu untuk rekonstruksi berdasarkan *edge* struktural dan satu lagi untuk edge temporal. Kedua jalur decoder menggunakan parameter yang dibagi (*shared weights*), sehingga memanfaatkan lapisan SAGEConv yang sama untuk memproyeksikan *embedding* laten kembali ke ruang fitur asli. Setiap jalur *decoding* terdiri dari dua lapisan: lapisan pertama menggunakan aktivasi ReLU untuk menghasilkan representasi antara, dan lapisan kedua menggunakan fungsi sigmoid agar

output berada dalam rentang [0,1], sesuai dengan hasil transformasi MinMax pada tahap *preprocessing*. *Decoder* menghasilkan dua output: x_recon_s dan x_recon_t, yang merupakan hasil rekonstruksi berdasarkan koneksi struktural dan temporal.

Sebagai tahap akhir, kedua hasil rekonstruksi ini difusikan kembali menggunakan rumus linier berbobot yang serupa, untuk menghasilkan x _recon, yaitu prediksi akhir dari *input* asli. Proses ini mencerminkan kemampuan model dalam mempelajari representasi laten yang memperkuat proses klasifikasi pada arsitektur *semi-supervised*. Perbedaan antara x dan x_recon dihitung sebagai MSE loss yang digunakan selama pelatihan untuk memperbarui bobot parameter melalui *backpropagation*. Dengan pendekatan ini, GNN-AE berperan sebagai komponen pembelajaran representasi yang mendukung generalisasi model terhadap variasi pola trafik jaringan, termasuk potensi serangan baru.

3.2.5 Integrasi Model

Integrasi model *Structemp*-GNN dengan *Autoencoder* (AE) berbasis *Graph Neural Network* (GNN) dirancang untuk memperkaya representasi fitur dan meningkatkan kemampuan generalisasi model melalui kombinasi pembelajaran tersupervisi dan *semi-supervised.* Arsitektur ini menerima *input* data graf yang telah diproses sebelumnya, terdiri dari sisi struktural (hubungan antar komponen berdasarkan topologi jaringan) dan sisi temporal (urutan kejadian berdasarkan waktu).

Pada tahap pertama, data graf tersebut diproses secara paralel melalui dua komponen utama. Pertama, komponen *Structemp*-GNN yang telah ada sebelumnya, menggunakan lapisan SAGEConv bertingkat untuk mengekstrak pola struktural dan temporal secara terpisah. Setiap lapisan SAGEConv dilengkapi dengan aktivasi ReLU, menghasilkan *embeddings* yang merepresentasikan hubungan logis antar komponen dan dinamika waktu secara mendalam. Kedua, komponen GNN-AE sebagai kontribusi baru berfungsi mengompresi data ke dalam ruang laten (*latent space*) melalui proses *encoding* dan *decoding*. Pada tahap ini, parameter adaptif α digunakan untuk menggabungkan representasi struktural dan temporal secara dinamis, memastikan fusi informasi yang optimal sesuai dengan karakteristik data.

Keluaran dari kedua komponen kemudian dipadukan dalam satu ruang fitur yang terintegrasi. *Embeddings* dari *Structemp*-GNN, yang memiliki dimensi tinggi (hidden_dim×2), diproyeksikan ke dimensi yang sesuai, sementara representasi laten dari GNN-AE (*latent_dim*) diproyeksikan melalui lapisan linear untuk menyelaraskan skalanya. Hasil penggabungan ini menghasilkan vektor fitur yang diperkaya (*hidden_dim*×4), yang menggabungkan informasi struktural-temporal dengan pola laten abstrak yang ditangkap oleh AE.

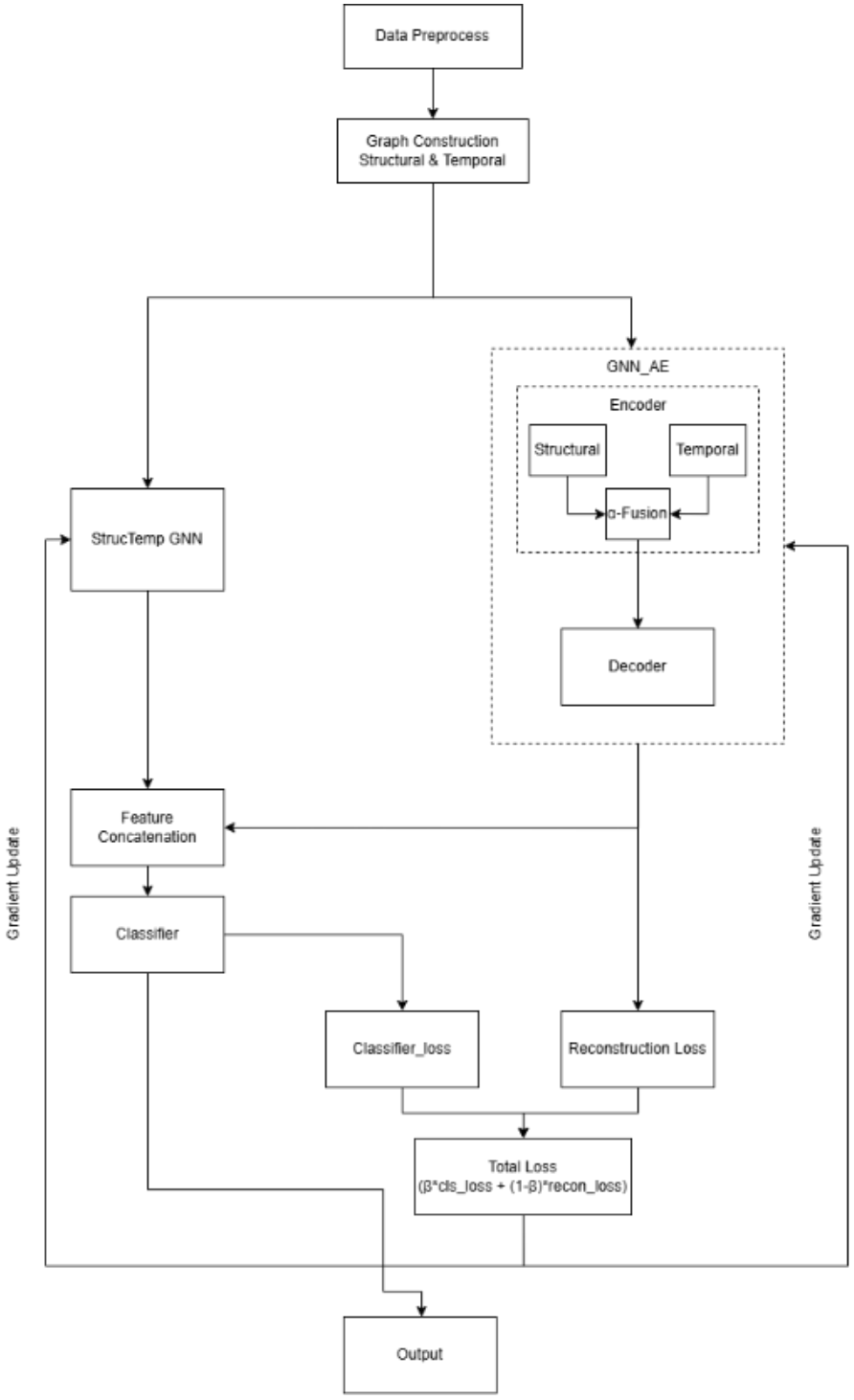
Selanjutnya, vektor fitur terpadu ini menjadi masukan bagi lapisan klasifikasi *semi-supervised*, yang terdiri dari dua lapisan linear dengan aktivasi ReLU. Lapisan ini menghasilkan prediksi biner (normal/anomali). *Reconstruction loss* yang dihasilkan oleh GNN-AE untuk memperkaya representasi laten, sehingga model lebih efektif dalam memanfaatkan data berlabel maupun tak berlabel, tetapi tidak digunakan secara langsung sebagai skor anomali.

Mekanisme pelatihan model dikendalikan oleh total *loss* yang merupakan kombinasi dari *loss* klasifikasi (untuk data berlabel) dan *reconstruction loss* (yang diterapkan pada semua data, termasuk yang tidak berlabel). *Hyperparameter* β berfungsi sebagai penyeimbang kontribusi kedua *loss* tersebut. Dengan pendekatan ini, model tidak hanya mempertahankan akurasi klasifikasi pada data berlabel, tetapi juga memanfaatkan *reconstruction loss* untuk

meningkatkan kualitas fitur dan kemampuan generalisasi terhadap pola trafik yang belum pernah dilihat sebelumnya. Total loss ini dapat dirumuskan secara matematis sebagai berikut:

$$L_{total} = \beta L_{cls} + (1 - \beta) L_{Recon}$$

Dimana L_{cls} merupakan loss dari klasifikasi *Structemp*-GNN dan L_{Recon} merupakan hasil dari *loss* rekonstruksi dari GNN-AE



Gambar 3.13 Proses Integrasi Kedua Model

Integrasi ini menciptakan sinergi antara komponen struktural-temporal (*Structemp*-GNN) dan pembelajaran representasi laten (GNN-AE), menghasilkan sistem *semi-supervised* yang responsif terhadap dinamika jaringan dan lebih efisien dalam pemanfaatan data. Dengan memanfaatkan kedua sumber informasi secara adaptif, model dapat mengenali pola trafik kompleks tanpa ketergantungan penuh pada data berlabel, menjadikannya solusi yang lebih fleksibel untuk lingkungan jaringan yang terus berkembang.

3.2.7 *Hyperparameter Tuning*

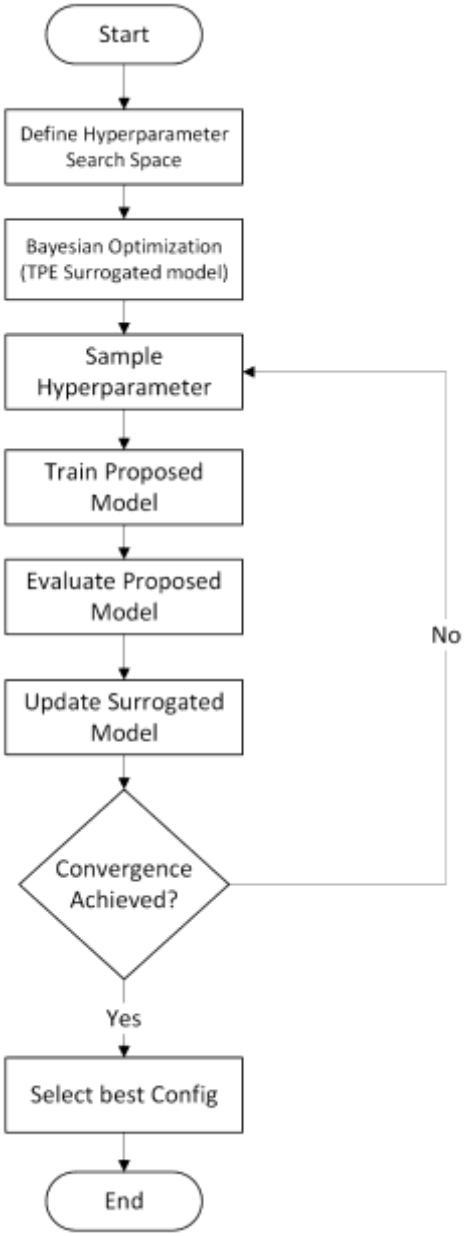
Proses *tuning hyperparameter* dalam penelitian ini dilakukan secara sistematis dengan memanfaatkan pendekatan *Bayesian Optimization* yang diimplementasikan melalui *framework* Optuna. Tujuan dari tahap ini adalah untuk menentukan konfigurasi *hyperparameter* terbaik yang dapat memaksimalkan performa dari *proposed* model. Proses dimulai dengan mendefinisikan ruang pencarian *hyperparameter*, yang meliputi dimensi laten, jumlah layer pada encoder dan decoder, nilai *learning rate*, jenis *optimizer*, serta bobot α dan β pada mekanisme fusi *embedding* dan total loss. Ruang pencarian ini dirancang dengan cakupan nilai yang luas namun tetap relevan agar eksplorasi konfigurasi berjalan optimal.

Setelah ruang pencarian ditentukan, Optuna memulai proses optimasi dengan beberapa percobaan awal yang diambil secara acak dari ruang pencarian. Konfigurasi awal ini digunakan untuk melatih proposed model pada data pelatihan penuh, kemudian performanya diukur pada data validasi menggunakan metrik seperti total loss, *reconstruction loss*, precision, recall, dan F1-score. Nilai evaluasi dari setiap konfigurasi disimpan sebagai fungsi objektif yang akan menjadi acuan bagi iterasi berikutnya.

Pada iterasi selanjutnya, Optuna menggunakan algoritma *Tree-structured Parzen Estimator* (TPE) untuk membangun model probabilistik berdasarkan hasil percobaan sebelumnya. Surrogate model ini memperkirakan distribusi parameter yang menghasilkan performa baik dan membandingkannya dengan distribusi parameter yang kurang optimal. Dari model probabilistik tersebut, sistem memilih konfigurasi *hyperparameter* berikutnya yang memiliki peluang tertinggi untuk memperbaiki nilai fungsi objektif.

Konfigurasi yang diusulkan kemudian kembali digunakan untuk melatih *proposed* model, diikuti dengan evaluasi dan pembaruan model probabilistik. Seluruh proses yang meliputi pemilihan konfigurasi, pelatihan, evaluasi, dan pembaruan model probabilistik ini dilakukan secara iteratif. Optuna juga menerapkan mekanisme *pruning* untuk menghentikan percobaan yang menunjukkan performa buruk lebih awal, sehingga sumber daya komputasi dapat difokuskan pada konfigurasi yang lebih menjanjikan.

Proses iterasi ini berlangsung hingga jumlah percobaan mencapai batas maksimum yang telah ditentukan atau hingga tidak ada peningkatan signifikan pada performa model. Pada tahap akhir, konfigurasi *hyperparameter* dengan hasil validasi terbaik dipilih untuk digunakan pada tahap evaluasi akhir sistem secara menyeluruh. Pendekatan ini memastikan proses tuning berjalan lebih efisien dibandingkan metode pencarian *grid search* sekaligus meningkatkan peluang mendapatkan konfigurasi optimal yang mampu beradaptasi dengan data trafik yang bervariasi.



Gambar 3.14 Proses *Hyperparameter Tuning* Menggunakan Metode *Bayesian*

Dengan pendekatan ini, proses *tuning hyperparameter* menjadi sistematis, adaptif, dan efisien. Hal ini memastikan model tidak hanya mencapai performa optimal tetapi juga mempertahankan kemampuan generalisasi terhadap data trafik dan pola serangan yang belum pernah dilihat sebelumnya.

3.2.8 Evaluasi dan Analisis Hasil Model Usulan

Tahap ini merupakan bagian penting dalam metodologi penelitian yang bertujuan untuk mengevaluasi dan menganalisis performa model usulan setelah pelatihan dengan konfigurasi *hyperparameter* terbaik. Evaluasi dilakukan untuk menilai kemampuan model dalam mendeteksi intrusi jaringan secara akurat dan efisien, sementara analisis hasil berfungsi untuk memberikan pemahaman mendalam mengenai kekuatan, kelemahan, serta kesiapan model untuk implementasi di lingkungan nyata.

Evaluasi kinerja dilakukan dengan menguji model pada *dataset* pengujian yang telah dipisahkan secara stratifikasi dari data pelatihan. Proses ini mencakup perhitungan sejumlah metrik performa klasifikasi yang meliputi akurasi, presisi, recall, dan F1-Score. Akurasi digunakan untuk memberikan gambaran umum mengenai proporsi prediksi yang benar terhadap seluruh prediksi. Namun, dalam sistem deteksi intrusi, akurasi saja tidak cukup karena sering kali terjadi ketidakseimbangan kelas antara trafik normal dan serangan.

Presisi digunakan untuk mengevaluasi seberapa tepat model dalam mengidentifikasi trafik sebagai serangan. Sementara itu, recall menjadi fokus utama karena berkaitan langsung dengan kemampuan model mendeteksi seluruh serangan yang ada. Nilai recall yang tinggi menunjukkan bahwa model berhasil meminimalkan *false negative* yang sangat berisiko pada sistem keamanan jaringan. F1-Score kemudian digunakan sebagai metrik gabungan untuk menyeimbangkan *trade-off* antara presisi dan recall, sehingga memberikan penilaian yang lebih komprehensif terhadap performa model dalam kondisi distribusi kelas yang tidak merata.

Selain metrik klasifikasi, evaluasi kinerja juga mempertimbangkan waktu inferensi sebagai indikator kelayakan model untuk digunakan dalam skenario real-time. Meskipun sistem deteksi intrusi jaringan berbasis *flow* seperti model usulan ini bekerja secara *out-of-band* dan tidak berada pada jalur pemrosesan paket utama, waktu inferensi yang terlalu tinggi tetap dapat menyebabkan keterlambatan dalam memberikan notifikasi atau respons terhadap serangan yang terdeteksi. Oleh karena itu, setiap unit data harus diproses dengan cepat untuk memastikan bahwa deteksi dan pelaporan ancaman tetap relevan dengan kondisi trafik jaringan saat ini. Target waktu ideal berada di bawah satu milidetik per flow agar sistem mampu memantau trafik secara efisien dan mendukung pengambilan keputusan yang cepat.

Setelah perhitungan metrik, analisis hasil dilakukan untuk menilai performa model secara menyeluruh. Analisis ini mencakup visualisasi seperti *confusion matrix* untuk mengidentifikasi distribusi *True Positive* (TP), *False Positive* (FP), *True Negative* (TN), dan *False Negative* (FN), serta grafik distribusi prediksi untuk mengevaluasi performa per kelas serangan. Hasil model kemudian dibandingkan dengan dua model pembanding: *Structemp*-GNN sebagai *baseline* dan NEGSC sebagai model eksternal yang menggunakan pendekatan *self-supervised learning*. Perbandingan ini bertujuan untuk menilai keunggulan arsitektur *semi-supervised* yang diusulkan baik dari segi akurasi deteksi maupun efisiensi komputasi.

Dengan pendekatan evaluasi dan analisis yang menyeluruh ini, penelitian memastikan bahwa model usulan tidak hanya unggul secara statistik, tetapi juga siap untuk diimplementasikan pada sistem deteksi intrusi jaringan yang memerlukan kecepatan dan ketahanan terhadap serangan siber yang terus berkembang.

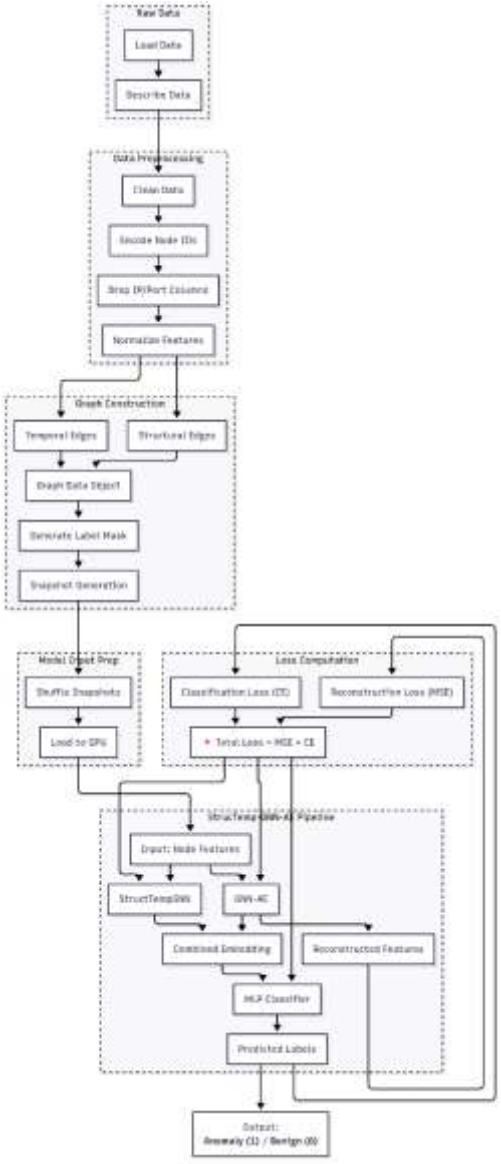
3.3 Aristektur Sistem yang Diusulkan

3.3.1 Arsitektur Model yang Diimplementasikan

Arsitektur sistem yang diimplementasikan dalam penelitian ini dirancang untuk mendeteksi intrusi jaringan dengan pendekatan *semi-supervised learning* menggunakan *Graph Neural Network Autoencoder* (GNN-AE) yang terintegrasi dalam kerangka *Structemp*-GNN. Proses dimulai dari pemuatan data aliran jaringan yang telah dilabeli, yang kemudian dibersihkan, dilakukan *encoding* terhadap *node*, penghapusan kolom yang tidak relevan seperti alamat IP dan *port*, serta normalisasi terhadap fitur numerik. Data hasil *preprocessing* ini selanjutnya digunakan dalam konstruksi graf dinamis yang merepresentasikan hubungan struktural dan

temporal antar aliran jaringan. Proses ini menciptakan objek graf yang kemudian dibagi menjadi *snapshot* menggunakan mekanisme *sliding window* berdasarkan urutan waktu.

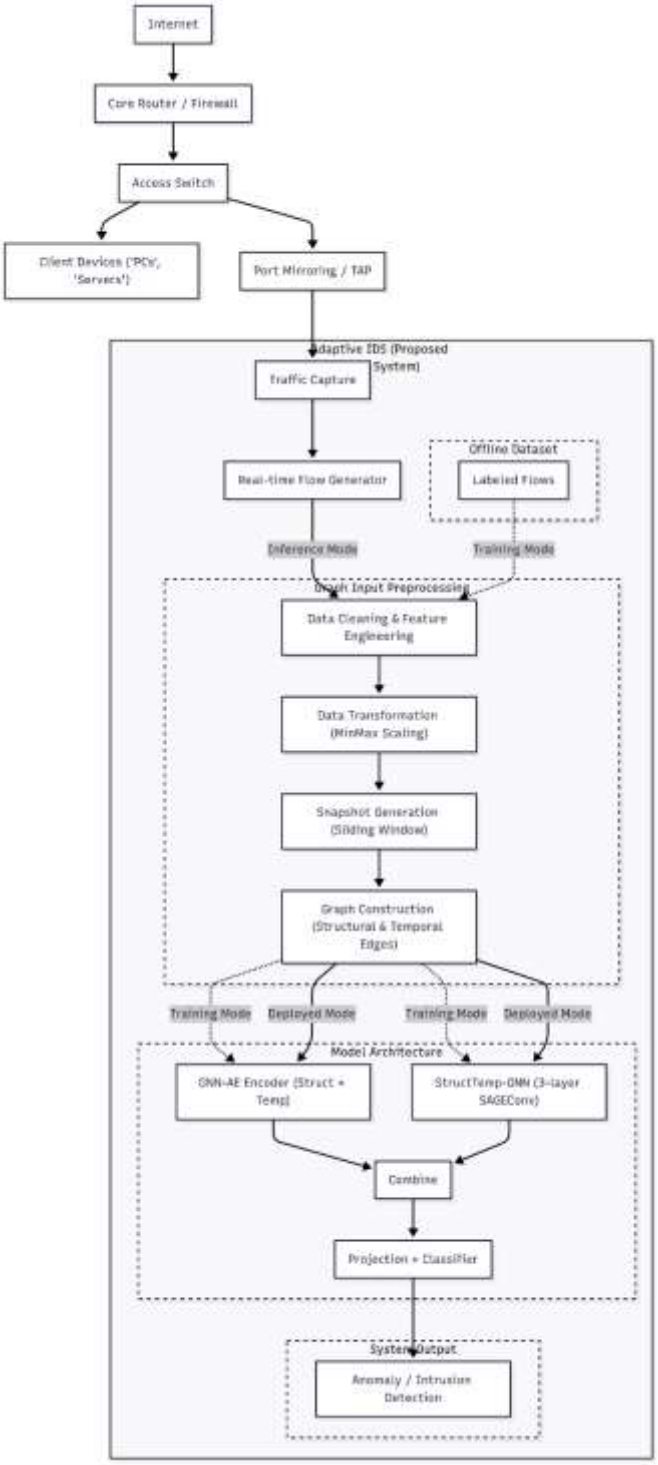
Snapshot yang terbentuk dimuat ke dalam GPU dan menjadi *input* bagi model. Arsitektur model terdiri atas dua jalur utama yang bekerja secara paralel. Jalur pertama menggunakan model *Structemp*-GNN untuk mengekstraksi representasi laten berdasarkan topologi jaringan dan dinamika temporal. Jalur kedua merupakan *Autoencoder* berbasis GNN yang merekonstruksi fitur dari node. *Reconstruction loss* digunakan untuk memperkaya representasi laten, sehingga model mampu memanfaatkan data berlabel maupun tak berlabel dengan lebih efektif. Hasil representasi dari kedua jalur digabungkan dan diteruskan ke *classifier* berbasis *multilayer perceptron* (MLP) untuk menghasilkan label prediksi apakah aliran tersebut tergolong normal atau serangan. Selama proses pelatihan, model dioptimalkan menggunakan fungsi loss gabungan yang terdiri dari *Binary cross-entropy loss* untuk klasifikasi dan *mean squared error* untuk rekonstruksi, dengan total *loss* dikombinasikan menggunakan koefisien β sebagai pengatur kontribusi masing-masing komponen *loss*.



Gambar 3.15 Arsitektur sistem yang diimplementasikan

3.3.2 Rancangan Sistem untuk Penerapan *Real-Time*

Bagian ini menjelaskan rancangan sistem secara menyeluruh apabila model yang dikembangkan diimplementasikan dalam skenario *real-time*. Perlu ditekankan bahwa arsitektur ini bersifat konseptual dan belum direalisasikan dalam penelitian ini. Trafik jaringan dikumpulkan melalui *port mirroring* atau TAP, kemudian diproses oleh *real-time flow generator* untuk menghasilkan aliran yang sesuai dengan format sistem.



Gambar 3.16 Rancangan sistem *real-time*

Aliran tersebut diproses melalui jalur yang sama seperti data *offline*, mulai dari pra-pemrosesan, pembentukan *snapshot*, hingga konstruksi graf dinamis, sebelum dievaluasi oleh model. Sistem dirancang untuk mendukung dua mode operasi yang berjalan secara paralel, yaitu pelatihan dengan data *offline* dan evaluasi terhadap trafik *real-time*. Dengan pendekatan ini, sistem deteksi dapat diintegrasikan langsung ke dalam infrastruktur jaringan dan bekerja secara adaptif terhadap kondisi trafik aktual.

3.4 Rancangan Pengujian

3.4.1 Setup Pengujian

Setup pengujian dalam penelitian ini mencakup perangkat keras sebagai media komputasi, perangkat lunak pendukung yang memastikan kelancaran proses eksekusi, serta kumpulan *dataset* yang menjadi dasar evaluasi performa model. Perangkat keras yang digunakan harus memiliki spesifikasi yang memadai untuk mendukung komputasi intensif, khususnya dalam proses pelatihan model *deep learning* berbasis *Graph Neural Network*. Perangkat lunak yang digunakan mencakup sistem operasi, bahasa pemrograman, serta berbagai *library* dan *framework* yang relevan untuk implementasi model, manipulasi data, visualisasi, hingga optimasi *hyperparameter*. Selain itu, beberapa *dataset* publik dipilih secara khusus untuk menguji kemampuan model dalam mendeteksi intrusi pada kondisi trafik jaringan yang bervariasi dan kompleks.

Tabel 3.1 Alat dan Bahan

Kategori	Alat & Komponen	Keterangan
Perangkat Keras	Laptop	RTX 3050 Laptop GPU, 32 Gb RAM, 1Tb SSD, I5 11400H
Framework	Python 3.11.9	Bahasa pemrograman utama dalam seluruh eksperimen
	Pytorch 2.5.1	*Framework* utama untuk pelatihan *deep learning*
	Pytorch Geometric 2.6.1	Untuk implementasi *Graph Neural Network* dan *Autoencoder*
Library Pendukung	Numpy 2.1.2, Pandas 2.2.3	Operasi numerik dan manipulasi data tabular
	Scikit-Learn 1.6.1	Preprocessing, evaluasi model, dan pembagian data stratifikasi
	Seaborn 0.13.2, Matplotlib 3.10.1, Missingno 0.5.2	Visualisasi data, distribusi nilai kosong, dan analisis eksploratif
	Lion Pytorch 0.2.3	Optimizer efisien berbasis momentum untuk pelatihan GNN
	Window_slider 0.8	Untuk pembentukan *snapshot* dengan pendekatan *sliding window*
	Optuna 4.3.0	Untuk tuning *hyperparameter* secara otomatis berbasis *Bayesian*

Kategori	Alat & Komponen	Keterangan
Lingkungan Pengembangan	Visual Studio Code	Digunakan sebagai lingkungan pengembangan lokal utama
	Google Colab	Digunakan untuk tuning *hyperparameter* menggunakan GPU berbasis cloud
Dataset	NF-UQ-NIDS-v2	*Dataset* utama untuk pelatihan dan evaluasi sistem deteksi intrusi
	NF-ToN-IoT-v2, NF-UNSW-NB15-v2, NF-ToN-IoT-v3, UNSW-NB15, HIKARI-2021	Lima *dataset* pendukung untuk pengujian generalisasi dan validasi model

3.4.2 Skenario Pengujian

Skenario pengujian pada penelitian ini dirancang untuk mengevaluasi kinerja model usulan dalam berbagai kondisi yang merepresentasikan variasi lingkungan trafik jaringan. Setiap skenario dirancang untuk menguji aspek tertentu dari sistem, mulai dari performa dasar hingga kemampuan generalisasi dan perbandingan dengan model lain. Dengan pendekatan ini, hasil pengujian dapat memberikan gambaran komprehensif mengenai keunggulan dan keterbatasan model usulan sebagai sistem deteksi intrusi berbasis *Graph Neural Network*.

Skenario pertama, Evaluasi *Baseline*, bertujuan untuk mendapatkan tolok ukur awal performa deteksi intrusi. Pada tahap ini, arsitektur *Structemp*-GNN dilatih pada *training* set dengan pemantauan performa menggunakan *validation* set untuk mencegah *overfitting* dan memilih *checkpoint* terbaik. Setelah pelatihan, model dievaluasi pada testing set, keduanya merupakan hasil pemisahan dari *dataset* utama NF-UQ-NIDS-v2. Pengujian dilakukan menggunakan konfigurasi *hyperparameter* sebagaimana diusulkan oleh Boukari et al. [9]. Hasil evaluasi *baseline* ini digunakan sebagai acuan untuk menilai peningkatan kinerja model usulan setelah integrasi modul GNN-AE.

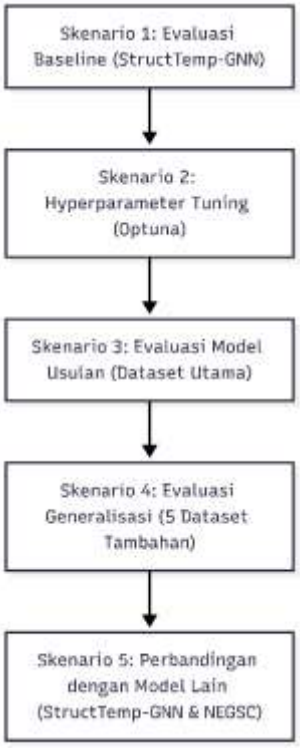
Skenario kedua, *Hyperparameter Tuning*, difokuskan pada optimasi konfigurasi model usulan menggunakan pendekatan *Bayesian Optimization* melalui f*ramework Optuna*. *Dataset* utama NF-UQ-NIDS-v2 dipisahkan menjadi tiga subset: *training* set untuk melatih model pada setiap iterasi, *validation* set untuk memantau performa dan menghitung metrik evaluasi seperti F1-Score. Proses *tuning* bertujuan untuk menemukan kombinasi *hyperparameter* terbaik termasuk jumlah layer, dimensi laten, *learning rate*, dan jenis *optimizer* yang menghasilkan performa optimal pada *validation* set. Konfigurasi terbaik kemudian digunakan pada tahap pelatihan model akhir di skenario berikutnya.

Skenario ketiga, Evaluasi Model Usulan, dilakukan setelah diperoleh konfigurasi *hyperparameter* terbaik dari tahap *tuning*. Pada tahap ini, *dataset* utama NF-UQ-NIDS-v2 dipisahkan menjadi tiga subset: *training* set, *validation* set, dan *testing* set, yang masing-masing digunakan untuk proses pelatihan, pemantauan performa selama pelatihan, dan evaluasi akhir model. Model usulan dilatih pada *training* set, dengan *validation* set digunakan untuk memantau performa dan mencegah *overfitting* melalui mekanisme *early stopping* serta penyimpanan *checkpoint* terbaik. Setelah pelatihan selesai, performa akhir model dievaluasi pada testing set yang tidak pernah digunakan selama pelatihan untuk memastikan generalisasi terhadap data yang benar-benar baru. Evaluasi ini mencakup penghitungan metrik akurasi,

presisi, recall, F1-Score, serta waktu inferensi per *flow* untuk menilai kelayakan model dalam skenario deteksi intrusi *real-time*.

Skenario keempat, Evaluasi Generalisasi, bertujuan untuk menguji kemampuan model usulan dalam mendeteksi serangan pada lingkungan trafik jaringan yang berbeda. Pada tahap ini, lima *dataset* tambahan digunakan, yaitu NF-ToN-IoT-v2, NF-UNSW-NB15-v2, NF-ToN-IoT-v3, UNSW-NB15, dan HIKARI2021. Setiap *dataset* dipisahkan menjadi *training* set, *validation* set, dan testing set. Model usulan, yang telah dilatih ulang pada *training* set untuk masing-masing *dataset*, memanfaatkan *validation* set untuk memantau performa selama pelatihan, mencegah *overfitting*, dan menentukan *checkpoint* terbaik. Performa akhir kemudian dievaluasi pada testing set untuk menilai kemampuan generalisasi model terhadap variasi karakteristik trafik jaringan. Evaluasi dilakukan dengan menghitung metrik akurasi, presisi, *recall*, F1-Score, dan waktu inferensi per *flow*.

Skenario kelima, Perbandingan dengan Model Lain, dilakukan dengan membandingkan hasil evaluasi model usulan dengan dua model pembanding: *Structemp*-GNN sebagai *baseline* dan NEGSC sebagai model eksternal yang menggunakan pendekatan *self-supervised learning*. Perbandingan dilakukan pada semua *dataset* untuk mengevaluasi keunggulan arsitektur *semi-supervised* yang diusulkan dari segi akurasi, efisiensi, dan waktu inferensi.



Gambar 3.17 Diagram Alir Skenario Pengujian yang Dilaksanakan dalam Penelitian

Urutan skenario pengujian dalam penelitian ini disusun untuk mengevaluasi setiap tahap penting dalam pengembangan dan pengujian model usulan. Proses dimulai dengan evaluasi model *baseline* yang digunakan untuk mendapatkan tolok ukur awal performa deteksi intrusi. Tahap selanjutnya adalah *hyperparameter tuning* yang bertujuan untuk mengoptimalkan konfigurasi model sehingga mampu mencapai hasil yang lebih baik. Setelah konfigurasi optimal diperoleh, model usulan dievaluasi pada *dataset* utama untuk mengukur performa akhir

dalam mendeteksi berbagai jenis serangan. Skenario keempat difokuskan pada pengujian kemampuan generalisasi model dengan menggunakan lima *dataset* tambahan yang memiliki karakteristik trafik jaringan berbeda-beda. Skenario terakhir membandingkan hasil evaluasi model usulan dengan dua model pembanding, yaitu *Structemp*-GNN dan NEGSC, untuk memberikan analisis komprehensif terhadap keunggulan arsitektur *semi-supervised* yang diusulkan.

3.4.2 Mekansime Pengujian

Mekanisme pengujian pada penelitian ini dibagi menjadi dua bagian utama, yaitu mekanisme evaluasi model dan mekanisme *tuning hyperparameter*. Kedua bagian ini dirancang untuk memastikan bahwa model yang diusulkan dapat mencapai performa optimal sekaligus diuji secara menyeluruh dalam berbagai skenario.

Mekanisme pengujian pada penelitian ini dirancang untuk memastikan setiap skenario evaluasi berjalan secara sistematis dan dapat direplikasi. Proses ini mencakup serangkaian langkah mulai dari *preprocessing* data, pembagian *dataset*, pelatihan model, validasi, hingga evaluasi akhir. Diagram alir menggambarkan urutan pelaksanaan mekanisme pengujian untuk kelima skenario yang telah dirumuskan, meliputi evaluasi *baseline*, *tuning hyperparameter*, evaluasi model usulan, pengujian generalisasi, serta perbandingan dengan model pembanding.

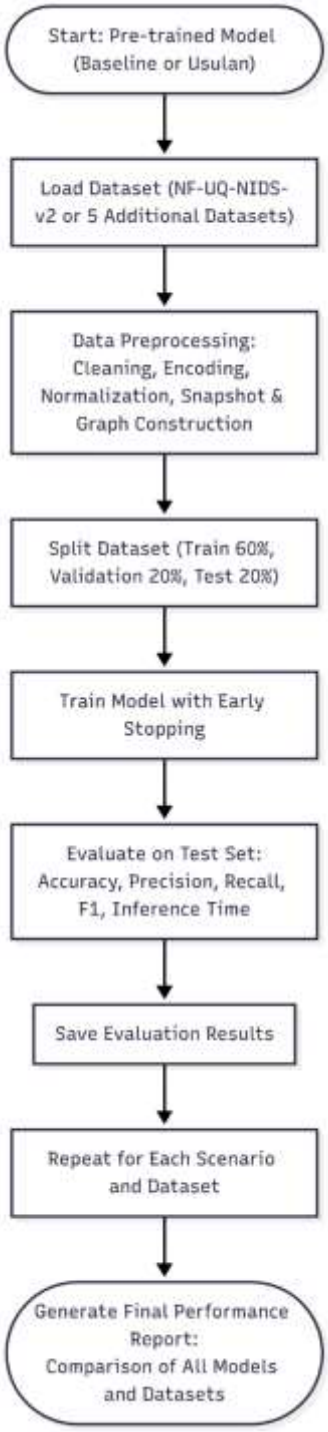
3.4.2.1 Mekanisme Evaluasi Model

Mekanisme evaluasi model dalam penelitian ini bertujuan untuk menguji performa arsitektur yang telah dibangun secara sistematis pada berbagai skenario pengujian. Proses dimulai dengan memuat model yang telah selesai dikembangkan, baik itu model *baseline Structemp*-GNN dengan konfigurasi *default* dari penelitian sebelumnya maupun model usulan dengan konfigurasi *hyperparameter* terbaik hasil optimasi. *Dataset* yang digunakan pada tahap ini mencakup *dataset* utama NF-UQ-NIDS-v2 serta lima *dataset* tambahan yaitu NF-ToN-IoT-v2, NF-UNSW-NB15-v2, NF-ToN-IoT-v3, UNSW-NB15, dan HIKARI2021 yang berfungsi untuk menguji kemampuan generalisasi model pada variasi trafik jaringan.

Sebelum dilakukan pelatihan dan evaluasi, setiap *dataset* melalui tahap *preprocessing* yang mencakup beberapa langkah penting. Langkah pertama adalah pembersihan data untuk menangani nilai tak valid seperti *NaN*, *infinite*, atau *outlier* ekstrem yang dapat mempengaruhi pembelajaran model. Selanjutnya, fitur kategorikal dikodekan menggunakan *OrdinalEncoder* agar dapat diterima sebagai *input* oleh *Graph Neural Network*, sedangkan fitur numerik dinormalisasi ke rentang [0,1] menggunakan *MinMaxScaler* untuk memastikan kontribusi fitur yang seimbang selama proses propagasi informasi pada jaringan graf. Setelah *preprocessing*, data dipecah menjadi tiga subset dengan proporsi 60% untuk data pelatihan, 20% untuk validasi, dan 20% untuk pengujian. Proses ini dilakukan secara stratifikasi untuk menjaga distribusi kelas agar tetap konsisten di seluruh subset.

Data yang telah diproses kemudian dikonversi menjadi representasi graf melalui pembentukan *snapshot* dengan pendekatan *sliding window*. Tahapan ini menghasilkan graf dinamis yang menangkap hubungan struktural dan temporal antar aliran data, yang menjadi *input* bagi model *Structemp*-GNN maupun model yang diusulkan. Model kemudian dilatih pada *training* set, dengan pemantauan performa secara periodik pada *validation* set untuk mendeteksi potensi *overfitting*. Mekanisme *early stopping* diterapkan untuk menghentikan pelatihan lebih

awal apabila tidak ada peningkatan pada metrik F1-Score selama beberapa *epoch*, sekaligus menyimpan *checkpoint* model dengan performa terbaik.



Gambar 3.18 Diagram Alur mekanisme Evaluasi Model

Setelah tahap pelatihan selesai, model dievaluasi pada testing set yang tidak pernah digunakan selama pelatihan untuk memastikan generalisasi terhadap data baru. Evaluasi dilakukan dengan menghitung metrik akurasi, presisi, recall, dan F1-Score untuk mengukur kemampuan deteksi intrusi secara menyeluruh. Selain itu, waktu inferensi rata-rata per aliran

data (flow) juga diukur sebagai indikator kelayakan model untuk implementasi pada skenario *real-time*. Analisis lanjutan dilakukan melalui visualisasi seperti *confusion matrix* untuk menilai distribusi prediksi benar dan salah. Grafik evolusi loss dan F1-Score per *epoch* juga dianalisis untuk memberikan gambaran tentang stabilitas pelatihan.

Hasil dari setiap evaluasi disimpan secara sistematis untuk dianalisis lebih lanjut dan menjadi dasar perbandingan antar skenario. Proses ini diulang pada setiap skenario pengujian yang telah dirancang, termasuk evaluasi *baseline*, evaluasi model usulan pada *dataset* utama, uji generalisasi pada lima *dataset* tambahan, serta perbandingan performa dengan model pembanding seperti NEGSC. Pendekatan ini memastikan bahwa evaluasi model dilakukan secara menyeluruh, konsisten, dan dapat direplikasi.

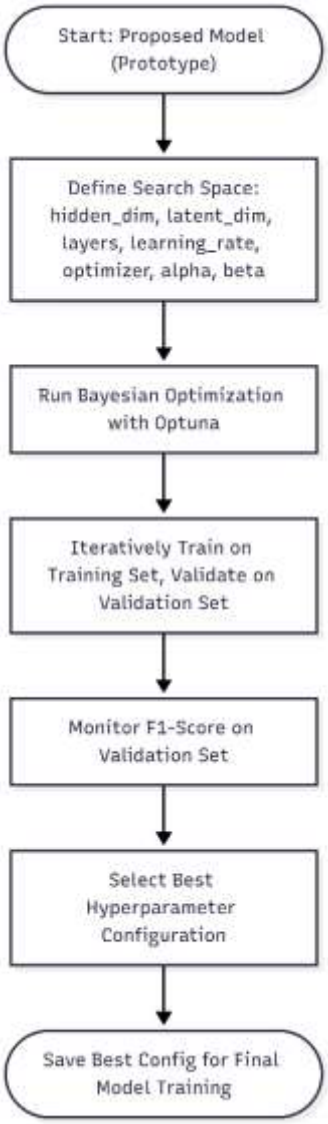
3.4.2.1 Mekanisme Evaluasi Model

Mekanisme *hyperparameter tuning* dalam penelitian ini bertujuan untuk menemukan konfigurasi parameter optimal bagi arsitektur model yang diusulkan agar mencapai performa deteksi intrusi yang maksimal. Proses ini dilakukan sebagai tahap eksperimental terpisah sebelum evaluasi akhir model, sehingga konfigurasi yang dihasilkan dapat digunakan secara konsisten pada semua skenario pengujian berikutnya.

Tahapan dimulai dengan mendefinisikan ruang pencarian *hyperparameter* yang mencakup berbagai parameter penting seperti dimensi tersembunyi (hidden_dim), dimensi laten (*latent_dim*), jumlah layer, *learning rate*, jenis *optimizer*, serta nilai alpha dan beta yang berfungsi sebagai bobot pada mekanisme fusi *embedding* dan *loss* fungsi. Ruang pencarian ini dirancang untuk mencakup kombinasi parameter yang luas namun relevan, sehingga memungkinkan eksplorasi terhadap berbagai konfigurasi yang mungkin menghasilkan generalisasi model yang baik.

Proses *tuning* menggunakan pendekatan *Bayesian Optimization* yang diimplementasikan melalui framework Optuna. Pendekatan ini dipilih karena kemampuannya dalam membangun model probabilistik dari ruang pencarian dan secara iteratif memfokuskan eksplorasi pada wilayah yang paling menjanjikan berdasarkan hasil percobaan sebelumnya. Setiap percobaan (*trial*) dalam optimasi ini melibatkan pelatihan model pada training set untuk kombinasi *hyperparameter* tertentu, diikuti dengan evaluasi performa pada validation set. Metode *early stopping* diterapkan untuk mencegah pemborosan sumber daya pada konfigurasi yang tidak menunjukkan perbaikan performa.

Selama proses *tuning*, metrik F1-Score pada *validation* set digunakan sebagai indikator utama untuk mengevaluasi performa setiap konfigurasi. F1-Score dipilih karena kemampuannya dalam menangani ketidakseimbangan kelas yang sering ditemukan pada data deteksi intrusi. Optuna secara otomatis *mempruning* percobaan yang tidak menjanjikan melalui mekanisme *Hyperband Pruner* untuk mempercepat proses pencarian.



Gambar 3.18 Diagram Alur mekanisme Evaluasi *Hyperparameter Tuning*

Setelah seluruh percobaan selesai, konfigurasi *hyperparameter* dengan F1-Score tertinggi pada *validation* set dipilih sebagai konfigurasi terbaik. Konfigurasi ini kemudian disimpan dan digunakan sebagai parameter tetap untuk pelatihan akhir model yang diusulkan pada semua *dataset* yang digunakan dalam skenario evaluasi.

3.5 Jadwal Pelaksanaan Penelitian

Jadwal pelaksanaan penelitian dalam tugas akhir ini telah dirancang untuk memungkinkan peneliti menetapkan tujuan dan batasan waktu untuk setiap kegiatan selama penelitian berlangsung. Rencana ini disusun dengan harapan dapat memberikan panduan yang jelas dan terstruktur bagi peneliti, sehingga setiap tahap dapat dilaksanakan dengan efektif dan efisien. Berikut adalah jadwal pelaksanaan yang telah disusun:

Tabel 3.2 Jadwal Pelaksanaan Penelitian

Keterangan	Minggu Ke-															
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
Studi Literatur																
Pemilihan *Dataset*																
Data Preprocessing																
Pengembangan Model Dasar																
Integrasi Model																
Evaluasi Kinerja Model																
Hyperparameter Tuning																
Analisis Hasil																
Penulisan Buku tugas akhir																

BAB 4 Hasil dan Pembahasan

4.1 Penelitian Baseline

4.1.1 Deskripsi *Structemp*-GNN Sebagai Baseline

Model baseline yang digunakan dalam penelitian ini merujuk pada arsitektur *Structemp*-GNN, sebagaimana diusulkan oleh Imed Eddine Boukari et al. dalam publikasinya pada tahun 2024 berjudul "*Structemp-GNN: An Intrusion Detection Framework in IoT Networks using Dynamic Heterogeneous Graph Neural Networks*". Model ini dirancang untuk lingkungan jaringan *Internet of Things* (IoT) yang sangat dinamis dan heterogen.

Structemp-GNN memanfaatkan *Graph Neural Network* heterogen dinamis, yang mampu menyesuaikan strukturnya secara otomatis terhadap perubahan topologi dan jenis data pada jaringan. Hal ini tidak hanya meningkatkan efisiensi deteksi anomali, tetapi juga mengurangi *overhead* komputasi dalam eksekusi model. Dalam eksperimen mereka, waktu deteksi per aliran terbukti lebih cepat dibandingkan berbagai model lainnya.

Namun demikian, terdapat beberapa keterbatasan pada studi tersebut, yaitu: Fokus eksperimen terbatas pada jaringan IoT, yang memiliki karakteristik berbeda dengan jaringan internet tradisional. Model dilatih sepenuhnya dengan pendekatan *supervised learning*, yang membutuhkan data berlabel dalam jumlah besar suatu hal yang tidak selalu tersedia dalam skenario nyata. Karena itu, dalam penelitian ini, model *Structemp*-GNN digunakan sebagai *baseline* awal, yang kemudian dikembangkan lebih lanjut dengan pendekatan *semi-supervised* dan penambahan *autoencoder* untuk meningkatkan generalisasi terhadap data serangan baru serta efisiensi dalam kebutuhan *labeling*.

4.1.2 Hasil Evaluasi *Structemp*-GNN Baseline

Model baseline diuji pada *dataset* NF-UQ-NIDS-v2 tanpa dilakukan *tuning* atau modifikasi arsitektur. Berikut adalah hasil performa yang diperoleh pada data validasi:

Tabel 4.1 Hasil Model Baseline

Metrics	Baseline
Accuracy (%)	90.22%
Precision (%)	97.75%
Recall (%)	87.39%
F1 Score (%)	92.28%
Inference time/Snapshot (ms)	8.10
Inference time/Flow (µs)	15.83

4.1.3 Analisis Kinerja Baseline

Evaluasi terhadap model baseline *Structemp*-GNN menunjukkan performa klasifikasi yang solid. Model ini mencatatkan accuracy sebesar 90.22%, dengan precision 97.75% dan recall

87.39%, menghasilkan F1-Score sebesar 92.28%. Tingginya nilai precision menunjukkan kemampuan model untuk meminimalkan false positive, namun nilai recall yang lebih rendah mengindikasikan masih adanya serangan yang tidak terdeteksi (false negative), yang merupakan kelemahan kritis dalam sistem deteksi intrusi.

Dari sisi efisiensi, *Structemp*-GNN unggul secara signifikan. Waktu inferensi rata-rata per snapshot adalah 0.0081 detik, dan per flow hanya 0.0000158 detik, menjadikannya sangat cocok untuk skenario real-time dengan kebutuhan latensi rendah.

Namun demikian, kecepatan inferensi ini dicapai melalui pendekatan supervised learning penuh, tanpa mekanisme pembelajaran representasi atau kemampuan untuk menangani data tak berlabel. Artinya, model ini memiliki keterbatasan dalam menghadapi zero-day attacks atau pola serangan baru yang belum pernah dilihat sebelumnya.

Sebaliknya, model yang diusulkan dalam penelitian ini mengintegrasikan Graph *Autoencoder* dan pendekatan semi-supervised, yang memang menimbulkan peningkatan waktu inferensi per flow. Meskipun waktu inferensi per flow pada model ini lebih tinggi dibandingkan *Structemp*-GNN, hal tersebut merupakan konsekuensi dari kompleksitas tambahan yang dimasukkan untuk meningkatkan generalisasi, deteksi anomali berbasis representasi, serta sensitivitas terhadap serangan yang tidak dikenal.

Dengan kata lain, terdapat *trade-off* antara efisiensi dan kapabilitas deteksi. *Structemp*-GNN cepat, namun kurang fleksibel terhadap data baru. Model usulan lebih lambat, tetapi memiliki kemampuan adaptif yang lebih baik, terutama dalam mendeteksi serangan tak dikenal pada *dataset* lain yang berbeda distribusi.

4.2 Hasil Tuning model

4.2.1 Proses Tuning *Hyperparameter*

Dalam penelitian ini, proses tuning *hyperparameter* dilakukan menggunakan framework Optuna untuk menemukan kombinasi *hyperparameter* terbaik yang dapat meningkatkan performa model semi-supervised *Structemp*-GNN dengan *autoencoder*. Space pencarian *hyperparameter* yang dipertimbangkan meliputi hidden_dim, dimensi, jumlah lapisan ,bobot loss alpha dan beta, learning rate, weight decay, fungsi aktivasi, dan optimizer.

Metode tuning dilakukan dengan memanfaatkan pendekatan Bayesian Optimization yang adaptif, dimana setiap iterasi mengarahkan pencarian ke kombinasi *hyperparameter* yang berpotensi memberikan performa validasi terbaik. Tuning dilakukan pada data validasi dari *dataset* NF-UQ-NIDS-v2 dengan jumlah trial sebanyak 155 iterasi untuk memastikan eksplorasi yang cukup dari ruang pencarian.

4.2.2 Search Space *Hyperparameter*

Ruang pencarian *hyperparameter* dirancang dengan mempertimbangkan fleksibilitas arsitektur dan stabilitas pelatihan dari model semi-supervised *Structemp*-GNN dengan *autoencoder*. Pemilihan rentang nilai untuk setiap *hyperparameter* dilakukan berdasarkan praktik terbaik dari literatur terkini serta eksperimen pendahuluan guna menghindari eksplorasi yang tidak relevan dan menghemat sumber daya komputasi.

Tabel berikut merinci setiap kategori *hyperparameter* beserta rentang atau pilihan nilai yang dijelajahi oleh framework Optuna selama proses tuning:

Tabel 4.2 Rentang pencarian *hyperparameter*

Kategori	Rentang / Pilihan	Keterangan
hidden_dim	{32, 64, 128, 256}	Mengontrol kapasitas representasi fitur dalam GNN encoder.
latent_dim	{16, 32, 64, 128}	Ukuran ruang laten untuk enkoding data, memengaruhi efisiensi *autoencoder*.
num_layers	2 hingga 4	Mengatur kedalaman jaringan, berkaitan langsung dengan kompleksitas struktural.
alpha	0.3 hingga 0.7 (step 0.1)	Koefisien bobot untuk menggabungkan informasi struktural dan temporal dalam *Autoencoder*.
beta	0.3 hingga 0.7 (step 0.1)	Koefisien bobot untuk menggabungkan *reconstruction* loss dan *classification loss* dalam model.
lr	1e-5 hingga 1e-4 (log-uniform)	Learning rate untuk kecepatan pembaruan bobot selama pelatihan, dikontrol secara logaritmik.
wd	{0, 1e-5, 1e-4}	Weight decay sebagai bentuk regularisasi untuk menghindari overfitting.
activation	{relu, leaky_relu, gelu, silu}	Fungsi aktivasi non-linear yang membantu model mempelajari hubungan kompleks antar layer.
optimizer	{AdamW, RAdam, Lion}	Algoritma optimisasi yang mengontrol proses pembaruan bobot selama pelatihan.

Ruang pencarian *hyperparameter* dirancang secara sistematis untuk menyeimbangkan fleksibilitas representasi model dengan stabilitas proses pelatihan dalam arsitektur semi-supervised *Structemp*-GNN dengan *autoencoder*. Setiap kategori *hyperparameter* disusun berdasarkan praktik umum dalam penelitian *Graph Neural Network*s, referensi literatur terkini, serta temuan empiris dari eksperimen pendahuluan. Parameter seperti hidden_dim dan latent_dim dipilih dari himpunan nilai berbasis pangkat dua yang umum digunakan dalam desain arsitektur jaringan untuk memastikan keseimbangan antara kompleksitas dan efisiensi komputasi. Jumlah lapisan (num_layers) dibatasi antara 2 hingga 4 guna menghindari masalah over-smoothing yang umum terjadi pada GNN berlapis dalam. Nilai alpha dan beta disesuaikan dalam rentang 0.3 hingga 0.7 untuk mengeksplorasi kombinasi optimal antara penggabungan informasi struktural-temporal dan kontribusi masing-masing loss dalam total objective function.

Untuk proses optimasi, digunakan rentang log-uniform untuk learning rate agar mencakup skala sensitivitas yang luas, sementara weight decay diterapkan dalam nilai kecil untuk menghindari *overfitting*. Dalam pemilihan fungsi aktivasi, selain ReLU dan Leaky ReLU yang bersifat konvensional, digunakan pula GELU dan SiLU (Swish) yang terbukti memberikan performa unggul dalam model-model modern. GELU telah banyak digunakan dalam model seperti BERT karena kemampuannya memberikan non-linearitas halus dan adaptif terhadap distribusi *input*, sedangkan SiLU dikenal meningkatkan aliran gradien pada jaringan yang dalam. Pada sisi optimizer, selain AdamW yang menjadi baseline populer, juga disertakan

RAdam yang mengatasi variansi tinggi pada learning rate adaptif tanpa perlu warm-up [44], serta Lion optimizer berbasis momentum sign yang dikembangkan dengan pendekatan symbolic discovery dan terbukti efektif dalam mempercepat konvergensi pada berbagai task deep learning [42]. Kombinasi komponen ini memungkinkan proses tuning yang eksploratif namun tetap terkendali secara komputasi, serta mendukung pengembangan model yang mampu mencapai performa optimal pada sistem deteksi intrusi berbasis graph.

4.2.3 Hasil *Hyperparameter* Terbaik

Setelah dilakukan eksplorasi sebanyak 155 iterasi, Optuna berhasil mengidentifikasi konfigurasi *hyperparameter* yang menghasilkan performa validasi terbaik untuk model semi-supervised *Structemp*-GNN. Konfigurasi optimal tersebut dirinci pada tabel berikut:

Tabel 4.3 Hasil *Hyperparameter* Terbaik

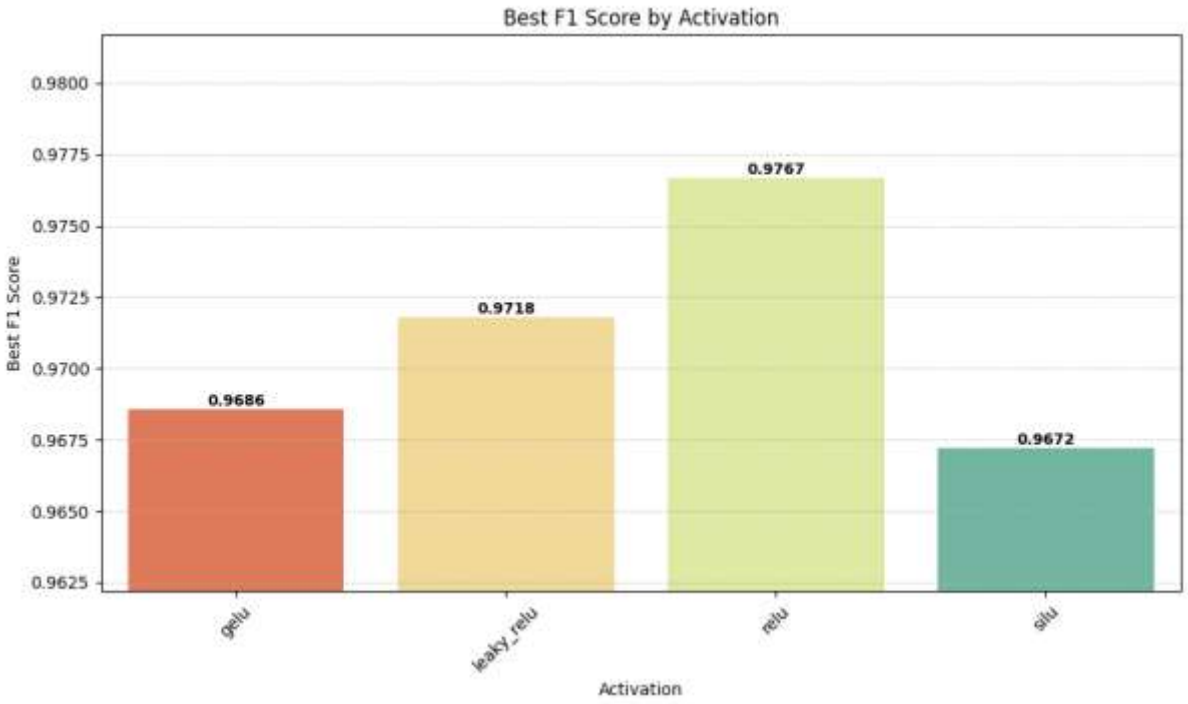
Hyperparameter	**Nilai Terbaik**
hidden_dim	128
latent_dim	128
num_layers	3
alpha	0.4
beta	0.7
lr	0.00001
wd	0.00001
activation	ReLu
optimizer	Lion

Konfigurasi ini menunjukkan bahwa performa terbaik dicapai ketika model memiliki kapasitas representasi yang besar yaitu hidden_dim dan latent_dim masing-masing sebesar 128. Ukuran ini memberikan keleluasaan bagi model untuk menangkap kompleksitas hubungan spasial dan temporal dalam data trafik jaringan.

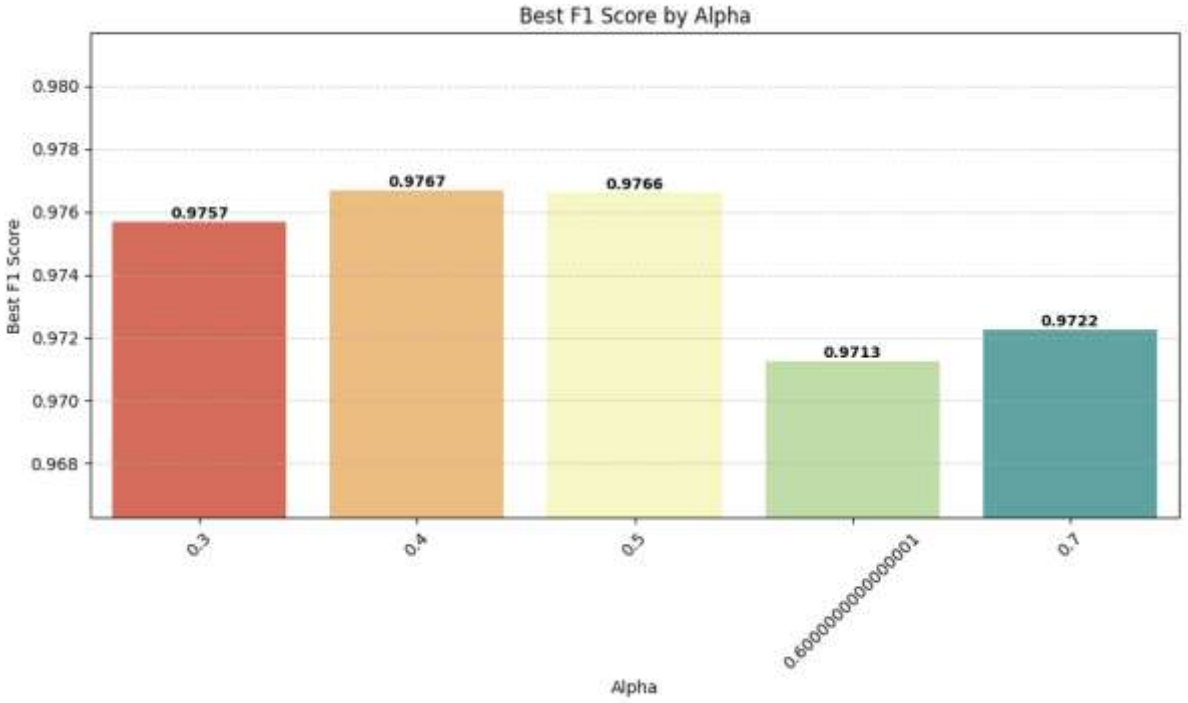
Jumlah lapisan sebanyak tiga terbukti cukup dalam memodelkan struktur grafik tanpa menyebabkan degradasi performa akibat *overfitting* maupun *vanishing gradient*. Fungsi aktivasi ReLU tetap menjadi pilihan optimal, menunjukkan bahwa pendekatan sederhana namun efisien masih sangat kompetitif dalam konteks ini.

Optimizer Lion dipilih oleh Optuna sebagai yang paling efektif. Optimizer ini dikenal karena gabungan kemampuan konvergensi cepat dan stabilitas numerik, yang sangat berguna ketika model beroperasi pada *learning rate* dan *weight decay* yang sangat kecil (1.0e-5). Kombinasi ini berkontribusi terhadap pembelajaran yang stabil sekaligus menghindari overfitting pada data pelatihan.

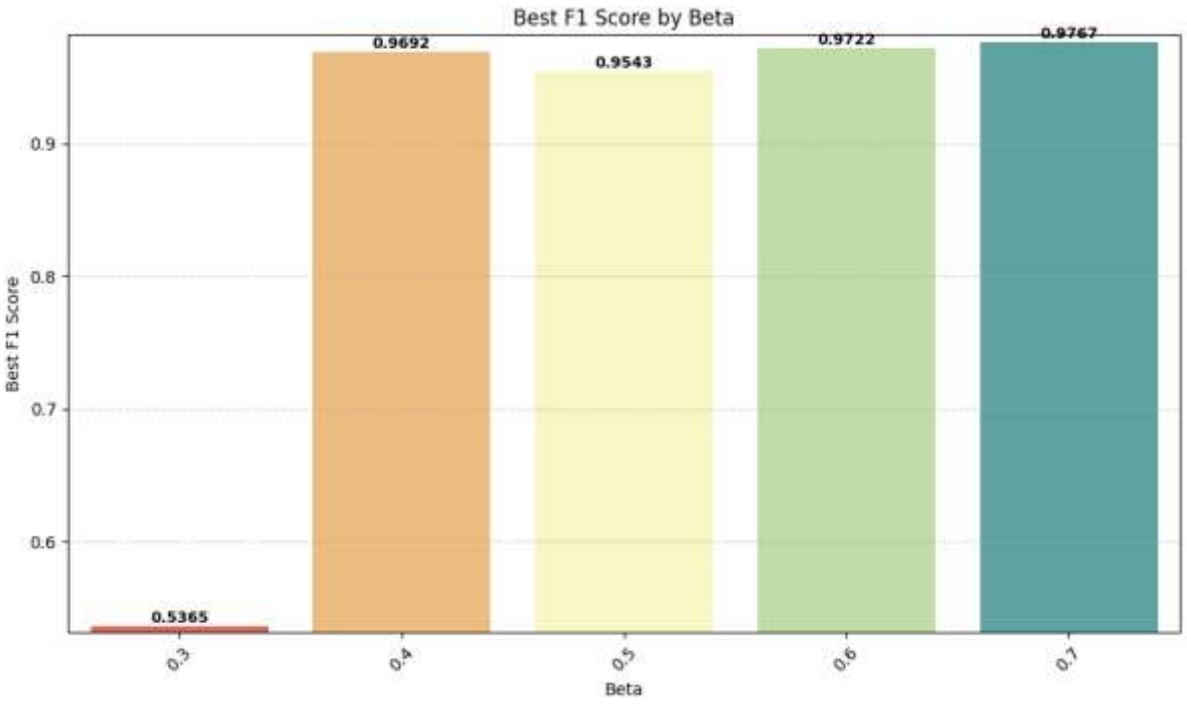
Dari sisi fungsi loss, nilai alpha = 0.4 dan beta = 0.7 mengindikasikan bahwa model memberikan penekanan lebih besar pada komponen klasifikasi dalam kerangka *semi-supervised learning*. Hal ini mencerminkan kebutuhan sistem deteksi intrusi untuk secara aktif memperbaiki performa klasifikasi terhadap data berlabel minimal.



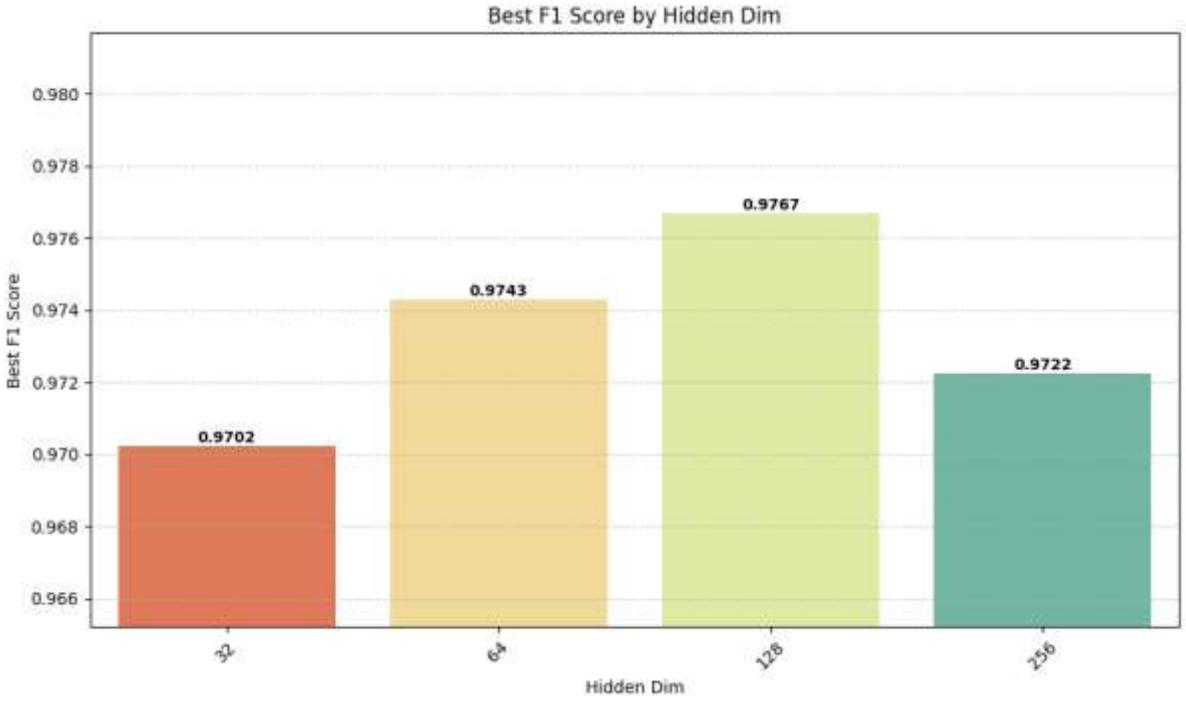
Gambar 4.1 Best F1 Score berdasarkan Fungsi Aktivasi



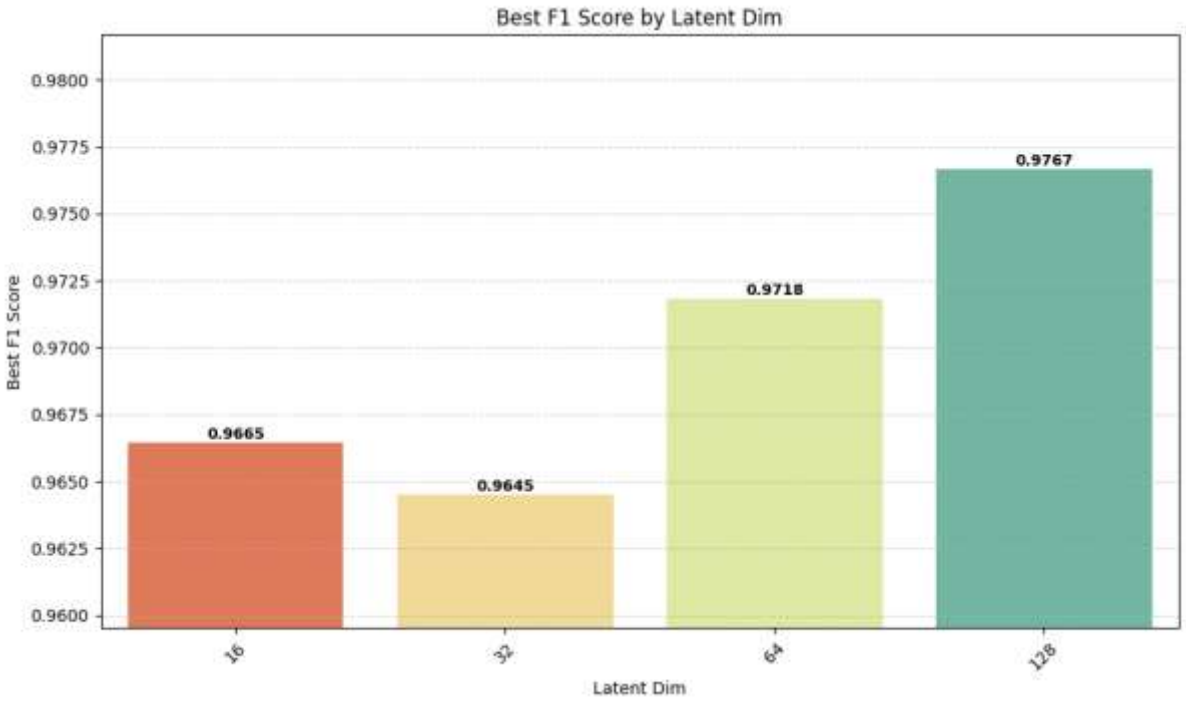
Gambar 4.2 Best F1 Score berdasarkan *Alpha*



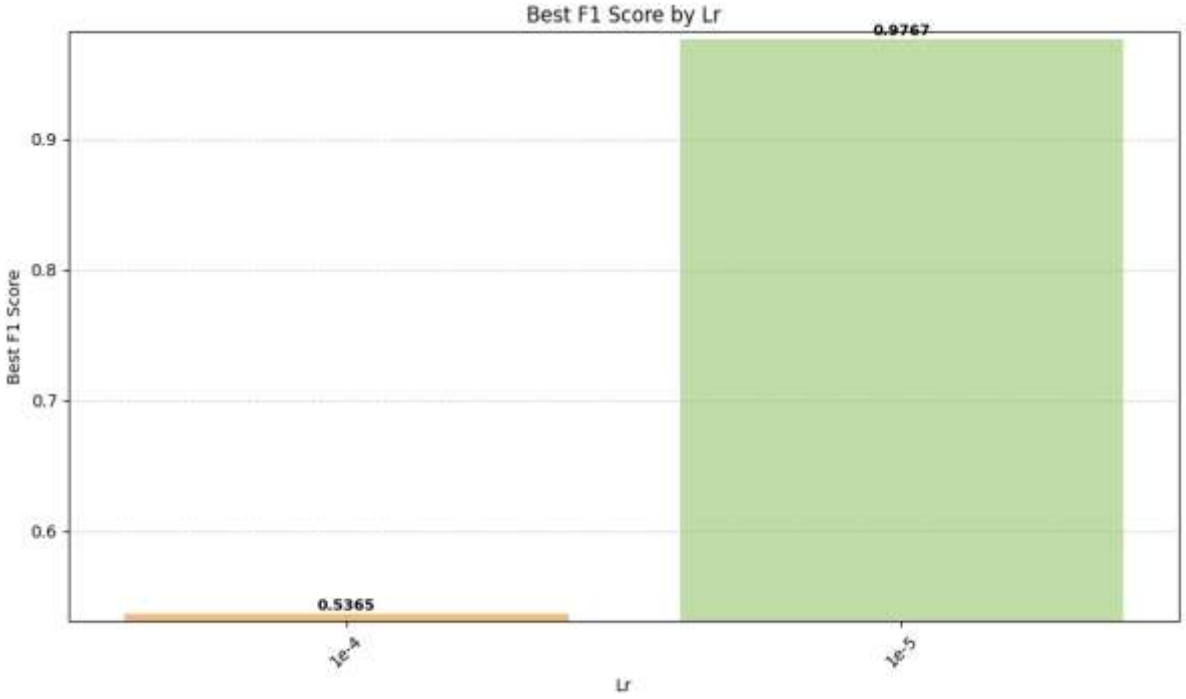
Gambar 4.3 Best F1 Score berdasarkan *Beta*



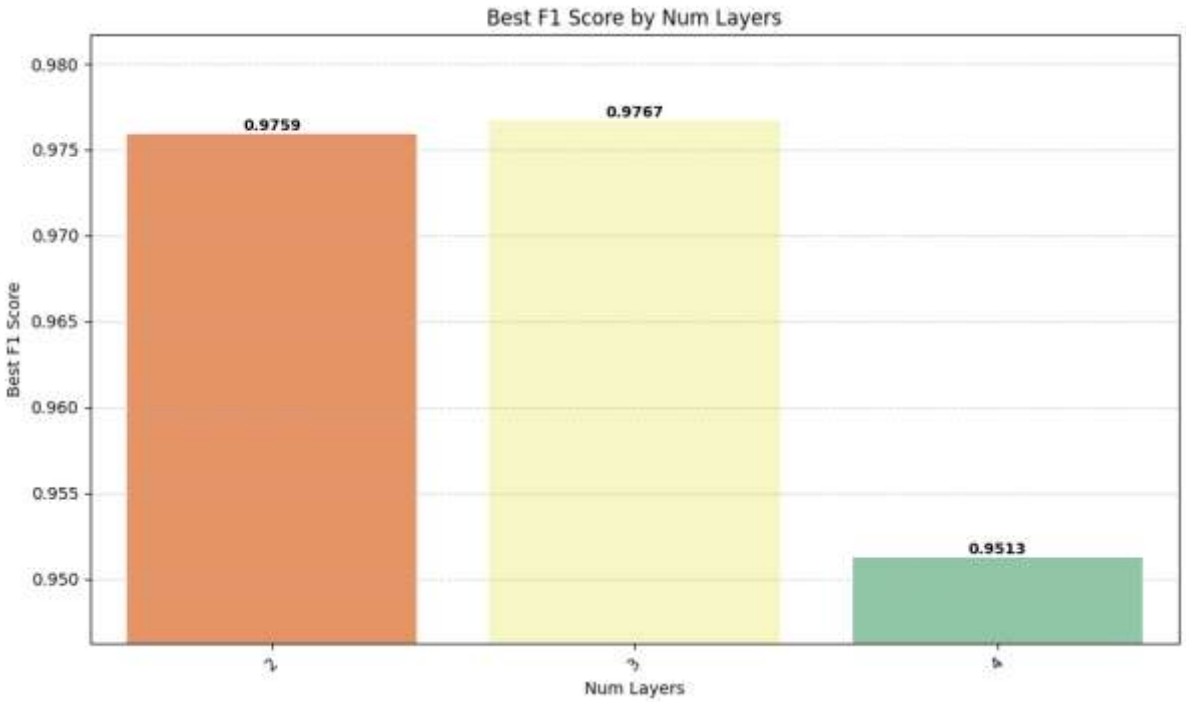
Gambar 4.4 Best F1 Score Berdasarkan *Hidden Dim*



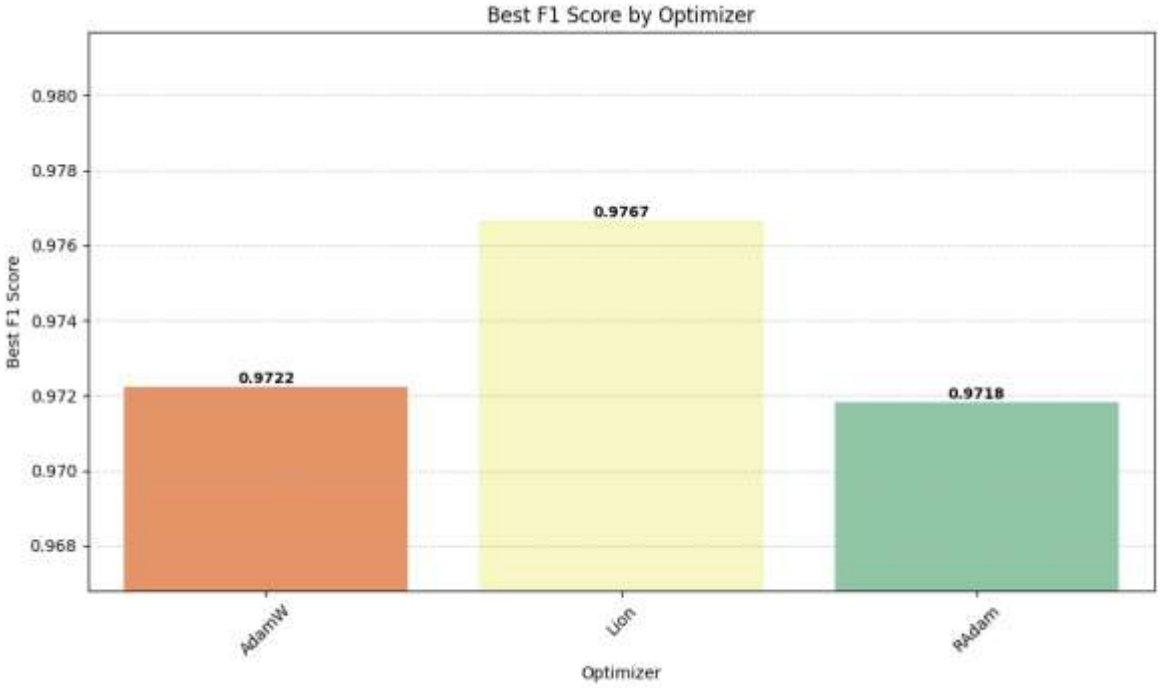
Gambar 4.5 Best F1 Score Berdasarkan *Latent Dim*



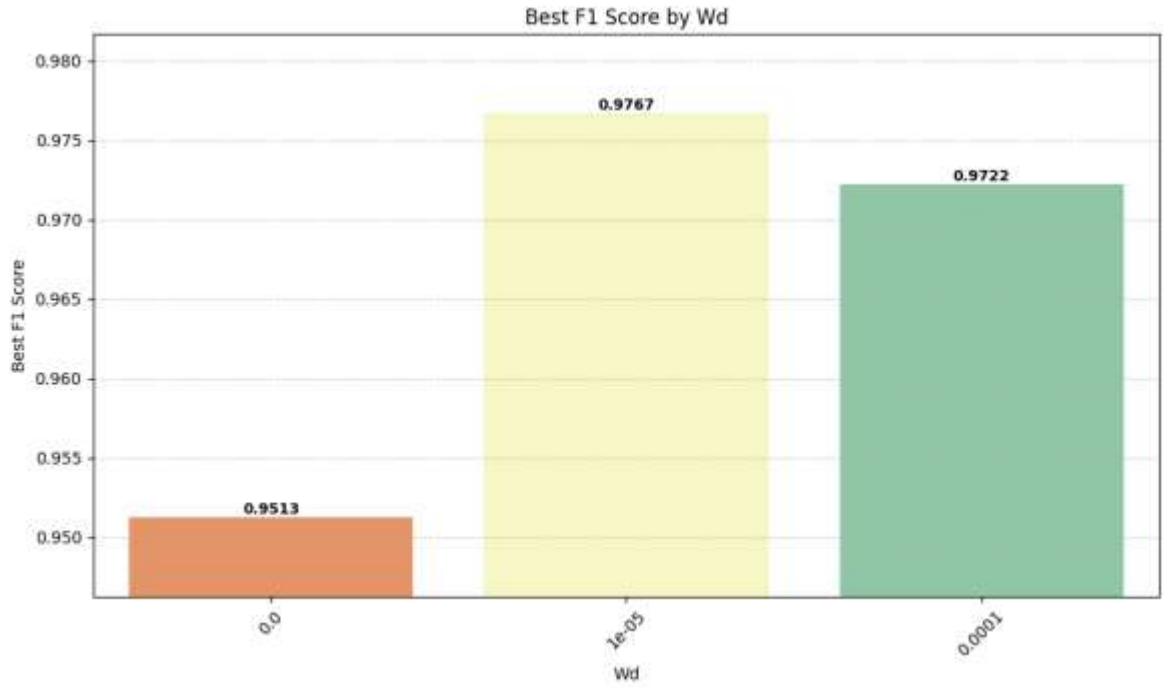
Gambar 4.6 Best F1 Score Berdasarkan *Learning Rate*



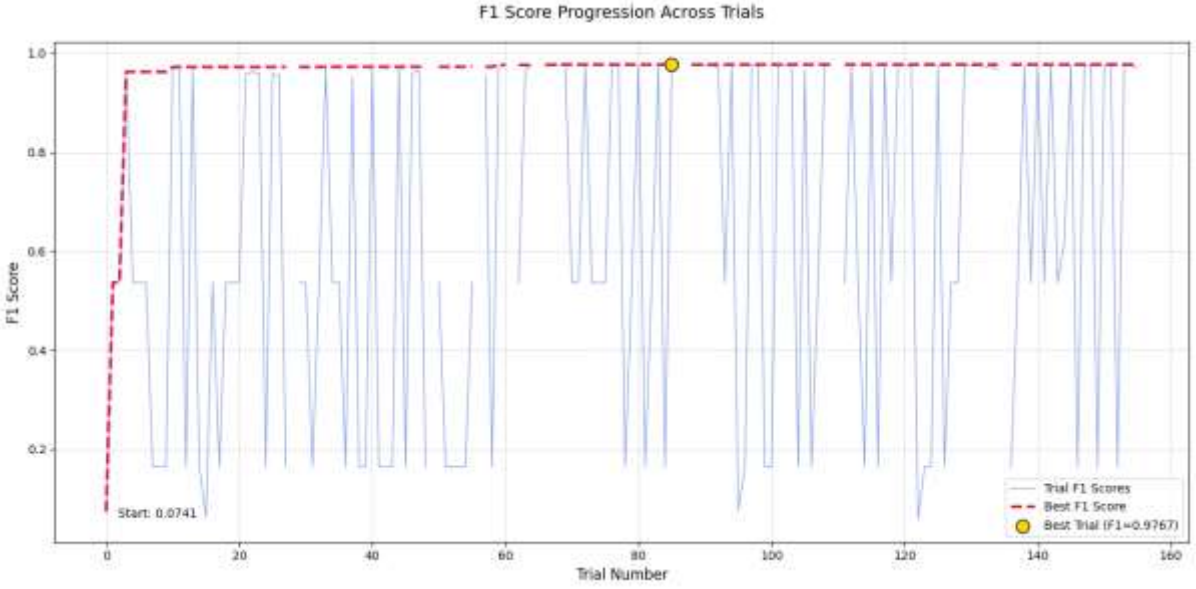
Gambar 4.7 Best F1 Score Berdasarkan Jumlah Layer



Gambar 4.7 Best F1 Score Berdasarkan *Optimizer*



Gambar 4.8 Best F1 Score Berdasarkan *Weight Decay*



Gambar 4.9 Perkembangan F1 Score selama Proses Optimasi

Optimasi menggunakan *Bayesian Optimization* menunjukkan peningkatan F1-score yang sangat cepat pada sepuluh iterasi awal, dari 0.07 menjadi di atas 0.95. Titik performa terbaik tercapai pada iterasi ke-85 dengan F1-score sebesar 0.9767. Setelah itu, tidak ditemukan lagi peningkatan berarti, dan hasil iterasi selanjutnya cenderung stagnan di sekitar nilai optimal. Karena tidak ada tren kenaikan yang signifikan, proses tuning dihentikan pada iterasi ke-155. Konsistensi skor tinggi setelah titik optimal menunjukkan bahwa model memiliki konfigurasi yang stabil dan *robust* terhadap variasi *hyperparameter*.

4.3 Hasil dan Analisis Model

Evaluasi dilakukan untuk mengukur performa model dalam mendeteksi anomali jaringan secara akurat dan efisien. Pengujian mencakup metrik seperti akurasi, *precision*, *recall*, F1-score, serta waktu inferensi.

4.3.1 Evaluasi pada *Dataset* Utama

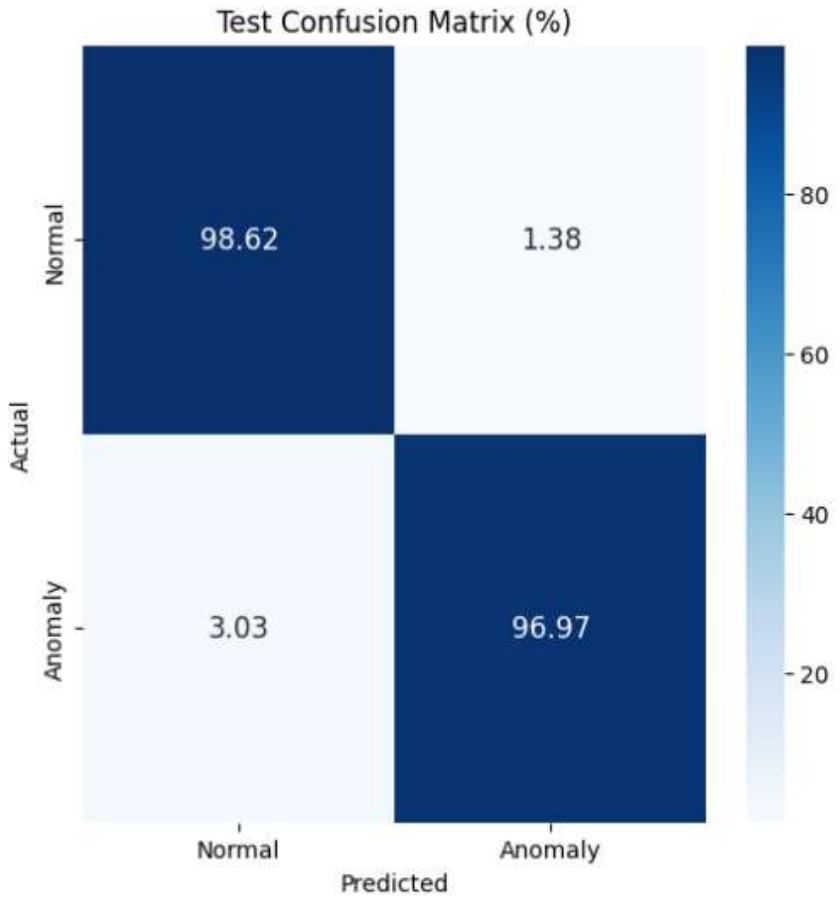
Setelah memperoleh konfigurasi *hyperparameter* optimal melalui proses tuning menggunakan Optuna, tahap berikutnya adalah evaluasi performa model *semi-supervised Structemp*-GNN pada data validasi dan pengujian. Evaluasi ini bertujuan untuk mengukur kemampuan model dalam mengklasifikasikan data dengan benar serta mengidentifikasi pola anomali secara efektif dalam konteks deteksi intrusi jaringan.

Penetapan koefisien β sebesar 0,7 dalam fungsi *loss* memberikan penekanan lebih pada komponen klasifikasi. Pendekatan ini terbukti efektif dalam meningkatkan performa deteksi anomali tanpa menyebabkan penurunan signifikan dalam kestabilan pelatihan. Hal ini menunjukkan bahwa pendekatan *semi-supervised* yang menggabungkan informasi struktural, temporal, dan representasi laten mampu bekerja secara sinergis untuk meningkatkan akurasi sistem secara keseluruhan.

Tabel 4.4 Hasil Proposed model NF-UQ-NIDS-v2

Metrik	Model Usulan
Accuracy	97.51%
Precision	99.34%
Recall	97.51%
F1 Score	97.53%
Inference time/Snapshot (ms)	5.15495
Inference time/Flow (µs)	10.07

Hasil evaluasi menunjukkan bahwa model usulan mencapai akurasi sebesar 97,51%, menandakan tingkat keseluruhan klasifikasi yang sangat baik terhadap data jaringan. *Precision* yang tinggi, yaitu 99,34%, mengindikasikan bahwa sebagian besar deteksi anomali yang dilakukan memang benar-benar merupakan serangan, sehingga risiko alarm palsu tetap rendah. Nilai *recall* sebesar 97,51% mencerminkan kemampuan model dalam mengenali hampir seluruh serangan yang ada, sementara F1-score sebesar 97,53% memperlihatkan keseimbangan optimal antara *precision* dan *recall*. Di sisi lain, performa waktu inferensi juga sangat mendukung penerapan real-time, dengan waktu rata-rata sebesar 5,15 milidetik per *snapshot* dan hanya 10 mikrodetik per aliran data, mempertegas efisiensi model dalam skenario operasional yang dinamis dan berkecepatan tinggi.

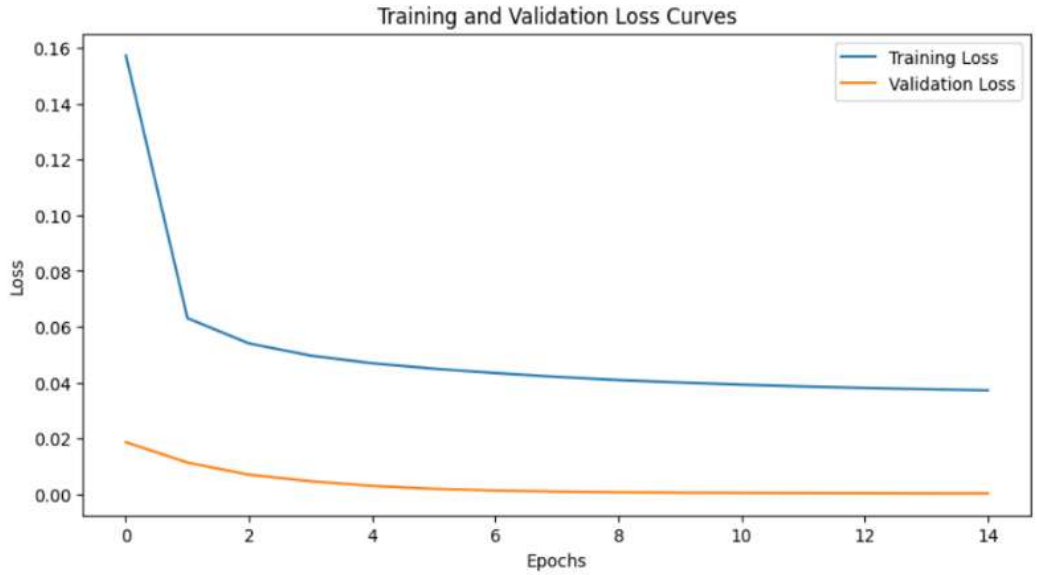


Gambar 4.10 Confusion Matrix *Dataset* Utama

Distribusi hasil klasifikasi memperlihatkan bahwa model memiliki kemampuan yang sangat baik dalam mengidentifikasi baik trafik normal maupun trafik anomali. Sebesar 98,62% dari data normal berhasil diklasifikasikan dengan benar sebagai normal (*True Negative*), menunjukkan tingkat akurasi yang sangat tinggi. Hal ini berarti model sangat jarang memberikan alarm palsu terhadap trafik yang sebenarnya aman, sebuah karakteristik penting dalam menjaga efisiensi operasional sistem keamanan jaringan. Sebaliknya, kesalahan klasifikasi terhadap trafik normal (*False Positive*) tercatat hanya 1,38%, angka yang sangat rendah dan menunjukkan bahwa model tidak reaktif secara berlebihan terhadap variasi yang tidak mencurigakan.

Lebih signifikan lagi, model mampu mendeteksi 96,97% dari seluruh trafik anomali sebagai serangan (*True Positive*), menandakan sensitivitas yang sangat tinggi terhadap pola-pola serangan yang ada dalam *dataset*. Dengan hanya 3,03% dari serangan yang tidak terdeteksi (*False Negative*), model menunjukkan performa yang sangat kompetitif mengingat dalam konteks *Network Intrusion Detection System* (NIDS), FN merupakan jenis kesalahan yang paling berisiko karena memungkinkan serangan lolos tanpa respons.

Dengan keseimbangan antara deteksi yang kuat terhadap anomali dan kestabilan dalam mengenali trafik normal, model ini tidak hanya menunjukkan kinerja tinggi secara metrik, tetapi juga memenuhi kriteria penting untuk diterapkan dalam lingkungan jaringan yang membutuhkan deteksi yang cepat, andal, dan aman dari serangan yang tidak terdeteksi.

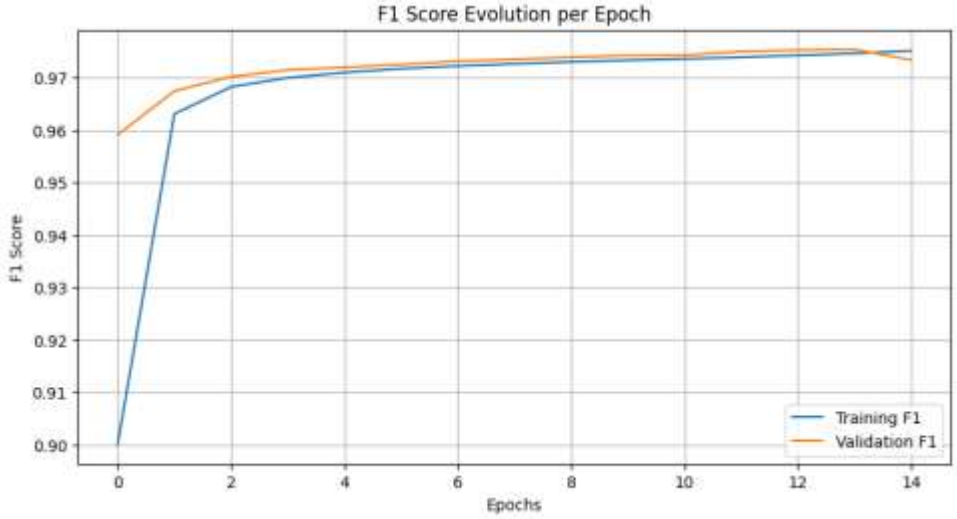


Gambar 4.11 Kurva Loss Pelatihan dan Validasi *Dataset* Utama

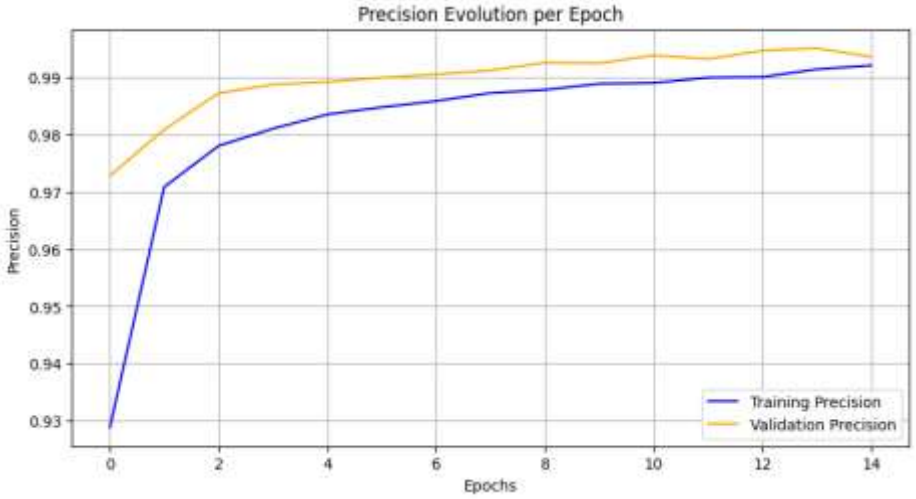
Selain menunjukkan performa klasifikasi yang sangat baik, proses pelatihan model juga ditandai oleh tren penurunan loss yang konsisten, sebagaimana tergambar dalam kurva training dan *validation loss*. *Loss function* mengukur seberapa jauh prediksi model menyimpang dari target yang sebenarnya; oleh karena itu, semakin kecil nilai *loss*, semakin baik model dalam memahami dan merepresentasikan pola dalam data. Pada kurva ini, *training loss* secara bertahap menurun dari sekitar 0.157 ke nilai sekitar 0.0356 seiring dengan bertambahnya *epoch*. Ini menunjukkan bahwa model berhasil mengoptimalkan parameter-parameter internalnya untuk merekonstruksi dan mengklasifikasikan data pelatihan dengan tingkat kesalahan yang terus menurun. *Validation loss*, yang dimulai dari 0.0057 dan turun mendekati 0.0000, bahkan menunjukkan performa yang lebih baik terhadap data yang tidak pernah dilihat model sebelumnya. Hal ini menandakan kemampuan generalisasi yang sangat tinggi, di mana model tidak hanya menghafal pola dari data pelatihan, tetapi juga mampu menangani variasi pada data validasi dengan akurasi tinggi.

Perbedaan nilai loss yang relatif kecil antara data pelatihan dan validasi menjadi indikator penting bahwa *overfitting* tidak terjadi selama proses pelatihan. Namun demikian, adanya selisih tipis antara *training loss* dan *validation loss* juga dapat dijelaskan oleh mekanisme label *masking* yang diterapkan khusus pada data pelatihan. Pada pendekatan *semi-supervised*, tidak semua label pada data pelatihan digunakan secara eksplisit dalam proses pembelajaran; sebagian besar data hanya berkontribusi melalui *reconstruction loss*, sedangkan label yang digunakan untuk *classification loss* hanya sebagian kecil dan dipilih secara strategis. Akibatnya, *training loss* secara keseluruhan cenderung lebih tinggi karena hanya sebagian data yang benar-benar memberi kontribusi terhadap *loss* klasifikasi. Sementara itu, pada data validasi, seluruh label digunakan untuk evaluasi, sehingga loss yang dihasilkan dapat mencerminkan performa penuh dari model terhadap data berlabel secara utuh. Tidak adanya fluktuasi ekstrem pada kurva loss juga menunjukkan bahwa proses pembelajaran berlangsung stabil, dan bahwa konfigurasi seperti *learning rate*, regularisasi, serta gabungan *loss function* antara *reconstruction loss* dan *classification loss* telah diatur dengan tepat.

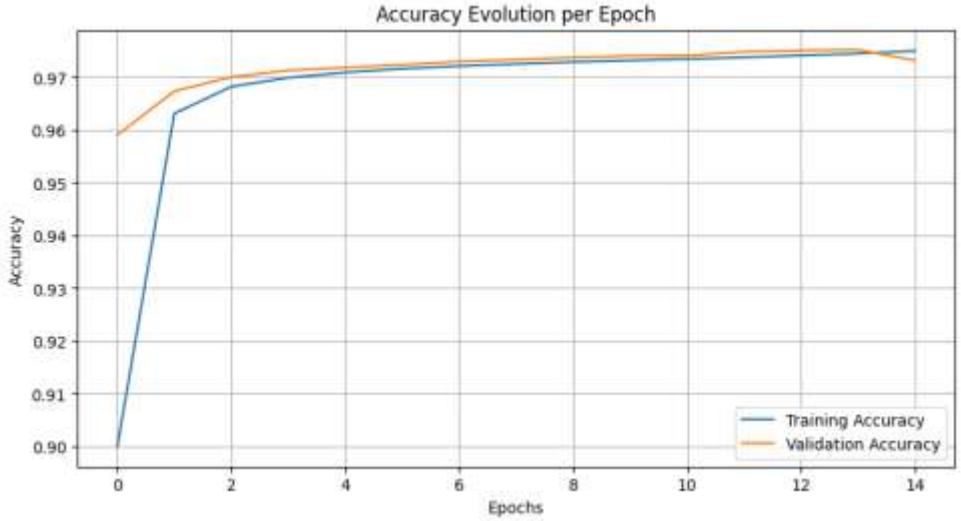
Proses pelatihan model menunjukkan adanya konvergensi yang stabil antara performa data pelatihan dan validasi. Hal ini terlihat dari visualisasi kurva evaluasi, yang menunjukkan tren positif pada metrik F1-score, precision, recall, dan akurasi seiring bertambahnya epoch. Kurva F1-score dan akurasi memperlihatkan bahwa performa model cepat meningkat sejak awal pelatihan dan kemudian mengalami stabilisasi tanpa fluktuasi ekstrem. Precision dan recall juga meningkat secara konsisten, dengan precision validation yang cenderung lebih tinggi dari precision training, mengindikasikan bahwa model tidak mengalami overfitting yang signifikan.



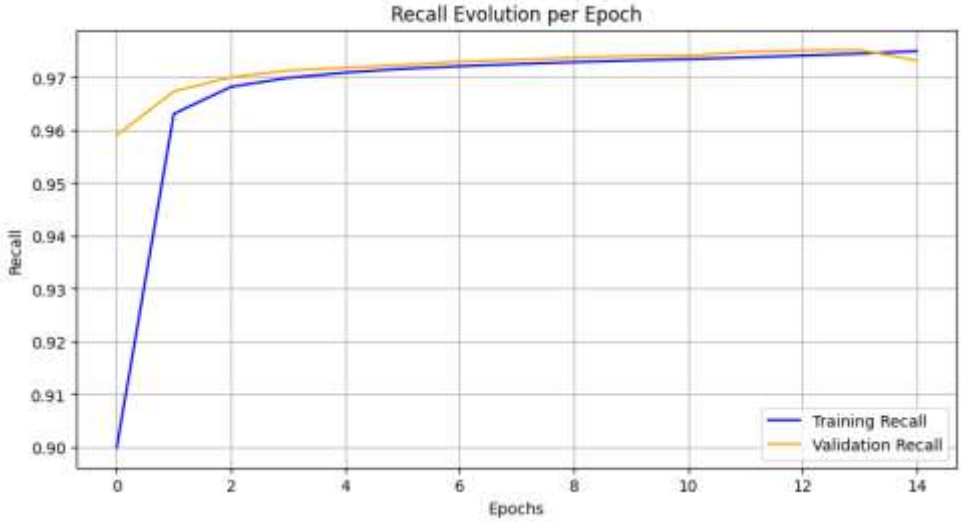
Gambar 4.12 Evolusi F1-Score Per Epoch



Gambar 4.13 Evolusi Precision Per Epoch



Gambar 4.14 Evolusi Akurasi Per Epoch



Gambar 4.15 Evolusi Recall Per Epoch

Kurva-kurva tersebut menunjukkan bahwa proses pelatihan berjalan stabil dan efektif, dengan peningkatan performa yang cepat pada beberapa epoch awal dan stabilisasi di epoch selanjutnya. F1-score dan akurasi mengalami tren naik yang konsisten, sedangkan precision dan recall menunjukkan nilai tinggi dengan gap minimal antara data pelatihan dan validasi. Hal ini menandakan bahwa model tidak hanya mampu belajar dengan baik dari data pelatihan, tetapi juga menunjukkan generalisasi yang baik terhadap data validasi, tanpa indikasi *overfitting* yang signifikan.

4.3.2 Evaluasi Fleksibilitas Arsitektur Model terhadap Variasi *Dataset*

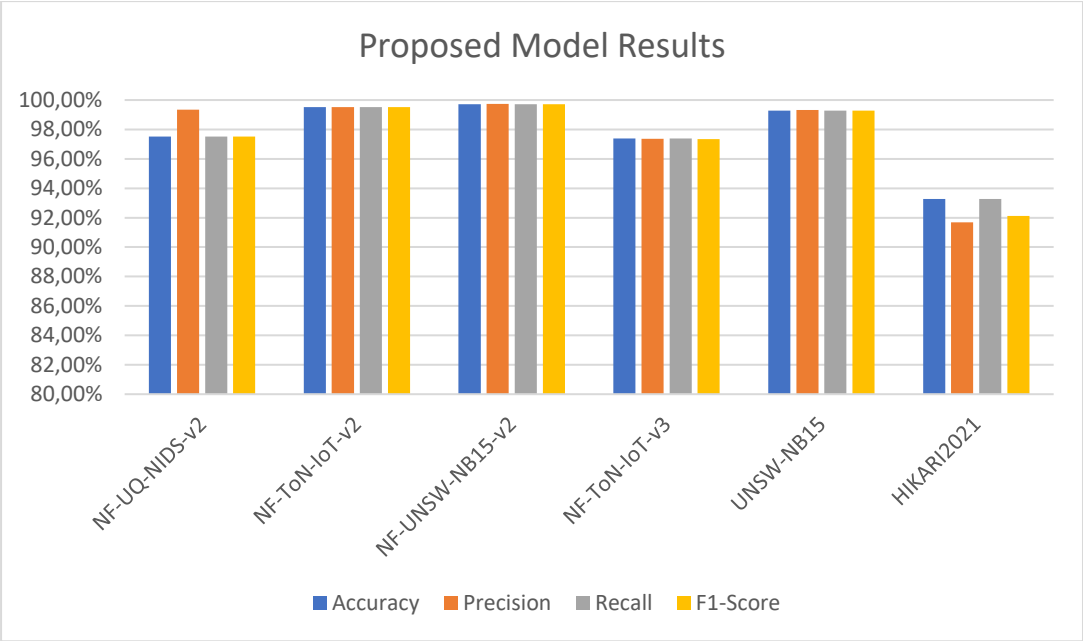
Untuk mengevaluasi kemampuan adaptasi dan fleksibilitas struktural dari arsitektur model yang diusulkan, dilakukan pelatihan ulang pada lima *dataset* tambahan yang memiliki karakteristik berbeda dari *dataset* utama (NF-UQ-NIDS-v2), yaitu NF-ToN-IoT-v2, NF-ToN-IoT-v3, NF-UNSW-NB15-v2, UNSW-NB15, dan HIKARI2021. *Dataset-dataset* ini dipilih untuk merepresentasikan berbagai variasi lingkungan jaringan, seperti *IoT* dan *enterprise*, serta kompleksitas trafik dan jenis serangan yang berbeda. Di antara kelima *dataset* tersebut, NF-ToN-IoT-v3 dengan 53 fitur dan HIKARI2021 dengan 88 fitur memiliki jumlah fitur yang berbeda dari *dataset* utama.

Perbedaan jumlah dan jenis fitur mengharuskan adanya penyesuaian pada bagian *input* model, khususnya pada dimensi lapisan *input encoder* GNN. Penyesuaian dilakukan dengan memperluas dimensi *input* agar sesuai dengan jumlah fitur pada masing-masing *dataset*, tanpa mengubah struktur internal lain dari arsitektur model yang diusulkan. Pelatihan ulang dilakukan dengan tetap mempertahankan proporsi *semi-supervised* yang sama, yaitu menggunakan masking label sebesar 50% selama proses pelatihan.

Tabel 4.5 Hasil *Proposed model* pada *dataset* lain

Dataset	Accuracy	Precision	Recall	F1-Score	*Inference time*/snapshot (ms)	*Inference time*/flow (µs)
NF-ToN-IoT-v2	99.51%	99.51%	99.51%	99.51%	5.93672	11.6
NF-UNSW-NB15-v2	99.71%	99.73%	99.71%	99.71%	6.12642	11.97
NF-ToN-IoT-v3	97.39%	97.37%	97.39%	97.34%	5.81336	11.36
UNSW-NB15	99.27%	99.31%	99.27%	99.29%	5.91989	11.56
HIKARI2021	93.26%	91.69%	93.26%	92.11%	5.29442	10.34

Model yang diusulkan menunjukkan performa terbaik pada NF-UNSW-NB15-v2 dengan akurasi 99,71%, precision 99,73%, recall 99,71%, dan F1-score 99,71%. Hal ini mengindikasikan bahwa model sangat efektif dalam mengenali pola serangan pada data flow-based dengan struktur yang bersih dan fitur yang terpisah dengan baik. Performa yang sangat tinggi juga dicapai pada NF-ToN-IoT-v2 dengan seluruh metrik evaluasi, yaitu akurasi, precision, recall, dan F1-score, berada pada angka 99,51%. Ini menunjukkan kestabilan model dalam menghadapi variasi trafik dan serangan dari lingkungan IoT. Pada *dataset* UNSW-NB15, model tetap menunjukkan kinerja yang kuat dengan akurasi 99,27%, precision 99,31%, recall 99,27%, dan F1-score 99,29%. Hal ini menegaskan bahwa model mampu menangani format data yang berbeda, seperti packet-level, tanpa kehilangan akurasi secara signifikan. Kinerja mulai menurun pada NF-ToN-IoT-v3, dengan akurasi 97,39%, precision 97,37%, recall 97,39%, dan F1-score 97,34%. Penurunan ini mencerminkan adanya tantangan tambahan dalam mengenali serangan pada data dengan kompleksitas fitur dan trafik yang lebih tinggi. Performa terendah tercatat pada HIKARI2021 dengan akurasi 93,26%, precision 91,69%, recall 93,26%, dan F1-score 92,11%. Perbedaan antara precision dan recall mengindikasikan peningkatan false positive, kemungkinan disebabkan oleh sifat trafik yang terenkripsi serta distribusi serangan yang tidak seimbang.



Gambar 4.16 Perbandingan metrik evaluasi model pada berbagai *dataset*.

Performa model yang diusulkan menunjukkan variasi yang signifikan ketika diterapkan pada *dataset* dengan karakteristik trafik, distribusi serangan, kompleksitas fitur, dan jenis serangan yang berbeda. Hasil terbaik dicapai pada NF-UNSW-NB15-v2, dengan F1-score sebesar 99,71%, meskipun proporsi serangannya hanya sekitar 3,98%. *Dataset* ini bersifat flow-based, memiliki 43 fitur, dan mencakup serangan seperti Exploits, Fuzzers, dan Reconnaissance. Perbedaan yang jelas antara trafik *benign* dan anomali, serta struktur data yang bersih, memungkinkan model yang diusulkan untuk belajar secara efektif meskipun terjadi ketidakseimbangan kelas yang signifikan.

Performa tinggi juga dicapai pada NF-ToN-IoT-v2 dengan F1-score 99,51%, yang memiliki 43 fitur dan proporsi serangan yang sangat tinggi, yaitu sekitar 63,99%. *Dataset* ini mencakup beragam jenis serangan seperti DDoS, DoS, Injection, XSS, dan Backdoor, yang berasal dari lingkungan Internet of Things (IoT). Tingginya variasi dan jumlah serangan justru memperkaya pola yang dapat dipelajari oleh model, sementara karakteristik lalu lintas IoT yang cenderung memiliki pola temporal yang stabil mendukung pembentukan graf yang representatif.

Dataset UNSW-NB15, meskipun berbasis packet-level dan hanya memiliki 49 fitur, tetap menghasilkan F1-score sebesar 99,29%. Proporsi serangan dalam *dataset* ini sekitar 4–5%, dengan tipe serangan seperti Analysis, Backdoor, Shellcode, dan Worms. Meskipun format datanya berbeda, fitur-fitur yang tersedia masih cukup relevan untuk menangkap karakteristik serangan, menunjukkan fleksibilitas arsitektur model yang diusulkan terhadap jenis *input* yang bervariasi.

Performa mulai menurun pada NF-ToN-IoT-v3, yang memiliki 53 fitur, proporsi serangan sekitar 38,98%, dan F1-score 97,34%. *Dataset* ini juga berasal dari domain IoT, tetapi pola trafik dan serangan di dalamnya cenderung lebih mirip dengan trafik normal, serta fitur-fiturnya kurang diskriminatif. Hal ini menyulitkan model dalam membentuk representasi laten yang konsisten, terutama saat menghadapi jenis serangan seperti Injection dan DDoS yang sangat menyerupai pola trafik benign.

Penurunan paling signifikan terjadi pada HIKARI2021, dengan F1-score 92,11% dan precision terendah 91,69%. *Dataset* ini mengandung 88 fitur, proporsi serangan hanya 6,8%, dan sebagian besar trafiknya menggunakan protokol terenkripsi seperti HTTPS dan TLS. Jenis serangan yang terkandung antara lain *Bruteforce*, *Cryptominer*, dan Port Scan, yang berlangsung dalam durasi pendek dan sulit dibedakan secara temporal. Selain itu, karena enkripsi menyembunyikan sebagian besar informasi penting, fitur yang tersedia menjadi kurang efektif dalam merepresentasikan serangan.

Secara keseluruhan, temuan ini menegaskan bahwa efektivitas model yang diusulkan sangat dipengaruhi oleh beberapa faktor utama, yaitu keterbukaan dan diskriminatifnya fitur, struktur data berbasis flow, serta eksplisitnya pola serangan. *Dataset* dengan struktur bersih, fitur representatif, dan distribusi serangan yang seimbang cenderung menghasilkan performa tinggi. Sebaliknya, *dataset* dengan enkripsi, ketidakseimbangan kelas ekstrem, atau serangan dengan durasi sangat singkat akan menurunkan performa. Untuk mengatasi hal ini, pendekatan seperti augmentasi fitur, ekstraksi metadata TLS, serta strategi penyeimbangan kelas perlu dipertimbangkan.

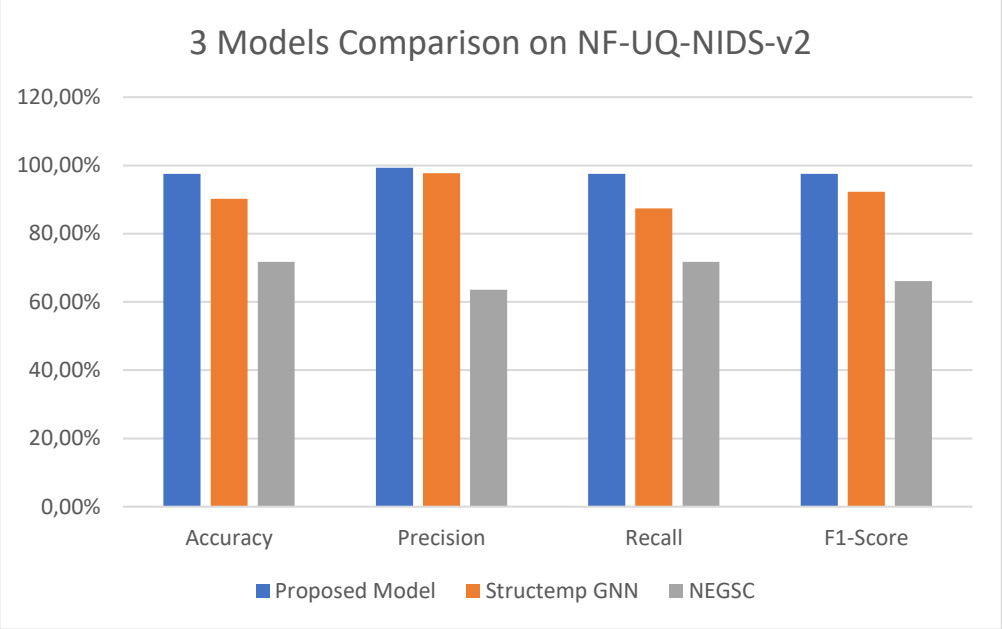
4.4 Perbandingan dengan Model Lain

Untuk mengevaluasi keunggulan model *semi-supervised Structemp*-GNN yang dikembangkan dalam penelitian ini, dilakukan perbandingan performa dengan dua model pembanding yang telah dikenal luas dalam sistem deteksi intrusi jaringan, yaitu *Structemp*-GNN dari studi sebelumnya serta NEGSC (*NetFlow-Edge Generative Subgraph Contrast*). Perbandingan dilakukan berdasarkan sejumlah metrik utama, meliputi akurasi, precision, recall, F1-score, serta waktu inferensi, guna menilai baik efektivitas deteksi maupun efisiensi komputasi.

Structemp-GNN menggunakan pendekatan pembelajaran terawasi penuh yang menggabungkan ketergantungan struktural dan temporal, namun belum dilengkapi dengan mekanisme representasi laten. Model ini memiliki waktu inferensi yang cepat, namun performanya cenderung menurun ketika dihadapkan pada distribusi data yang berbeda dari lingkungan pelatihan, terutama pada *dataset* di luar domain IoT. Pemilihan *Structemp*-GNN sebagai model pembanding dilakukan karena arsitekturnya merupakan dasar dari pengembangan model dalam penelitian ini, sehingga perbandingan secara langsung dapat mengukur kontribusi dari komponen tambahan yang diusulkan, seperti *autoencoder* dan mekanisme pembelajaran semi-supervised.

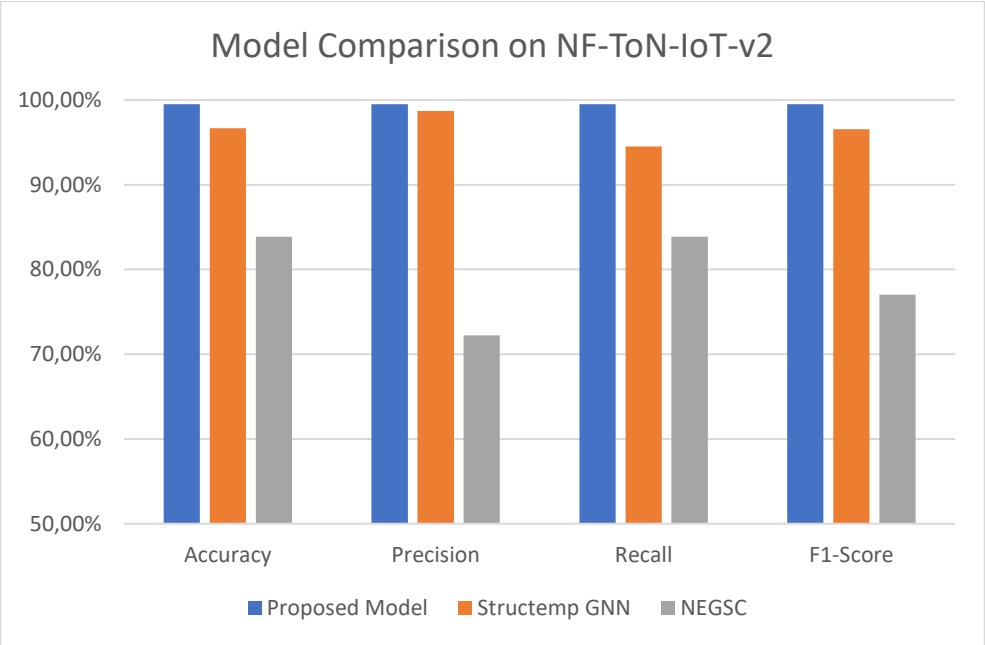
Sementara itu, NEGSC merupakan model berbasis *contrastive learning* dengan mekanisme *generative subgraph* yang bekerja secara *self-supervised*, tanpa membutuhkan data berlabel. Meskipun mampu membentuk representasi yang kuat, NEGSC memiliki kompleksitas arsitektur yang tinggi dan membutuhkan waktu inferensi yang relatif lama, yang dapat menjadi kendala dalam implementasi real-time. NEGSC dipilih sebagai pembanding karena mewakili pendekatan mutakhir yang sepenuhnya tidak bergantung pada label, berbeda dengan model usulan yang menggunakan *semi-supervised learning*. Dengan membandingkan model usulan terhadap NEGSC, dapat dievaluasi apakah strategi yang lebih ringan dan fleksibel, namun masih memanfaatkan sebagian data berlabel, mampu mencapai kinerja yang setara atau bahkan lebih baik dalam hal deteksi intrusi dan efisiensi komputasi.

Model yang diusulkan dalam penelitian ini dievaluasi pada enam *dataset* berbeda, yaitu NF-UQ-NIDS-v2, NF-ToN-IoT-v2, NF-UNSW-NB15-v2, NF-ToN-IoT-v3, UNSW-NB15, dan HIKARI2021. Hasil pengujian menunjukkan bahwa model ini secara konsisten menghasilkan nilai recall dan F1-score yang lebih tinggi dibandingkan dua model pembanding pada sebagian besar *dataset*, menunjukkan sensitivitas tinggi dalam mendeteksi serangan dan keseimbangan yang baik antara deteksi dan kesalahan klasifikasi. Selain itu, waktu inferensi yang rendah pada model ini memperkuat karakteristiknya sebagai solusi yang efisien dan layak untuk implementasi real-time.



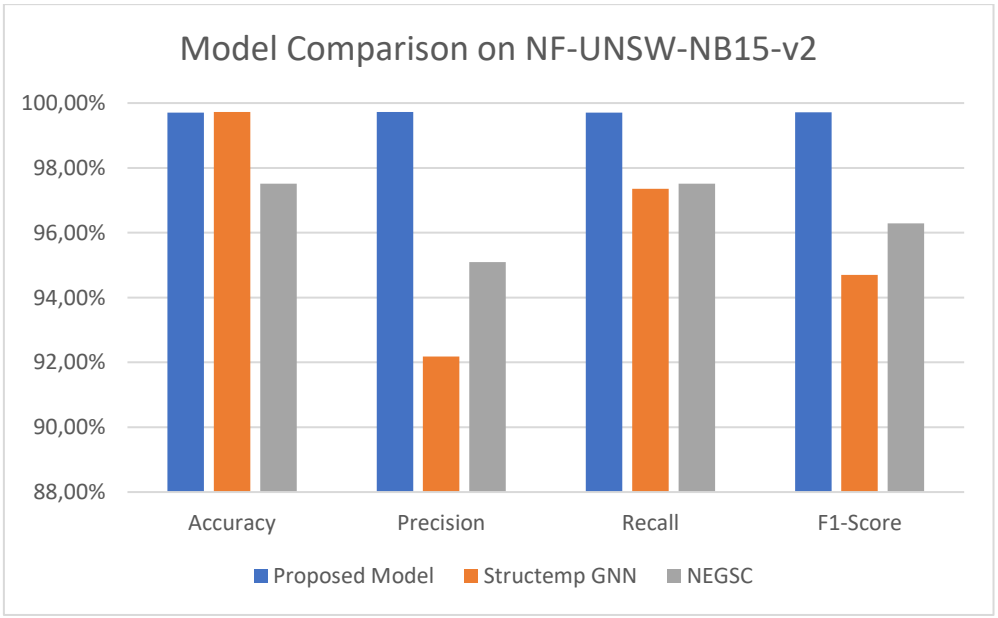
Gambar 4.17 Perbandingan pada NF-UQ-NIDS-v2

Pada *dataset* NF-UQ-NIDS-v2, model yang diusulkan mencatatkan performa terbaik dengan F1-score sebesar 97,53%, recall 97,51%, dan precision 99,34%. *Structemp*-GNN memiliki precision yang hampir sebanding (97,75%), namun recall-nya turun signifikan ke 87,39%, yang berdampak pada penurunan F1-score menjadi 92,28%. Di sisi lain, NEGSC hanya mencapai F1-score 66,10%, dengan precision 63,60% dan recall 71,69%, jauh di bawah dua model lainnya. Hasil ini menegaskan bahwa pendekatan semi-supervised pada model usulan mampu menjaga sensitivitas dan akurasi deteksi secara seimbang, bahkan pada *dataset* nyata yang kompleks seperti NF-UQ-NIDS-v2.



Gambar 4.18 Perbandingan pada NF-ToN-IoT-v2

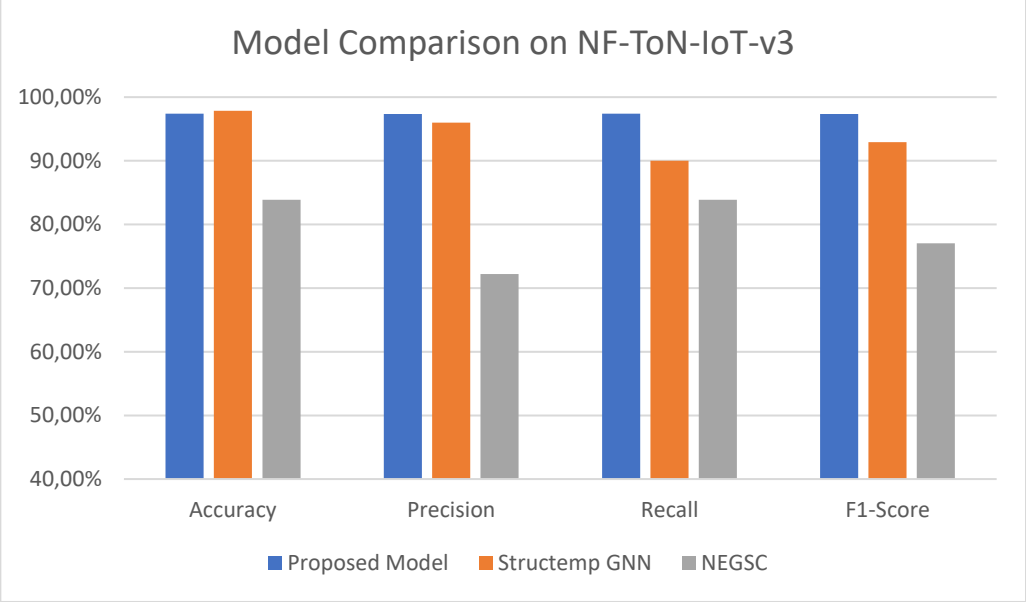
Pada *dataset* NF-ToN-IoT-v2, model usulan mencatatkan hasil hampir sempurna dengan precision, recall, dan F1-score masing-masing sebesar 99,51%, serta akurasi keseluruhan 99,51%. *Structemp*-GNN menunjukkan precision tinggi sebesar 98,73%, namun recall menurun ke 94,52%, sehingga F1-score berada di angka 96,58%. Sementara itu, NEGSC hanya memperoleh precision 72,23%, recall 83,88%, dan F1-score 77,01%, menempati posisi terendah di antara ketiganya. Performa NEGSC yang tidak seimbang antara precision dan recall memperlihatkan kelemahan dalam mendeteksi serangan secara konsisten, terutama pada trafik IoT yang memerlukan sensitivitas tinggi dan latensi rendah. Dengan demikian, model usulan tidak hanya unggul dalam akurasi, tetapi juga lebih stabil untuk aplikasi real-time.



Gambar 4.19 Perbandingan pada NF-UNSW-NB15-v2

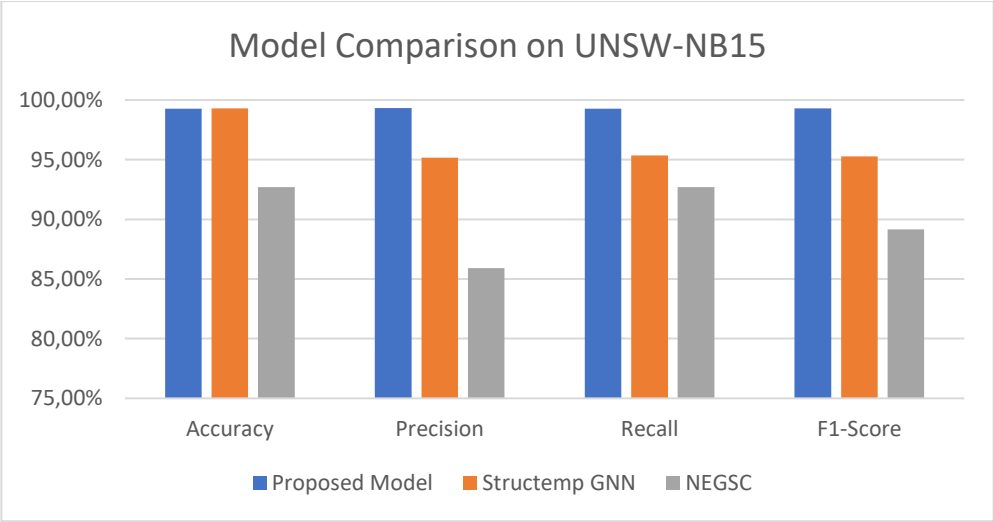
Pada *dataset* NF-UNSW-NB15-v2, model usulan menunjukkan performa yang sangat kuat dengan akurasi 99,71%, precision 99,73%, recall 99,71%, dan F1-score 99,71%, mencerminkan

generalisasi yang sangat baik terhadap *dataset* yang kompleks dan bervariasi. *Structemp*-GNN memang memiliki recall yang tinggi yaitu 97,36%, namun precision-nya turun signifikan menjadi 92,18%, sehingga F1-score hanya mencapai 94,70%, menunjukkan kecenderungan menghasilkan false positive. NEGSC mencatatkan recall 97,52% dan precision 95,10%, menghasilkan F1-score 96,29%, yang meskipun cukup kompetitif tetap berada di bawah performa model usulan. Hasil ini menunjukkan bahwa pendekatan *semi-supervised* yang digunakan mampu mengungguli baik model *fully-supervised* maupun *self-supervised* pada *dataset* besar dengan variasi serangan yang tinggi.



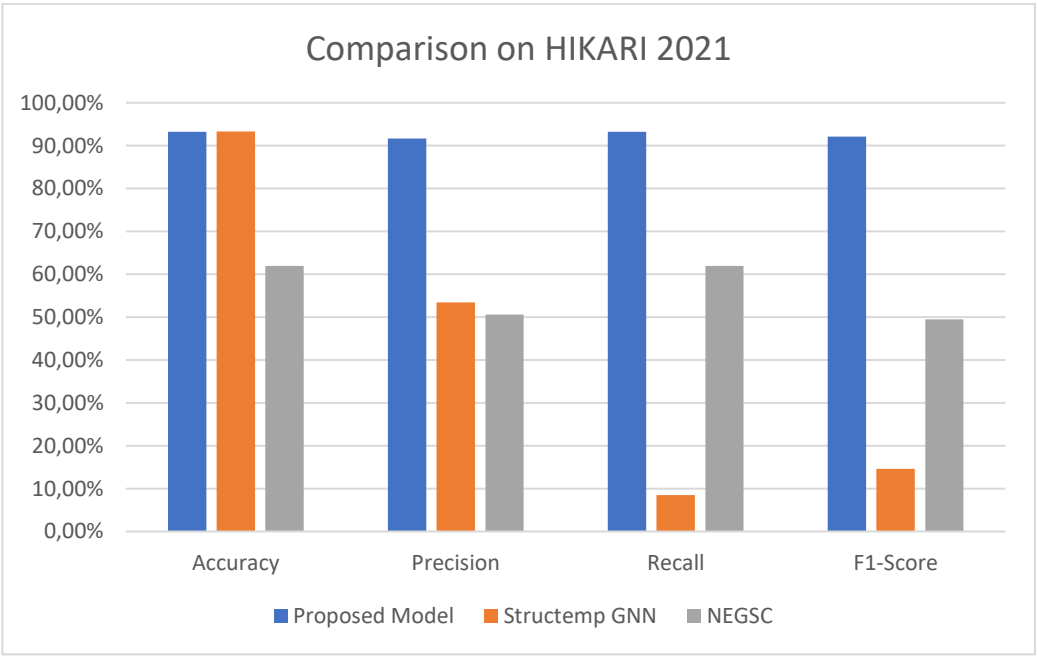
Gambar 4.20 Perbandingan pada NF-ToN-IoT-v3

Pada *dataset* NF-ToN-IoT-v3, model usulan kembali mencatatkan performa yang dominan dengan akurasi 97,39%, precision 97,37%, recall 97,39%, dan F1-score 97,34%, menunjukkan kestabilan performa meskipun pada data dengan karakteristik trafik khas IoT dan volume besar. *Structemp*-GNN memiliki precision 96,01% dan recall 90,01%, menghasilkan F1-score 92,91%, menandakan adanya penurunan deteksi yang berdampak pada keseluruhan efektivitas. Sementara itu, NEGSC hanya mencatatkan precision 72,23%, recall 83,88%, dan F1-score 77,01%, kembali menempati posisi terendah di semua metrik utama. Hasil ini memperkuat keunggulan model usulan dalam mempertahankan efisiensi dan efektivitas deteksi intrusi, bahkan pada *dataset* dengan trafik besar dan dinamika khas IoT.



Gambar 4.21 Perbandingan pada UNSW-NB15

Pada *dataset* UNSW-NB15, model usulan mencatatkan performa unggul dengan akurasi 99,27%, precision 99,31%, recall 99,27%, dan F1-score 99,29%, menunjukkan konsistensi tinggi dalam mengenali berbagai jenis serangan, baik klasik maupun modern. *Structemp*-GNN cukup kompetitif pada precision (95,17%) dan recall (95,36%), dengan F1-score 95,26%, namun tetap tertinggal dari model usulan. NEGSC berada di posisi ketiga dengan precision 85,91%, recall 92,69%, dan F1-score 89,17%, menunjukkan adanya ketidakseimbangan dalam deteksi yang berdampak pada menurunnya akurasi keseluruhan. Kinerja model usulan yang stabil dan tinggi pada *dataset* ini menunjukkan bahwa pendekatan semi-supervised yang digunakan mampu menangani skenario intrusi nyata dengan kompleksitas tinggi secara lebih andal daripada baseline lainnya.



Gambar 4.22 Perbandingan pada HIKARI 2021

Pada *dataset* HIKARI2021, model usulan tetap mempertahankan performa yang kuat dengan akurasi 93,26%, precision 91,69%, recall 93,26%, dan F1-score 92,11%, meskipun *dataset* ini memiliki karakteristik yang paling menantang, termasuk data terenkripsi dan

distribusi serangan yang tidak seimbang. Sebaliknya, *Structemp*-GNN mengalami penurunan drastis dengan recall hanya 8,47%, yang menyebabkan F1-score merosot ke 14,62%, meskipun precision-nya masih mencapai 53,41%. NEGSC tampil sedikit lebih baik dari *Structemp*-GNN dengan precision 50,60%, recall 61,92%, dan F1-score 49,45%, namun tetap jauh di bawah model usulan. Hasil ini menunjukkan bahwa hanya model semi-supervised yang mampu menjaga sensitivitas dan akurasi secara konsisten, bahkan ketika dihadapkan pada *dataset* dengan kompleksitas tinggi, serangan terenkripsi, dan pola data yang sangat bervariasi seperti HIKARI2021.

Secara keseluruhan, hasil perbandingan memperlihatkan bahwa pendekatan *semi-supervised* yang diintegrasikan dengan representasi struktural-temporal dan *autoencoder* memberikan keunggulan signifikan baik dari sisi akurasi deteksi maupun efisiensi operasional. Kombinasi arsitektur yang adaptif dan mekanisme pembelajaran berbasis representasi menjadikan model ini relevan untuk diterapkan pada sistem deteksi intrusi jaringan modern dengan kebutuhan respons cepat dan akurasi tinggi.

Tabel 4.6 Perbandingan dengan model lain

Model	*Dataset*	Accuracy	Precision	Recall	F1-Score	*Inference time*/snapshot (ms)	*Inference time*/flow (µs)
Proposed Model	NF-UQ-NIDS-v2	97.51%	99.34%	97.51%	97.53%	5.15	10.07
	NF-ToN-IoT-v2	99.51%	99.51%	99.51%	99.51%	5.93	11.6
	NF-UNSW-NB15-v2	99.71%	99.73%	99.71%	99.71%	6.12	11.97
	NF-ToN-IoT-v3	97.39%	97.37%	97.39%	97.34%	5.81	11.36
	UNSW-NB15	99.27%	99.31%	99.27%	99.29%	5.91	11.56
	HIKARI2021	93.26%	91.69%	93.26%	92.11%	5.29	10.34
Structemp GNN	NF-UQ-NIDS-v2	90.22%	97.75%	87.39%	92.28%	8.1	15.83
	NF-ToN-IoT-v2	96.69%	98.73%	94.52%	96.58%	3.2	6.27
	NF-UNSW-NB15-v2	99.73%	92.18%	97.36%	94.70%	2.5	4.9
	NF-ToN-IoT-v3	97.85%	96.01%	90.01%	92.91%	2.91	5.7
	UNSW-NB15	99.31%	95.17%	95.36%	95.26%	3.13	6.13
	HIKARI2021	93.27%	53.41%	8.47%	14.62%	2.41	4.73
NEGSC	NF-UQ-NIDS-v2	71.69%	63.60%	71.69%	66.10%	300.09	0.31
	NF-ToN-IoT-v2	83.88%	72.23%	83.88%	77.01%	1120.06	1.17

Model	*Dataset*	Accuracy	Precision	Recall	F1-Score	*Inference time*/snapshot (ms)	*Inference time*/flow (µs)
	NF-UNSW-NB15-v2	97.52%	95.10%	97.52 %	96.29%	245.73	0.26
	NF-ToN-IoT-v3	83.88%	72.23%	83.88 %	77.01%	1120.06	1.17
	UNSW-NB15	92.69%	85.91%	92.69 %	89.17%	211.09	0.22
	HIKARI2021	61.92%	50.60%	61.92 %	49.45%	214.26	0.64

Setelah dilakukan evaluasi terhadap enam *dataset* yang berbeda, model yang diusulkan menunjukkan keunggulan tidak hanya dalam akurasi deteksi, tetapi juga dalam efisiensi komputasi, khususnya dari sisi waktu inferensi. Waktu inferensi menjadi krusial dalam konteks implementasi *real-time*, di mana deteksi yang cepat dan responsif merupakan kebutuhan utama.

Model usulan secara konsisten mempertahankan waktu inferensi yang rendah di seluruh *dataset*, dengan rata-rata hanya 11,15 µs per flow. Nilai ini diperoleh dari waktu inferensi per *snapshot* antara 5,15 hingga 6,12 ms, dengan *snapshot* berisi 512 *flow*. Efisiensi ini menunjukkan bahwa model mampu memproses trafik dalam jumlah besar secara cepat, menjadikannya cocok untuk kebutuhan deteksi intrusi dengan latensi rendah.

Sebaliknya, NEGSC menunjukkan waktu inferensi per *snapshot* yang jauh lebih besar, seperti pada NF-ToN-IoT-v2 yang mencapai 1120,06 ms per *snapshot*. Sekilas, waktu per *flow-nya* tampak kecil (1,17 µs/flow), tetapi perlu dicatat bahwa satu *snapshot* pada NEGSC terdiri dari 35.000 *flow*, jauh lebih besar daripada model usulan. Ukuran *snapshot* yang besar ini justru menjadi hambatan dalam skenario real-time, karena *delay* total per unit data masuk sangat tinggi. Dengan kata lain, meskipun waktu per *flow* terlihat efisien, latensi respons model tetap tidak dapat diterima untuk sistem yang membutuhkan deteksi cepat dan berkelanjutan.

Model *Structemp*-GNN memiliki waktu inferensi per *snapshot* yang lebih cepat daripada model usulan pada beberapa *dataset*, seperti 2,41 ms (HIKARI2021) dan 2,5 ms (NF-UNSW-NB15-v2), atau sekitar 4,73–6,27 µs/flow. Namun, performanya tidak konsisten, terutama pada *dataset* dengan distribusi serangan yang berbeda dan fitur yang lebih kompleks. Pada HIKARI2021, misalnya, recall *Structemp*-GNN hanya mencapai 8,47%, menunjukkan kegagalan dalam mengenali mayoritas serangan.

Dengan mempertimbangkan aspek performa dan waktu inferensi secara bersamaan, model yang diusulkan menunjukkan *trade-off* yang paling seimbang: waktu inferensi rendah, performa deteksi tinggi dan konsisten, serta kemampuan generalisasi yang baik. NEGSC gagal memenuhi persyaratan latensi, sedangkan *Structemp*-GNN gagal menjaga sensitivitas. Oleh karena itu, dari perspektif efisiensi dan reliabilitas operasional, model usulan merupakan solusi paling layak untuk *deployment real-time* pada sistem deteksi intrusi jaringan.

BAB 5 Kesimpulan dan Saran

5.1 Kesimpulan

Penelitian ini diawali dengan tahap pengumpulan *dataset* publik yang telah diberi label, yaitu NF-UQ-NIDS-v2, NF-ToN-IoT-v2, NF-UNSW-NB15-v2, NF-ToN-IoT-v3, UNSW-NB15, dan HIKARI2021. Pemilihan keenam *dataset* ini dilakukan secara strategis untuk memastikan bahwa model diuji dalam berbagai konteks jaringan yang berbeda, termasuk lingkungan *enterprise*, infrastruktur *Internet of Things* (IoT), serta trafik terenkripsi dan realistis. *Dataset* NF-UQ-NIDS-v2 digunakan sebagai *dataset* utama dalam pelatihan dan validasi, sementara lima lainnya digunakan untuk menguji kemampuan generalisasi model. Keberagaman distribusi dan jenis serangan pada *dataset* ini memungkinkan sistem yang dikembangkan mampu menghadapi serangan zero-day dengan lebih baik.Hasil evaluasi juga menunjukkan bahwa model ini memiliki kemampuan generalisasi yang baik terhadap distribusi data yang baru dan tidak bergantung pada pola statis, menjadikannya lebih adaptif dalam menghadapi serangan yang tidak dikenal, atau dikenal sebagai zero-day attacks. Konfigurasi *hyperparameter* yang dioptimasi menggunakan framework Optuna memberikan hasil yang sangat efektif dalam meningkatkan kinerja model.

Setelah *dataset* terkumpul, data mentah yang diperoleh melalui masing-masing *dataset* diproses melalui tahap preprocessing yang mencakup pembersihan nilai kosong atau anomali, encoding atribut identitas seperti IP address dan port menjadi bentuk numerik, penghapusan kolom non-informatif, serta normalisasi nilai numerik. Selanjutnya, data diubah menjadi snapshot berbasis waktu menggunakan pendekatan sliding window. Tujuan dari snapshot generation ini adalah untuk memungkinkan pemodelan berbasis graf yang mempertimbangkan dinamika temporal dalam aliran trafik jaringan. Proses ini juga membantu mengurangi beban komputasi dengan membagi data ke dalam unit-unit waktu yang lebih kecil dan terstruktur.

Setelah proses *preprocessing* selesai, tahap pengembangan baseline *Structemp*-GNN dilakukan dengan mengacu pada arsitektur dari Boukari et al. Model baseline ini terdiri atas dua jenis *edge*, yaitu *edge* struktural dan *edge* temporal, untuk menangkap informasi spasial dan dinamika waktu antar komponen jaringan. Reimplementasi *baseline* ini dilakukan untuk memastikan perbandingan performa yang adil ketika model usulan diuji. Hasil dari baseline digunakan sebagai titik referensi untuk mengevaluasi kontribusi dari arsitektur yang dikembangkan dalam penelitian ini.

Kontribusi utama dalam penelitian ini adalah pengembangan GNN-AE (*Graph Neural Network Autoencoder*) yang dirancang untuk memperkaya representasi laten dari graf jaringan sebagai bagian dari arsitektur *semi-supervised learning* untuk deteksi intrusi. Pendekatan *dual-path encoding* diterapkan untuk memisahkan pemrosesan sisi struktural dan temporal dalam graf, dengan fusi *embedding* sebagai penggabungan kedua informasi tersebut. Representasi laten ini kemudian digunakan dalam proses *decoding* untuk memastikan fitur-fitur penting tetap terjaga selama pelatihan. *Reconstruction loss* yang dihasilkan berfungsi untuk memperkuat pembelajaran representasi, bukan sebagai indikator anomali secara langsung.

Komponen GNN-AE yang telah dibangun kemudian diintegrasikan ke dalam arsitektur *Structemp*-GNN dalam bentuk sistem *semi-supervised*. Proses integrasi ini mencakup penyusunan fungsi loss gabungan antara loss rekonstruksi dan loss klasifikasi, dengan penyesuaian koefisien β untuk menyeimbangkan fokus antara keduanya. Arsitektur ini

memungkinkan model untuk tidak hanya belajar dari data berlabel, tetapi juga dari distribusi laten data tak berlabel. Dengan demikian, sistem mampu memanfaatkan kekayaan struktur data jaringan secara lebih maksimal untuk menghasilkan representasi fitur yang lebih *robust*, mendukung deteksi intrusi termasuk pada pola-pola yang belum pernah terlihat sebelumnya.

Proses optimasi dilakukan menggunakan Optuna dengan pendekatan *Bayesian Optimization* untuk menemukan konfigurasi *hyperparameter* terbaik. Beberapa parameter seperti *learning rate*, dimensi hidden layer, dan nilai β pada *loss function* disesuaikan guna mencapai keseimbangan antara akurasi klasifikasi dan kualitas rekonstruksi. Optimasi ini menghasilkan stabilitas pelatihan yang lebih baik dan peningkatan performa model secara keseluruhan, serta mempercepat konvergensi pada proses pelatihan.

Setelah seluruh tahapan pengembangan dan optimasi selesai, dilakukan evaluasi menyeluruh terhadap performa model menggunakan enam *dataset* publik. Model usulan mencapai performa terbaik pada NF-UNSW-NB15-v2 dengan akurasi 99,71%, precision 99,73%, recall 99,71%, dan F1-score 99,71%. Performa tinggi juga dicapai pada *dataset* lain seperti NF-ToN-IoT-v2 (F1-score 99,51%) dan NF-UQ-NIDS-v2 (F1-score 97,53%). Bahkan pada *dataset* yang paling kompleks, yaitu HIKARI2021, model tetap mempertahankan F1-score sebesar 92,11%, jauh melampaui *Structemp*-GNN yang hanya mencapai 14,62% dan NEGSC yang mencatatkan 49,45% pada *dataset* yang sama.

Rata-rata waktu inferensi model usulan berkisar antara 5,15 hingga 6,12 milidetik per *snapshot*, yang terdiri dari 512 flow, sehingga menghasilkan waktu inferensi per flow antara 10,07 hingga 11,97 mikrodetik. *Structemp*-GNN mencatat waktu inferensi yang lebih cepat di beberapa kasus, seperti 2,41 ms pada HIKARI2021 dan 2,5 ms pada NF-UNSW-NB15-v2, tetapi dengan penurunan performa signifikan, terutama pada HIKARI2021 yang hanya mencatatkan recall 8,47% dan F1-score 14,62%.

NEGSC, di sisi lain, memiliki waktu inferensi per snapshot yang jauh lebih tinggi, yakni 245,73 ms pada NF-UNSW-NB15-v2 dan mencapai 1120,06 ms pada NF-ToN-IoT-v2. Meskipun setiap snapshot NEGSC mencakup 35.000 *flow*, dan waktu per *flow* terlihat kecil (sekitar 1,17 μs/flow), total *delay* per unit waktu tetap tidak memenuhi batas kebutuhan *real-time* karena ukuran *batch* yang besar dan kompleksitas arsitektur. Selain itu, F1-score NEGSC berkisar antara 49,45% hingga 96,29%, tetapi tidak menunjukkan konsistensi seperti model usulan.

5.2 Saran

Berdasarkan hasil penelitian ini, beberapa saran yang dapat dipertimbangkan untuk pengembangan lebih lanjut adalah:

- Pengujian pada *Dataset* Lain

Penelitian ini telah menggunakan enam *dataset*, namun masih terdapat banyak *dataset* lain dengan karakteristik trafik dan arsitektur jaringan yang berbeda. Pengujian lebih lanjut pada jaringan IoT yang lebih heterogen atau jaringan *enterprise* berskala besar dapat memberikan wawasan tambahan terkait kemampuan generalisasi dan ketahanan model dalam berbagai skenario.

- Perbaikan dalam Penanganan False Negatives

Meskipun nilai False Negative yang dihasilkan model relatif rendah (3,03%), pengurangan lebih lanjut tetap penting dalam konteks sistem keamanan. Strategi seperti peningkatan recall, penyesuaian threshold klasifikasi, atau integrasi teknik ensemble dapat dipertimbangkan untuk menekan kemungkinan lolosnya trafik berbahaya.

- Optimasi Waktu Inferensi

Waktu inferensi model saat ini sudah tergolong cepat dan layak untuk sistem real-time, namun pengoptimalan lebih lanjut terhadap efisiensi komputasi baik pada tingkat snapshot maupun per flow tetap disarankan, terutama jika model akan diterapkan pada perangkat dengan sumber daya terbatas.

- Implementasi di Lingkungan Nyata

Penerapan model dalam sistem jaringan operasional yang sebenarnya sangat dianjurkan untuk menguji ketahanan terhadap trafik dinamis dan anomali dunia nyata. Langkah ini juga dapat mengungkap tantangan baru terkait *latency*, throughput, serta integrasi dengan sistem keamanan yang sudah ada.

- Pengembangan ke *Multiclass Classification*

Model saat ini berfokus pada klasifikasi biner (benign vs anomali). Untuk meningkatkan relevansi terhadap kebutuhan praktis, disarankan agar penelitian selanjutnya mengembangkan model ke arah klasifikasi multi kelas, seperti mendeteksi jenis serangan secara spesifik (DDoS, Malware, Phishing, dan lain lain), sehingga sistem dapat memberikan informasi yang lebih detail dalam proses mitigasi. Dengan mengikuti saran-saran ini, model dapat ditingkatkan untuk lebih efektif dan efisien dalam deteksi intrusi jaringan di masa depan.

DAFTAR PUSTAKA

[1] Charles Henderson, "IBM X-Force Threat Intelligence Index 2024," IBM Security. Diakses: 7 Juli 2025. [Daring]. Tersedia pada: https://www.ibm.com/think/x-force/2024-x-force-threat-intelligence-index

[2] Research Check Point, "Q1 2025 Global Cyber Attack Report: An almost 50% surge in cyber threats worldwide with a rise of 126% in ransomware attacks," Check Point. [Daring]. Tersedia pada: https://blog.checkpoint.com/research/q1-2025-global-cyber-attack-report-from-check-point-software-an-almost-50-surge-in-cyber-threats-worldwide-with-a-rise-of-126-in-ransomware-attacks/

[3] Federal Bureau of Investigation, "2024 IC3 Annual Report," Washington, D.C., 2024. Diakses: 7 Juli 2025. [Daring]. Tersedia pada: https://www.ic3.gov/Media/PDF/AnnualReport/2024_IC3Report.pdf

[4] Jacob Fox, "Top Cybersecurity Statistics for 2025," Cobalt. Diakses: 7 Maret 2025. [Daring]. Tersedia pada: https://www.cobalt.io/blog/top-cybersecurity-statistics-2025

[5] T. Yi, X. Chen, Y. Zhu, W. Ge, dan Z. Han, "Review on the application of deep learning in network attack detection," 1 Maret 2023, *Academic Press*. doi: 10.1016/j.jnca.2022.103580.

[6] O. H. Abdulganiyu, T. Ait Tchakoucht, dan Y. K. Saheed, "A systematic literature review for network intrusion detection system (IDS)," *Int J Inf Secur*, vol. 22, no. 5, hlm. 1125–1162, Okt 2023, doi: 10.1007/s10207-023-00682-2.

[7] T. Bilot, N. El Madhoun, K. Al Agha, dan A. Zouaoui, "Graph Neural Networks for Intrusion Detection: A Survey," *IEEE Access*, vol. 11, hlm. 49114–49139, 2023, doi: 10.1109/ACCESS.2023.3275789.

[8] A. Venturi, M. Ferrari, M. Marchetti, dan M. Colajanni, "ARGANIDS: A novel Network Intrusion Detection System based on adversarially Regularized Graph Autoencoder," dalam *Proceedings of the ACM Symposium on Applied Computing*, Association for Computing Machinery, Mar 2023, hlm. 1540–1548. doi: 10.1145/3555776.3577651.

[9] I. E. Boukari, I. A. Derdouha, S. Bouzefrane, L. Hamdad, S. Nait-Bahloul, dan T. Huraux, "*Structemp*-GNN: An Intrusion Detection Framework in IoT Networks Using Dynamic Heterogeneous Graph Neural Networks," dalam *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, Springer Science and Business Media Deutschland GmbH, 2024, hlm. 17–39. doi: 10.1007/978-3-031-52426-4_2.

[10] N. O. Miracle, "The Importance of Network Security in Protecting Sensitive Data and Information", doi: 10.51584/IJRIAS.

[11] O. Mitchell dan C. Osazuwa, "Confidentiality, Integrity, and Availability in Network Systems: A Review of Related Literature," 2023.

[12] N. A. Azeez, T. M. Bada, S. Misra, A. Adewumi, C. Van der Vyver, dan R. Ahuja, "Intrusion Detection and Prevention Systems: An Updated Review," dalam *Advances in*

Intelligent Systems and Computing, Springer, 2020, hlm. 685–696. doi: 10.1007/978-981-32-9949-8_48.

[13] P. K. Keserwani, M. C. Govil, dan E. S. Pilli, "An effective NIDS framework based on a comprehensive survey of feature optimization and classification techniques," *Neural Comput Appl*, vol. 35, no. 7, hlm. 4993–5013, Mar 2023, doi: 10.1007/s00521-021-06093-5.

[14] Y. Li dan Q. Liu, "A comprehensive review study of cyber-attacks and cyber security; Emerging trends and recent developments," *Energy Reports*, vol. 7, hlm. 8176–8186, Nov 2021, doi: 10.1016/j.egyr.2021.08.126.

[15] A. I. Jony dan S. A. Hamim, "Navigating the Cyber Threat Landscape: A Comprehensive Analysis of Attacks and Security in the Digital Age," *Journal of Information Technology and Cyber Security*, vol. 1, no. 2, hlm. 53–67, Des 2024, doi: 10.30996/jitcs.9715.

[16] S. Kumar, M. Mahajan, dan S. Batra, "A Recent Study of Machine Learning Based Techniques for the Detection of Cyber-Attacks on Web Applications," dalam *Proceedings of International Conference on Contemporary Computing and Informatics, IC3I 2023*, Institute of Electrical and Electronics Engineers Inc., 2023, hlm. 153–158. doi: 10.1109/IC3I59117.2023.10397832.

[17] Ö. Aslan, S. S. Aktuğ, M. Ozkan-Okay, A. A. Yilmaz, dan E. Akin, "A Comprehensive Review of Cyber Security Vulnerabilities, Threats, Attacks, and Solutions," 1 Maret 2023, *MDPI*. doi: 10.3390/electronics12061333.

[18] R. Xu, G. Wu, W. Wang, X. Gao, A. He, dan Z. Zhang, "Applying self-supervised learning to network intrusion detection for network flows with graph neural network," *Computer Networks*, vol. 248, Jun 2024, doi: 10.1016/j.comnet.2024.110495.

[19] J. Zhou *dkk.*, "Graph neural networks: A review of methods and applications," 1 Januari 2020, *Elsevier B.V.* doi: 10.1016/j.aiopen.2021.01.001.

[20] X. Yang, Z. Song, I. King, dan Z. Xu, "A Survey on Deep Semi-Supervised Learning," *IEEE Trans Knowl Data Eng*, vol. 35, no. 9, hlm. 8934–8954, Sep 2023, doi: 10.1109/TKDE.2022.3220219.

[21] Z. Wu, S. Pan, F. Chen, G. Long, C. Zhang, dan P. S. Yu, "A Comprehensive Survey on Graph Neural Networks," *IEEE Trans Neural Netw Learn Syst*, vol. 32, no. 1, hlm. 4–24, Jan 2021, doi: 10.1109/TNNLS.2020.2978386.

[22] N. A. Asif *dkk.*, "Graph Neural Network: A Comprehensive Review on Non-Euclidean Space," 2021, *Institute of Electrical and Electronics Engineers Inc.* doi: 10.1109/ACCESS.2021.3071274.

[23] G. Lachaud, P. Conde-Cespedes, dan M. Trocan, "Mathematical Expressiveness of Graph Neural Networks," *Mathematics*, vol. 10, no. 24, Des 2022, doi: 10.3390/math10244770.

[24] A. G. Vrahatis, K. Lazaros, dan S. Kotsiantis, "Graph Attention Networks: A Comprehensive Review of Methods and Applications," *Future Internet*, vol. 16, no. 9, hlm. 318, Sep 2024, doi: 10.3390/fi16090318.

[25] X. Xu, X. Zhao, M. Wei, dan Z. Li, "A comprehensive review of graph convolutional networks: approaches and applications," 2023, *American Mathematical Society*. doi: 10.3934/ERA.2023213.

[26] B. H. Kim dan J. C. Ye, "Understanding Graph Isomorphism Network for rs-fMRI Functional Connectivity Analysis," *Front Neurosci*, vol. 14, Jun 2020, doi: 10.3389/fnins.2020.00630.

[27] D. H. Tran dan M. Park, "FN-GNN: A Novel Graph Embedding Approach for Enhancing Graph Neural Networks in Network Intrusion Detection Systems," *Applied Sciences (Switzerland)*, vol. 14, no. 16, Agu 2024, doi: 10.3390/app14166932.

[28] G. Duan, H. Lv, H. Wang, dan G. Feng, "Application of a Dynamic Line Graph Neural Network for Intrusion Detection With Semisupervised Learning," *IEEE Transactions on Information Forensics and Security*, vol. 18, hlm. 699–714, 2023, doi: 10.1109/TIFS.2022.3228493.

[29] W. L. Hamilton, R. Ying, dan J. Leskovec, "Inductive Representation Learning on Large Graphs," Jun 2017, [Daring]. Tersedia pada: http://arxiv.org/abs/1706.02216

[30] K. Berahmand, F. Daneshfar, E. S. Salehi, Y. Li, dan Y. Xu, "Autoencoders and their applications in machine learning: a survey," *Artif Intell Rev*, vol. 57, no. 2, Feb 2024, doi: 10.1007/s10462-023-10662-6.

[31] S. Chen dan W. Guo, "Auto-Encoders in Deep Learning—A Review with New Perspectives," 1 April 2023, *MDPI*. doi: 10.3390/math11081777.

[32] I. D. Mienye dan T. G. Swart, "Deep Autoencoder Neural Networks: A Comprehensive Review and New Perspectives," 2025, *Springer Science and Business Media B.V.* doi: 10.1007/s11831-025-10260-5.

[33] H. Bourlard dan S. H. Kabil, "Autoencoders reloaded," *Biol Cybern*, vol. 116, no. 4, hlm. 389–406, Agu 2022, doi: 10.1007/s00422-022-00937-6.

[34] A. Paszke *dkk.*, "PyTorch: An Imperative Style, High-Performance Deep Learning Library," 2019.

[35] M. Ring, S. Wunderlich, D. Scheuring, D. Landes, dan A. Hotho, "A Survey of Network-based Intrusion Detection Data Sets," Mar 2019, doi: 10.1016/j.cose.2019.06.005.

[36] M. Sarhan, S. Layeghy, dan M. Portmann, "Towards a Standard Feature Set for Network Intrusion Detection System *Dataset*s," *Mobile Networks and Applications*, vol. 27, no. 1, hlm. 357–370, Feb 2022, doi: 10.1007/s11036-021-01843-0.

[37] M. Luay, S. Layeghy, S. Hosseininoorbin, M. Sarhan, N. Moustafa, dan M. Portmann, "Temporal Analysis of NetFlow *Dataset*s for Network Intrusion Detection Systems," Mar 2025, [Daring]. Tersedia pada: http://arxiv.org/abs/2503.04404

[38] N. Moustafa dan J. Slay, "UNSW-NB15: A Comprehensive Data set for Network Intrusion Detection systems (UNSW-NB15 Network Data Set)." doi: 10.1109/MilCIS.2015.7348942.

[39] R. Fernandes dan N. Lopes, “Network Intrusion Detection Packet Classification with the HIKARI-2021 *Dataset*: a study on ML Algorithms,” dalam *10th International Symposium on Digital Forensics and Security, ISDFS 2022*, Institute of Electrical and Electronics Engineers Inc., 2022. doi: 10.1109/ISDFS55398.2022.9800807.

[40] F. Hutter, L. Kotthoff, dan J. Vanschoren, “The Springer Series on Challenges in Machine Learning Automated Machine Learning Methods, Systems, Challenges.” [Daring]. Tersedia pada: http://www.springer.com/series/15602

[41] R. M. Schmidt, F. Schneider, dan P. Hennig, “Descending through a Crowded Valley - Benchmarking Deep Learning Optimizers,” Jul 2020, [Daring]. Tersedia pada: http://arxiv.org/abs/2007.01547

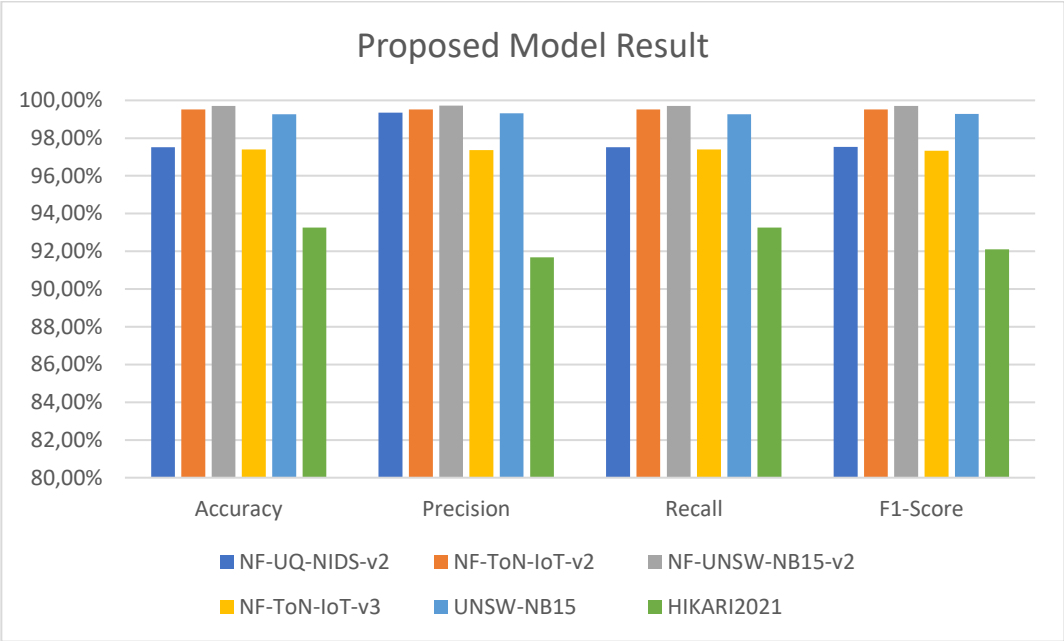
[42] X. Chen *dkk.*, “Symbolic Discovery of Optimization Algorithms,” Feb 2023, [Daring]. Tersedia pada: http://arxiv.org/abs/2302.06675

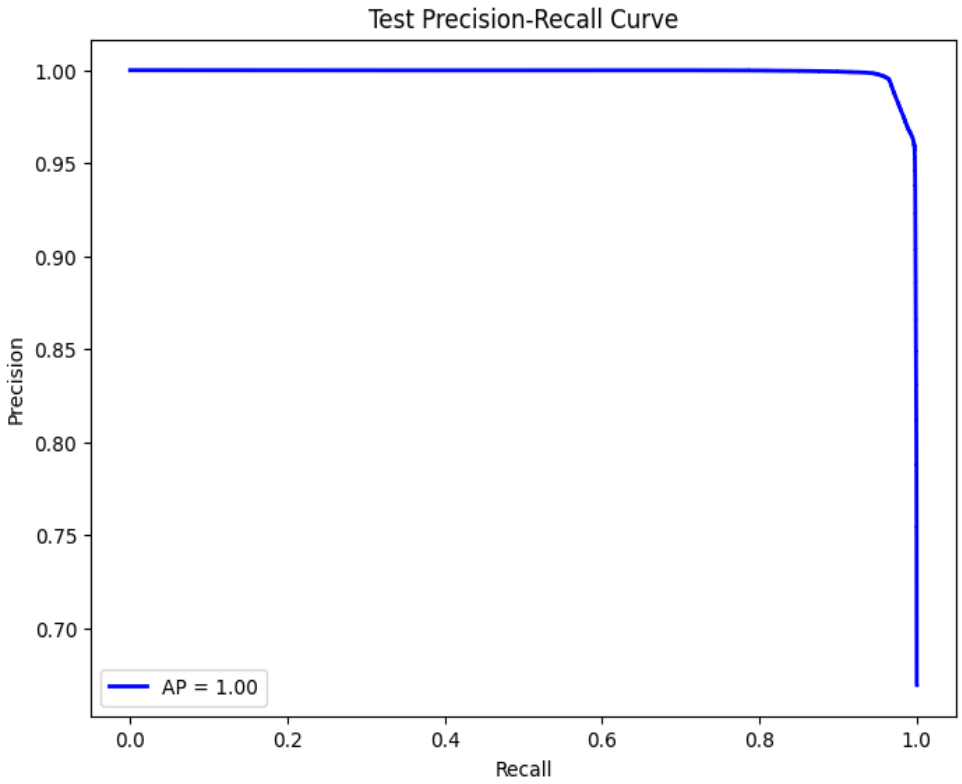
[43] I. Loshchilov dan F. Hutter, “Decoupled Weight Decay Regularization,” Jan 2019, [Daring]. Tersedia pada: http://arxiv.org/abs/1711.05101

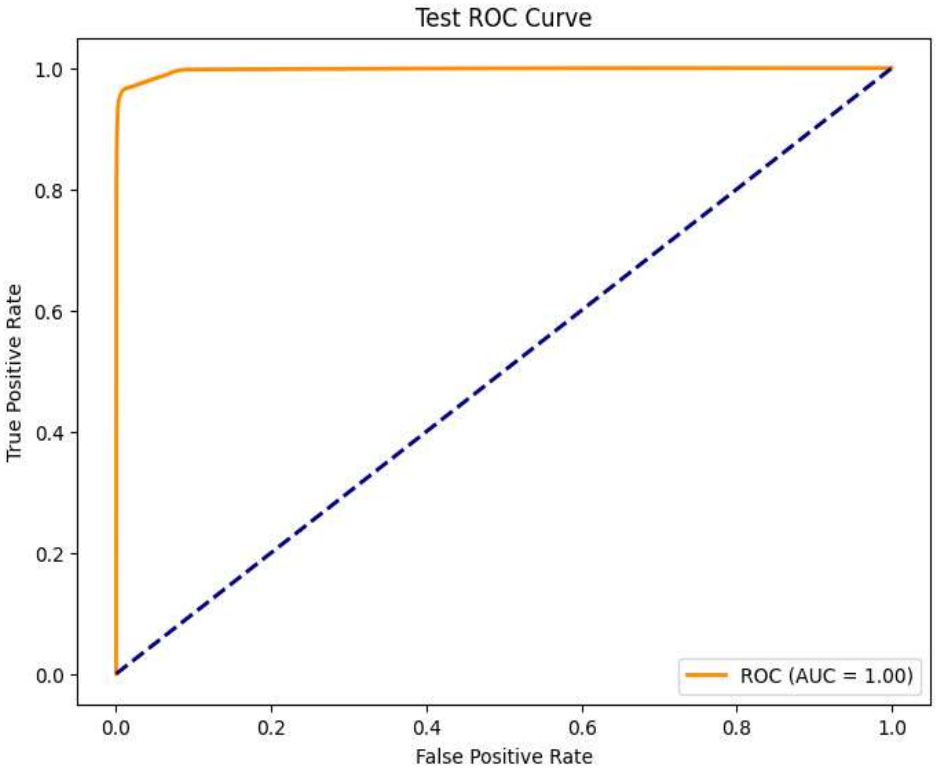
[44] L. Liu *dkk.*, “On the Variance of the Adaptive Learning Rate and Beyond,” Agu 2019, [Daring]. Tersedia pada: http://arxiv.org/abs/1908.03265

[45] S. R. Dubey, S. K. Singh, dan B. B. Chaudhuri, “Activation Functions in Deep Learning: A Comprehensive Survey and Benchmark,” Sep 2021, [Daring]. Tersedia pada: http://arxiv.org/abs/2109.14545

LAMPIRAN


Proposed Model Result
100,00%
98,00%
96,00%
94,00%
92,00%
90,00%
88,00%
86,00%
84,00%
82,00%
80,00%
Accuracy
Precision
Recall
F1-Score
NF-UQ-NIDS-v2
NF-ToN-IoT-v2
NF-UNSW-NB15-v2
NF-ToN-IoT-v3
UNSW-NB15
HIKARI2021


Test Precision-Recall Curve
Precision
1.00
0.95
0.90
0.85
0.80
0.75
0.70
AP = 1.00
0.0
0.2
0.4
0.6
0.8
1.0
Recall


Test ROC Curve
True Positive Rate
False Positive Rate
0.0
0.2
0.4
0.6
0.8
1.0
ROC (AUC = 1.00)

BIODATA PENULIS

Photo closed-up



Penulis lahir di Jakarta pada 28 Juni 2003 sebagai anak pertama dari dua bersaudara. Pendidikan dasar ditempuh di SDS Kartika Jakarta, kemudian melanjutkan jenjang SMP dan SMA di *Jakarta Islamic School* (JISC). Pada tahun 2021, Penulis diterima melalui jalur Seleksi Mandiri di Departemen Teknik Elektro, Fakultas Teknologi Elektro dan Informatika Cerdas (FTEIC), Institut Teknologi Sepuluh Nopember (ITS) Surabaya, dengan NRP 5022211233.

Selama menjalani studi di Departemen Teknik Elektro ITS, Penulis berperan sebagai asisten laboratorium di Laboratorium Jaringan Telekomunikasi, di mana Penulis membantu pelaksanaan praktikum serta kegiatan teknis yang berkaitan dengan bidang telekomunikasi.